

Data pipelines & management for visualization

CS424: Visualization & Visual Analytics

Fabio Miranda

<https://fmiranda.me>

Why data pipelines & management in a visualization class?



Interactivity depends on
latency + throughput

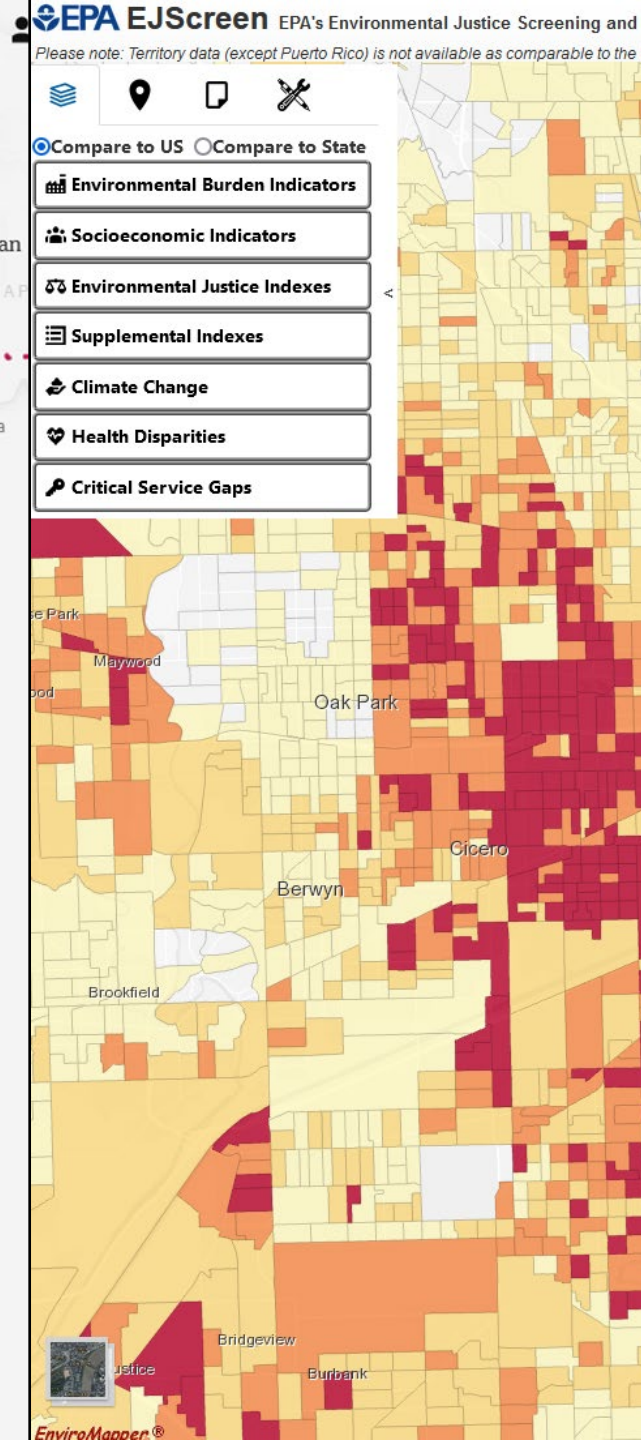
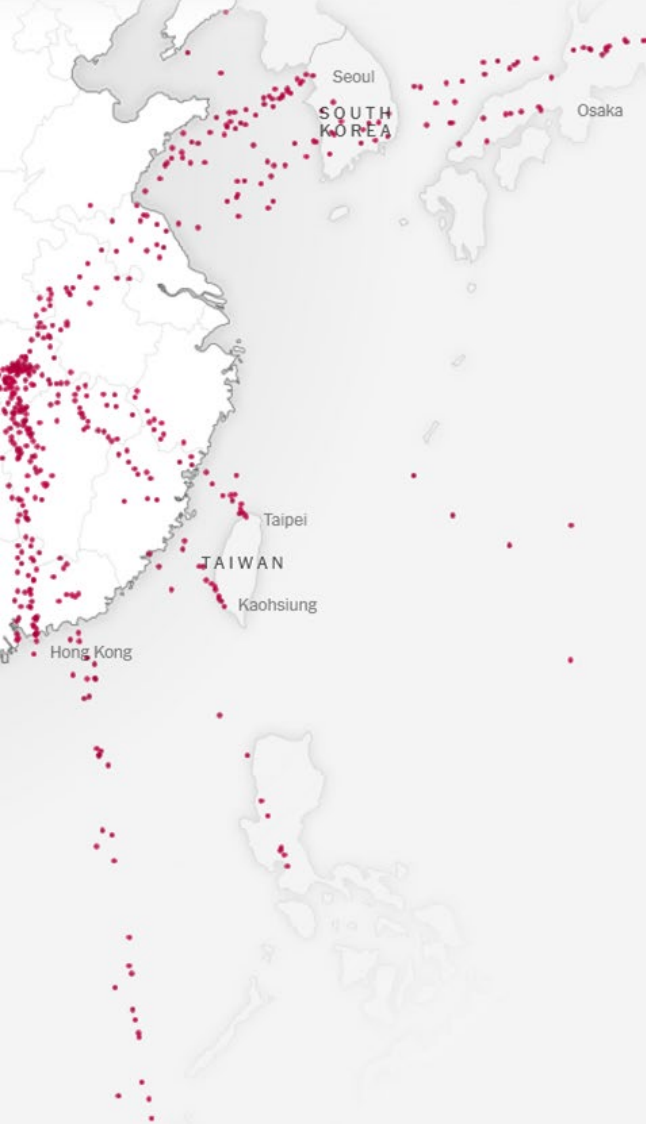


Scaling:
notebook → dashboard → system
requires **architecture**

How the Virus Got Out

By Jin Wu, Weiyei Cai, Derek Watkins and James Glanz March 22, 2020

The most extensive travel restrictions to stop an outbreak in human history haven't been enough. We analyzed the movements of hundreds of millions of people to show why.



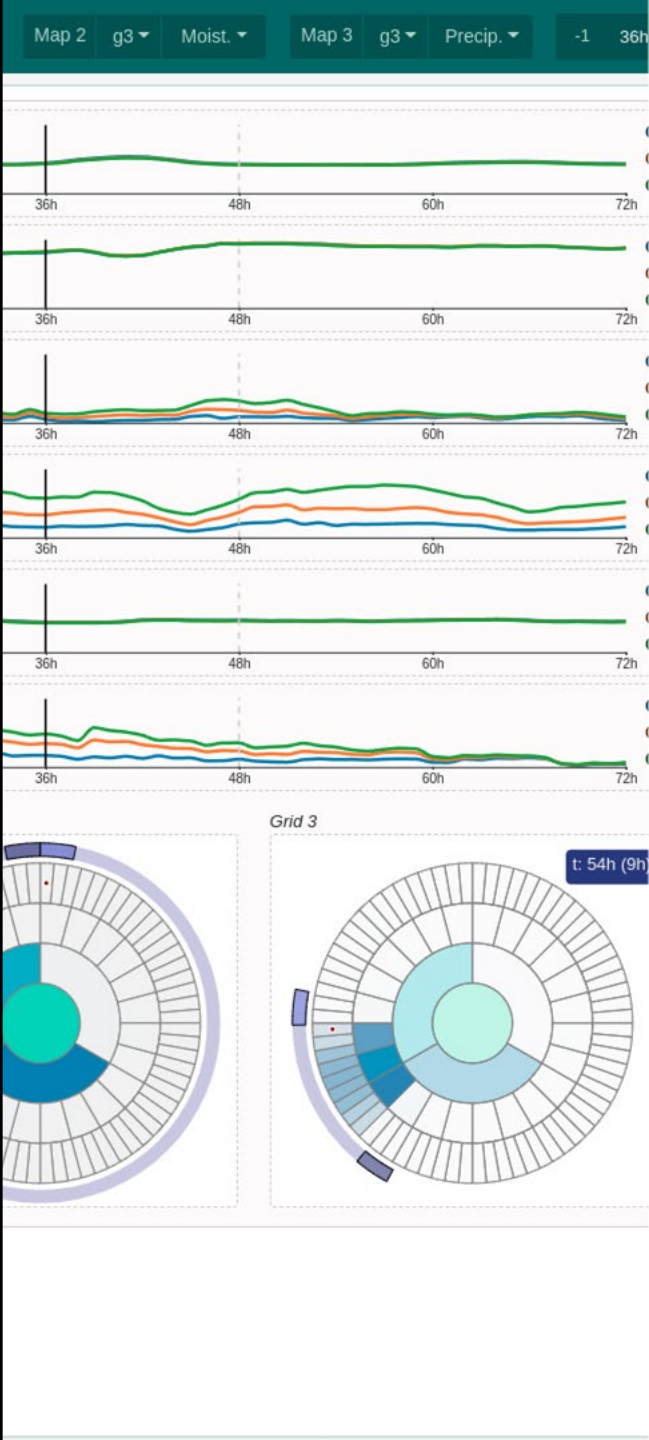
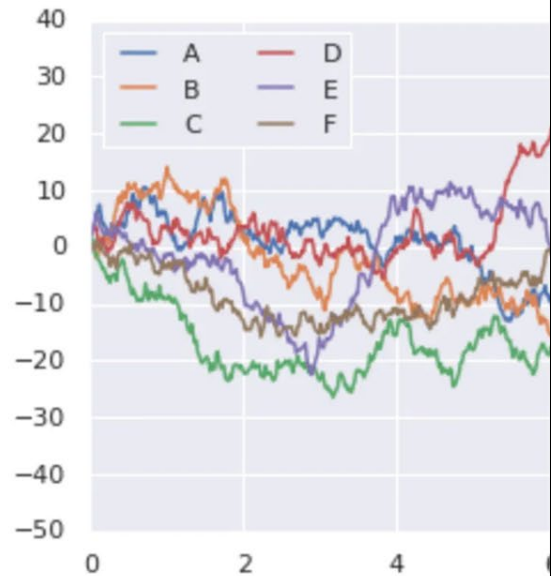
```
[5]: import matplotlib.pyplot as plt
plt.style.use('classic')
%matplotlib inline
import numpy as np
import pandas as pd
import seaborn as sns
sns.set()
```

```
[6]: rng = np.random.RandomState(0)
x = np.linspace(0, 10, 500)
y = np.cumsum(rng.randn(500, 6))
```

Next step

Now, create a graph.

```
[7]: plt.plot(x, y)
plt.legend('ABCDEF', ncol=2, loc='upper left')
```

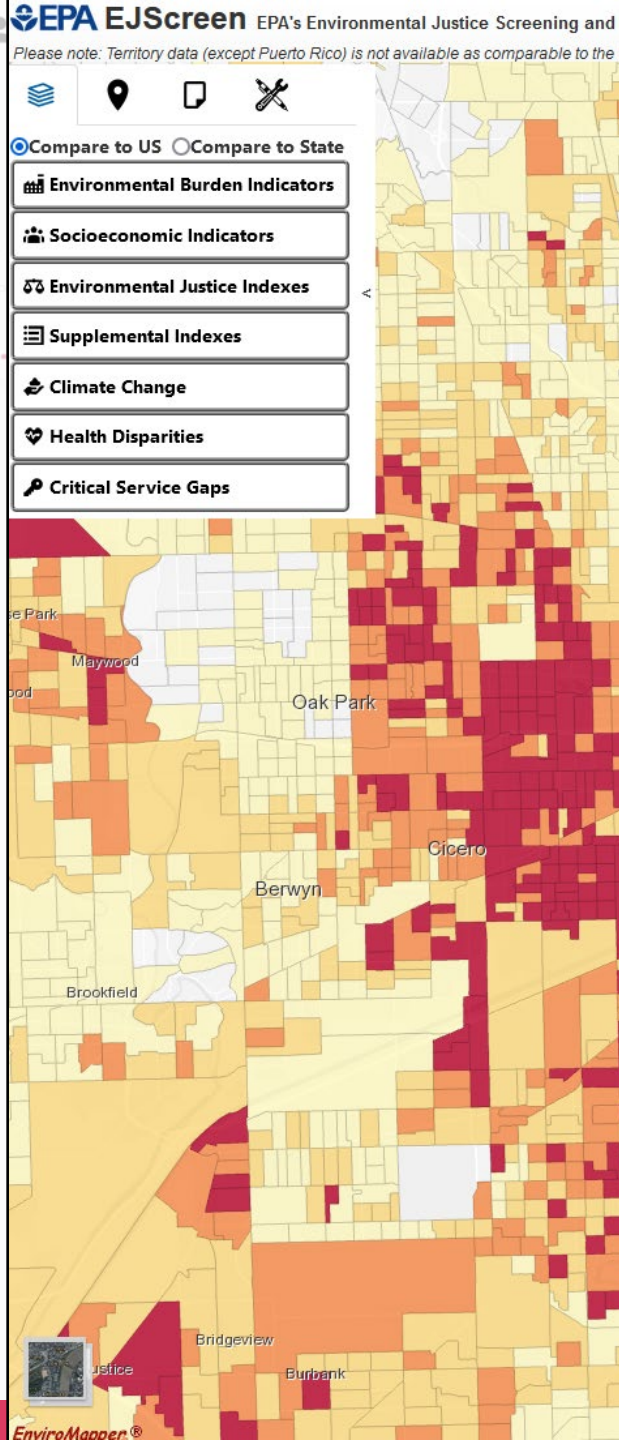


How the Virus Got Out

By Jin Wu, Weiyei Cai, Derek Watkins and James Glanz March 22, 2020

The most extensive travel restrictions to stop an outbreak in human history haven't been enough. We analyzed the movements of hundreds of millions of people to show why.

Data journalism



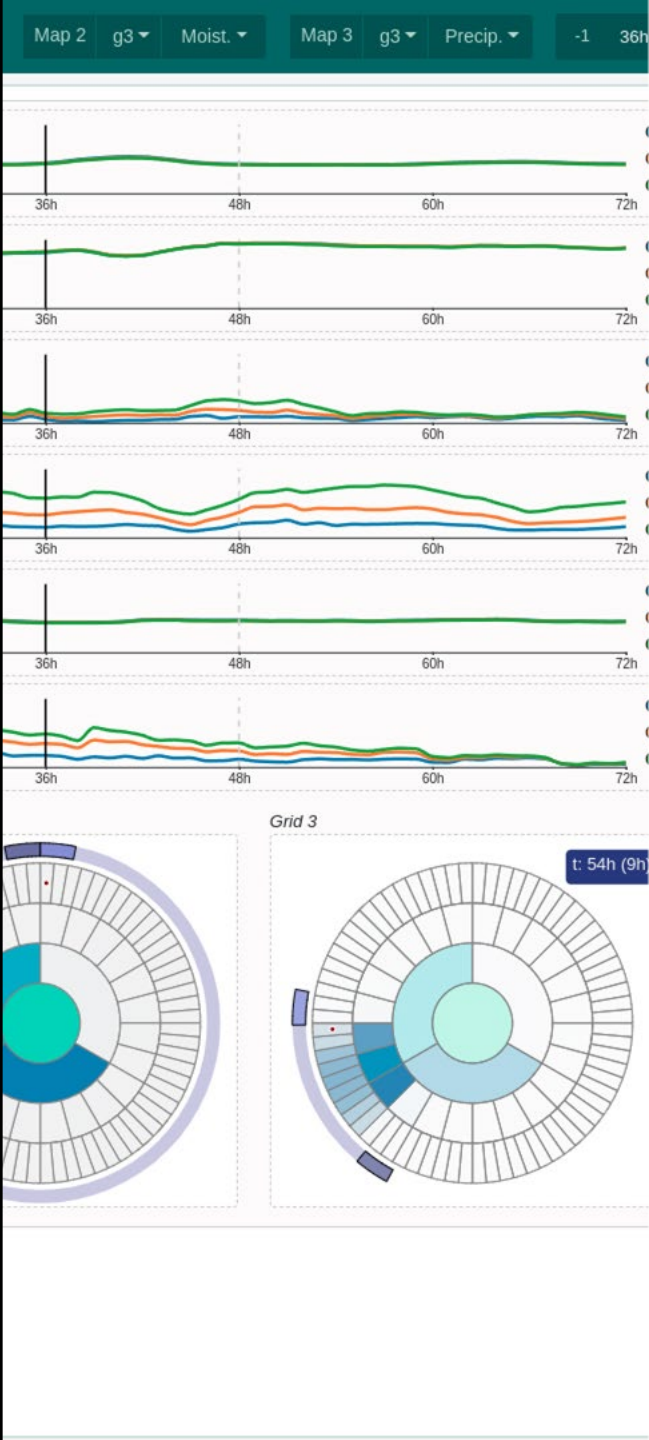
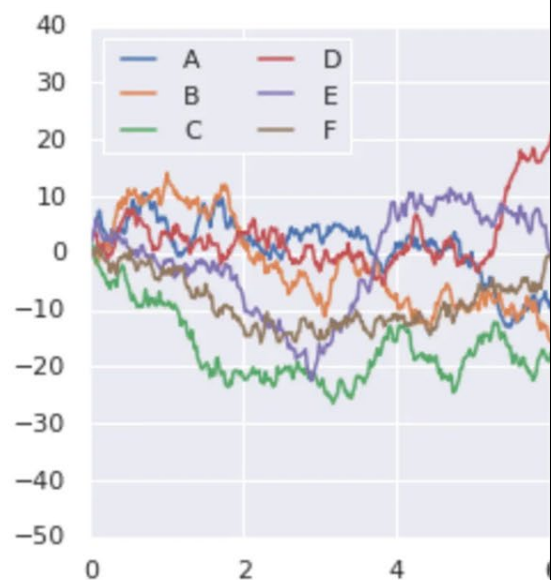
```
[5]: import matplotlib.pyplot as plt
plt.style.use('classic')
%matplotlib inline
import numpy as np
import pandas as pd
import seaborn as sns
sns.set()
```

```
[6]: rng = np.random.RandomState(0)
x = np.linspace(0, 10, 500)
y = np.cumsum(rng.randn(500, 6))
```

Next step

Now, create a graph.

```
[7]: plt.plot(x, y)
plt.legend('ABCDEF', ncol=2, loc='upper left')
```



The New York Times

How the Virus Got Out

By Jin Wu, Weiyi Cai, Derek Watkins and James Glanz March 22, 2020

The most extensive travel restrictions to stop an outbreak in human history haven't been enough. We analyzed the movements of hundreds of millions of people to show why.

Data journalism

EPA EJScreen

EPA's Environmental Justice Screening and Mapping Tool

Please note: Territory data (except Puerto Rico) is not available as comparable to the United States.

Compare to US

Compare to State

Environmental Burden Indicators

Socioeconomic Indicators

Environmental Justice Indexes

Supplemental Indexes

Climate Change

Health Disparities

Critical Service Gaps

EnviroMapper

[5]:

```
import matplotlib.pyplot as plt
plt.style.use('classic')
%matplotlib inline
import numpy as np
import pandas as pd
import seaborn as sns
sns.set()
```

[6]:

```
rng = np.random.RandomState(0)
x = np.linspace(0, 10, 500)
y = np.cumsum(rng.randn(500, 6))
```

Next step

Now, create a graph.

[7]:

```
plt.plot(x, y)
plt.legend('ABCDEF', ncol=2, loc='upper left')
```

Map 2

g3

Moist

Map 3

g3

Precip

-1

36h

Grid 3

t: 54h (9h)

The New York Times

How the Virus Got Out

By Jin Wu, Weiyi Cai, Derek Watkins and James Glanz March 22, 2020

The most extensive travel restrictions to stop an outbreak in human history haven't been enough. We analyzed the movements of hundreds of millions of people to show why.



Data journalism

EPA EJScreen

EPA's Environmental Justice Screening and Mapping Tool

Please note: Territory data (except Puerto Rico) is not available as comparable to the United States.

Compare to US

Compare to State

Environmental Burden Indicators

Socioeconomic Indicators

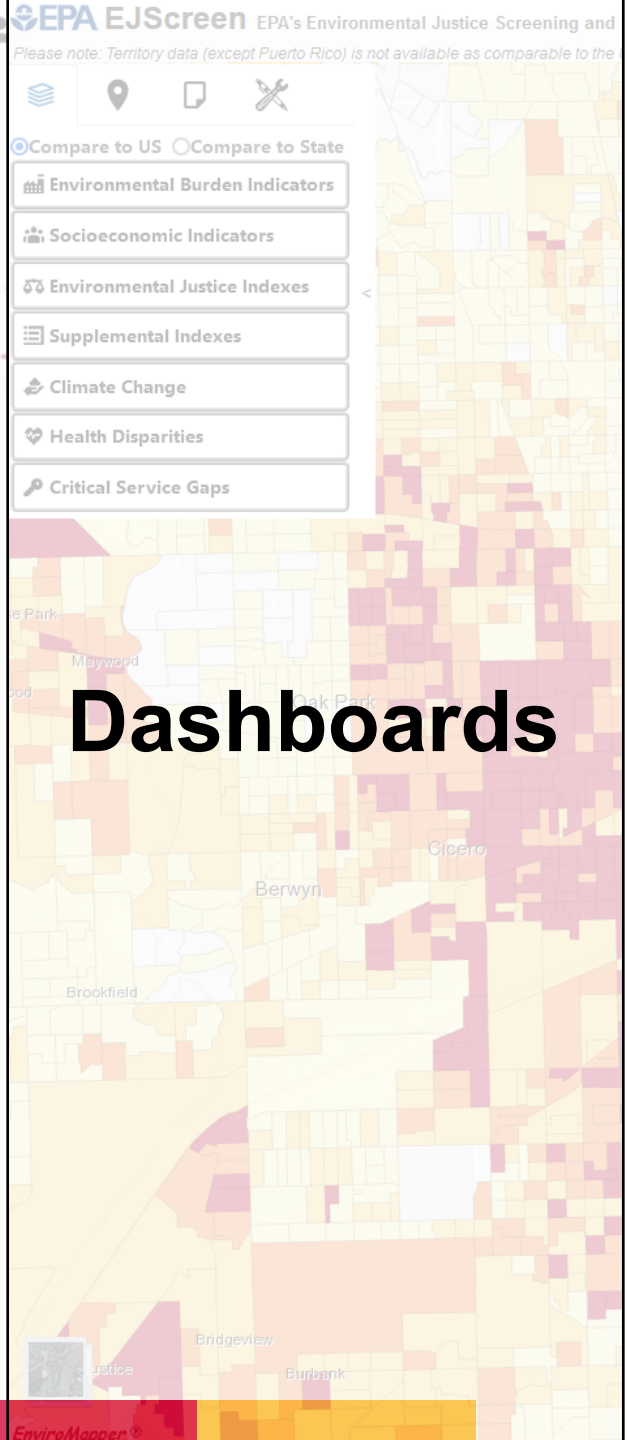
Environmental Justice Indexes

Supplemental Indexes

Climate Change

Health Disparities

Critical Service Gaps



Dashboards

[5]:

```
import matplotlib.pyplot as plt
plt.style.use('classic')
%matplotlib inline
import numpy as np
import pandas as pd
import seaborn as sns
sns.set()
```

[6]:

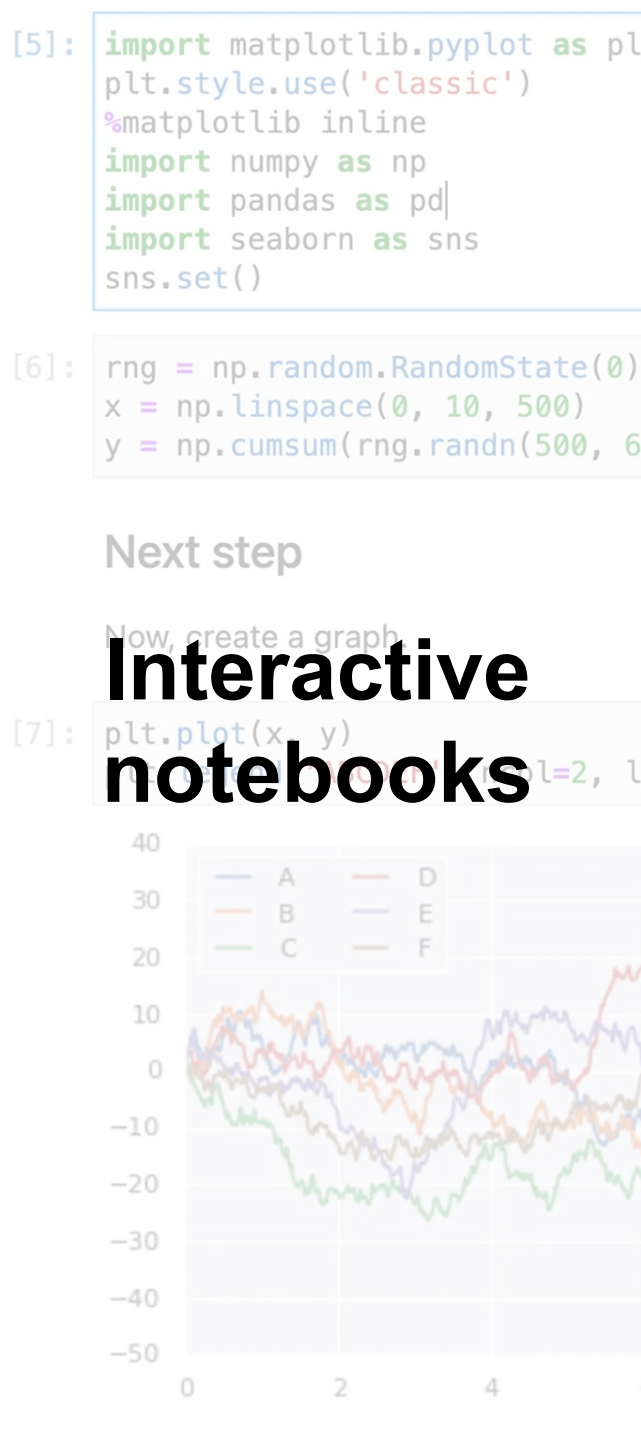
```
rng = np.random.RandomState(0)
x = np.linspace(0, 10, 500)
y = np.cumsum(rng.randn(500, 6))
```

Next step

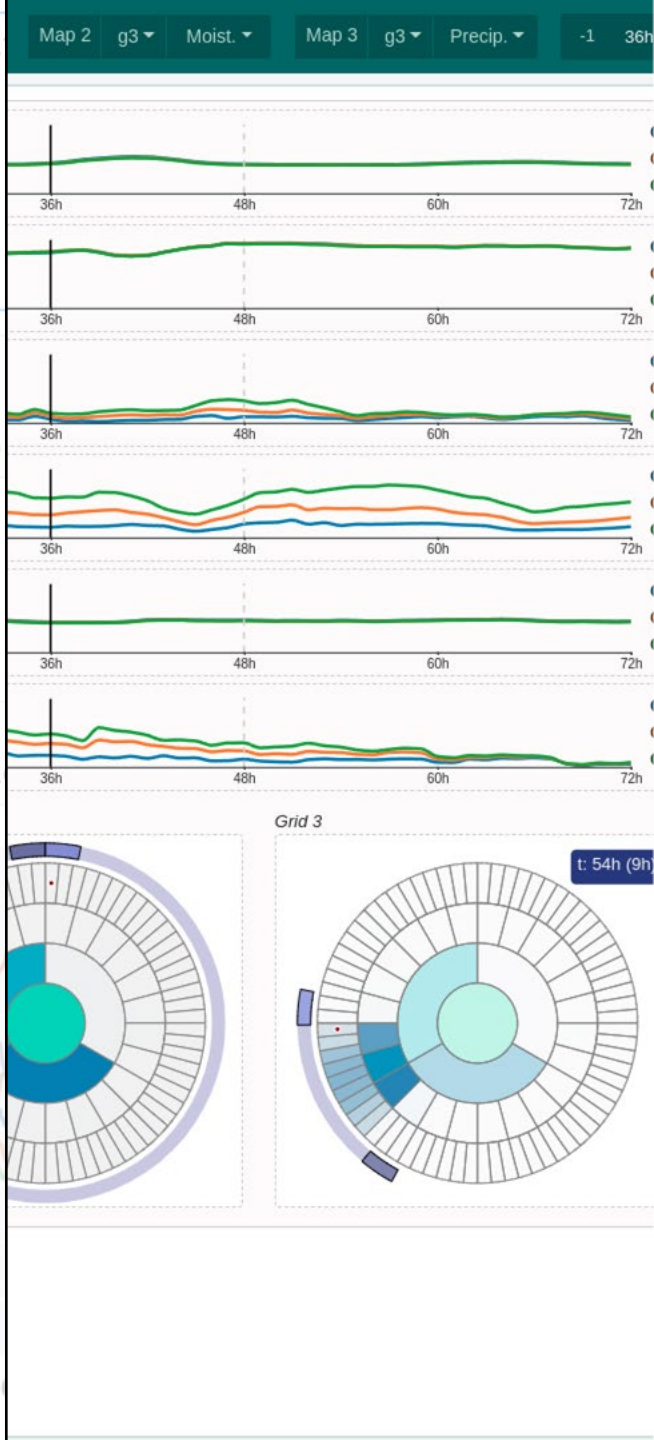
Now, create a graph

[7]:

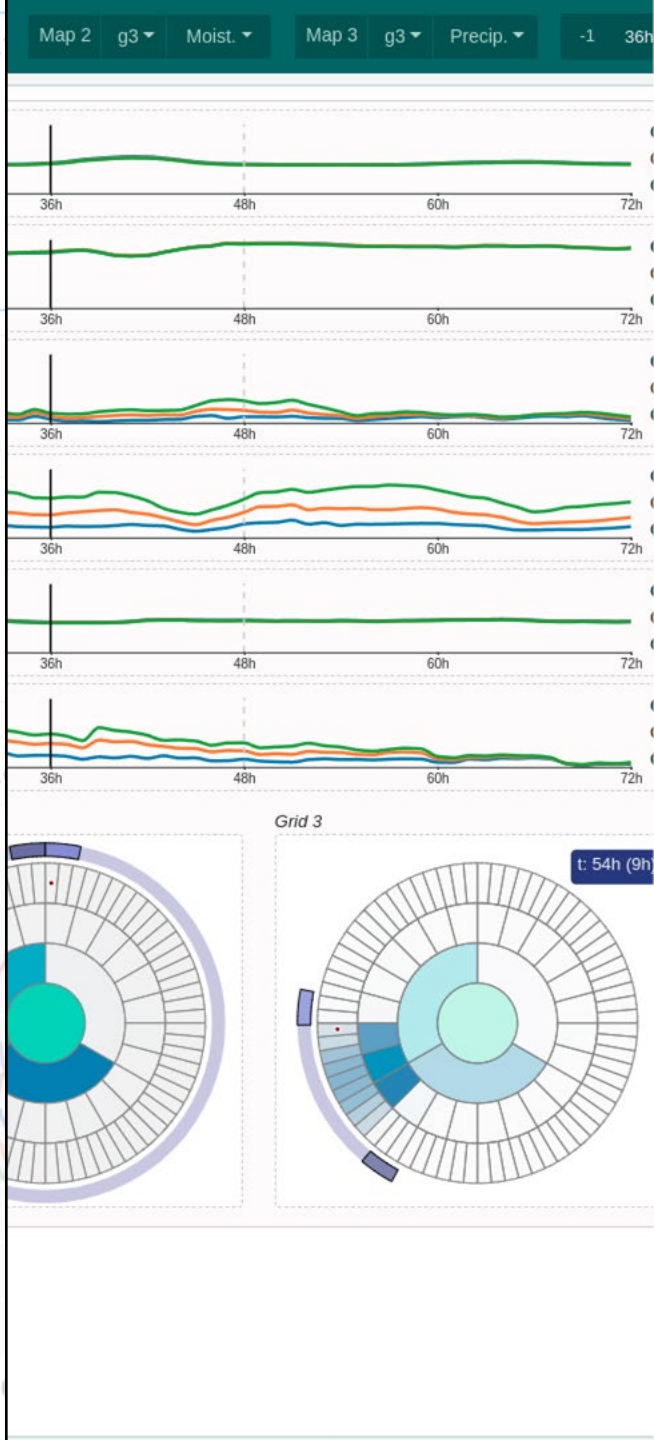
```
plt.plot(x, y)
```



Map 2 g3 Moist. Map 3 g3 Precip. -1 36h



Grid 3



The New York Times

How the Virus Got Out

By Jin Wu, Weiyi Cai, Derek Watkins and James Glanz March 22, 2020

The most extensive travel restrictions to stop an outbreak in human history haven't been enough. We analyzed the movements of hundreds of millions of people to show why.



A map showing travel movements between East Asia and North America. Red dots represent travel paths from locations like Seoul, Osaka, Taipei, and Kaohsiung to Hong Kong and other regions. The map is titled 'How the Virus Got Out' and is from The New York Times, dated March 22, 2020.

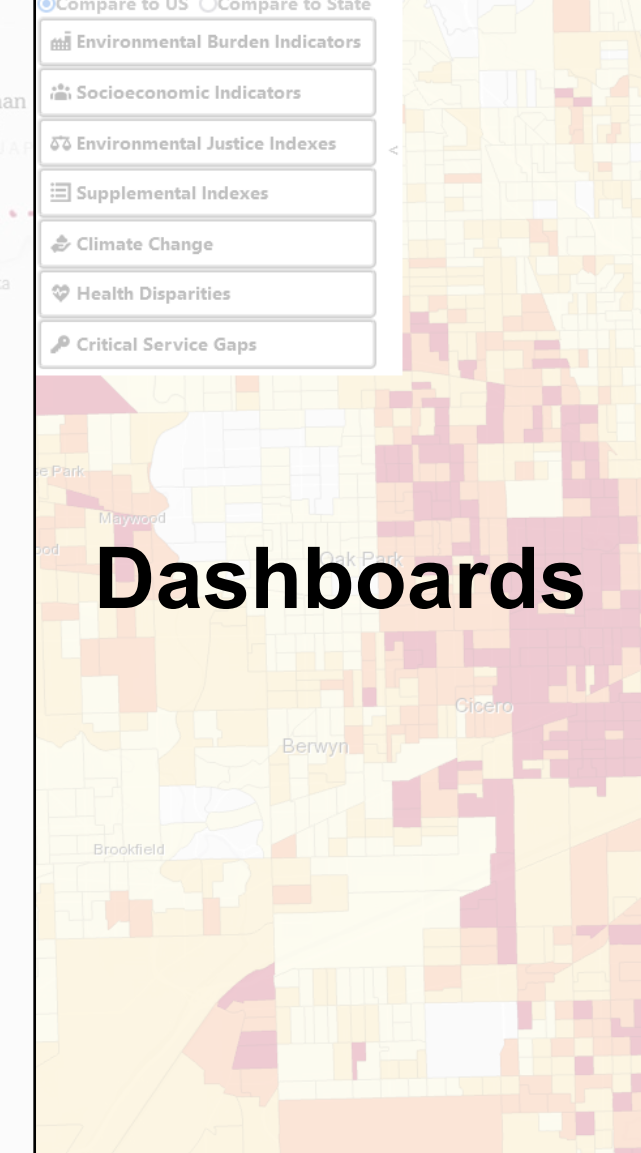
Data journalism

EPA EJScreen EPA's Environmental Justice Screening and Reinvestment Map

Please note: Territory data (except Puerto Rico) is not available as comparable to the

Compare to US Compare to State

- Environmental Burden Indicators
- Socioeconomic Indicators
- Environmental Justice Indexes
- Supplemental Indexes
- Climate Change
- Health Disparities
- Critical Service Gaps



A heatmap showing environmental justice indicators in Chicago. The map is color-coded from yellow (low burden) to red (high burden). High burden areas are concentrated in the south and east of the city, including areas like Cicero, Berwyn, and Bridgeview. The map is titled 'EPA EJScreen' and 'EPA's Environmental Justice Screening and Reinvestment Map'.

Dashboards

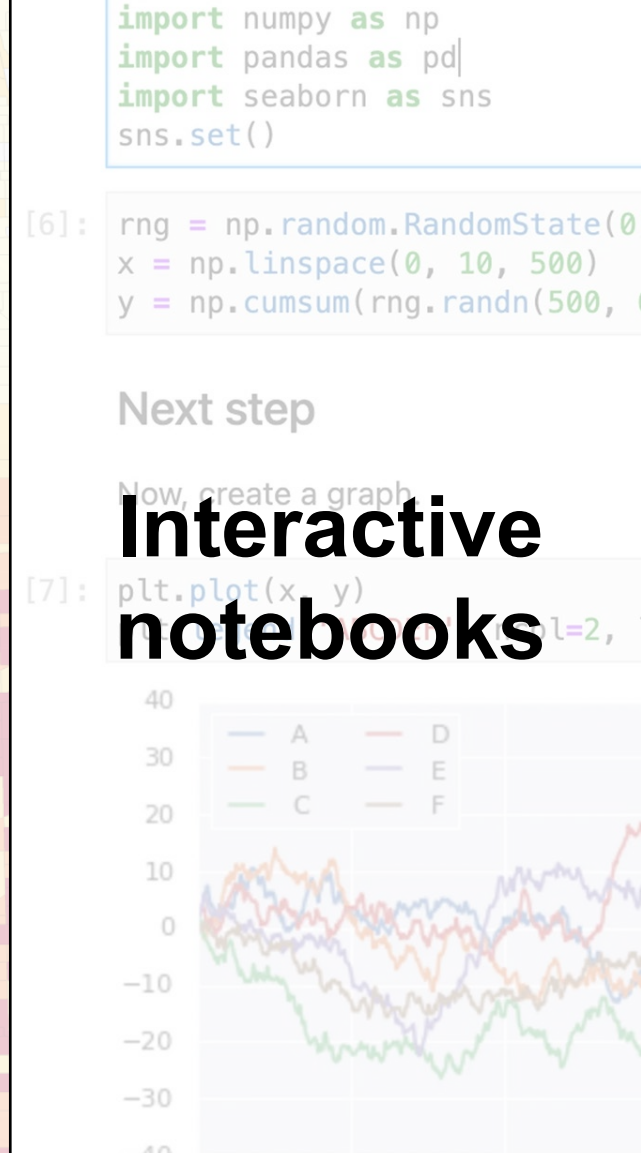
```
[5]: import matplotlib.pyplot as plt
plt.style.use('classic')
%matplotlib inline
import numpy as np
import pandas as pd
import seaborn as sns
sns.set()
```

```
[6]: rng = np.random.RandomState(0)
x = np.linspace(0, 10, 500)
y = np.cumsum(rng.randn(500, 6))
```

Next step

Now, create a graph

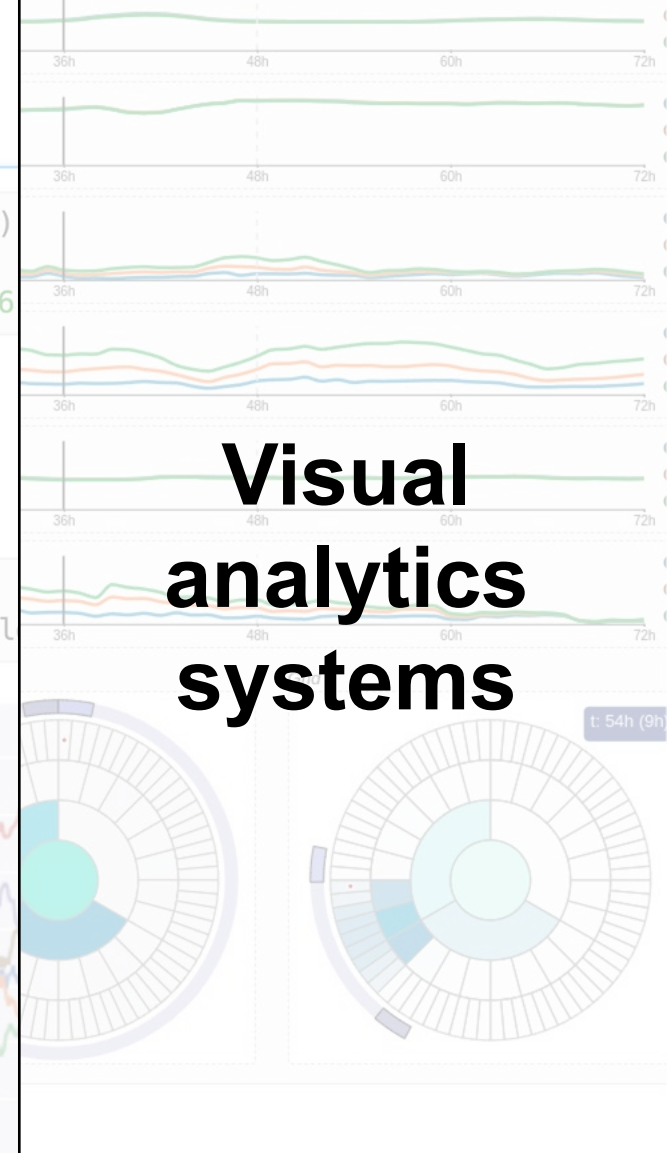
```
[7]: plt.plot(x, y)
```



A line plot showing the cumulative sum of random noise. The x-axis ranges from 0 to 10, and the y-axis ranges from -50 to 40. The plot shows a noisy, upward-trending line. The plot is titled 'Next step' and 'Now, create a graph'.

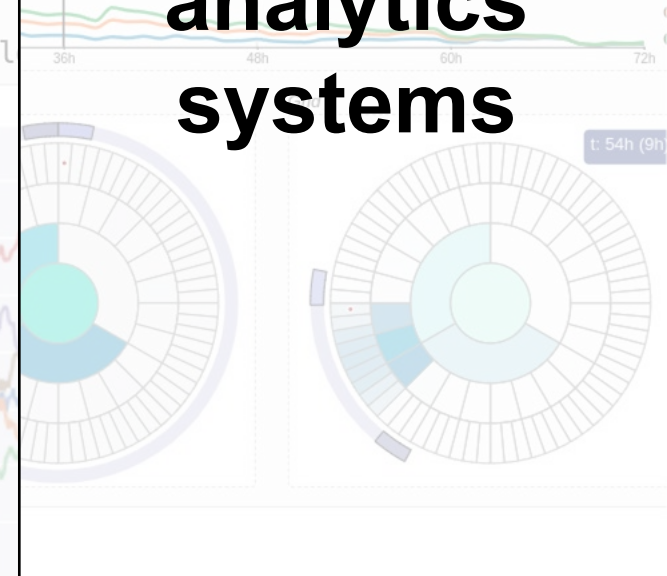
Interactive notebooks

Map 2 g3 Moist Map 3 g3 Precip. -1 36h



A series of time-series plots showing precipitation and moisture over time. The x-axis represents time in hours (36h, 48h, 60h, 72h). The y-axis represents precipitation and moisture. The plots show multiple lines representing different variables. The plots are titled 'Map 2 g3 Moist Map 3 g3 Precip. -1 36h'.

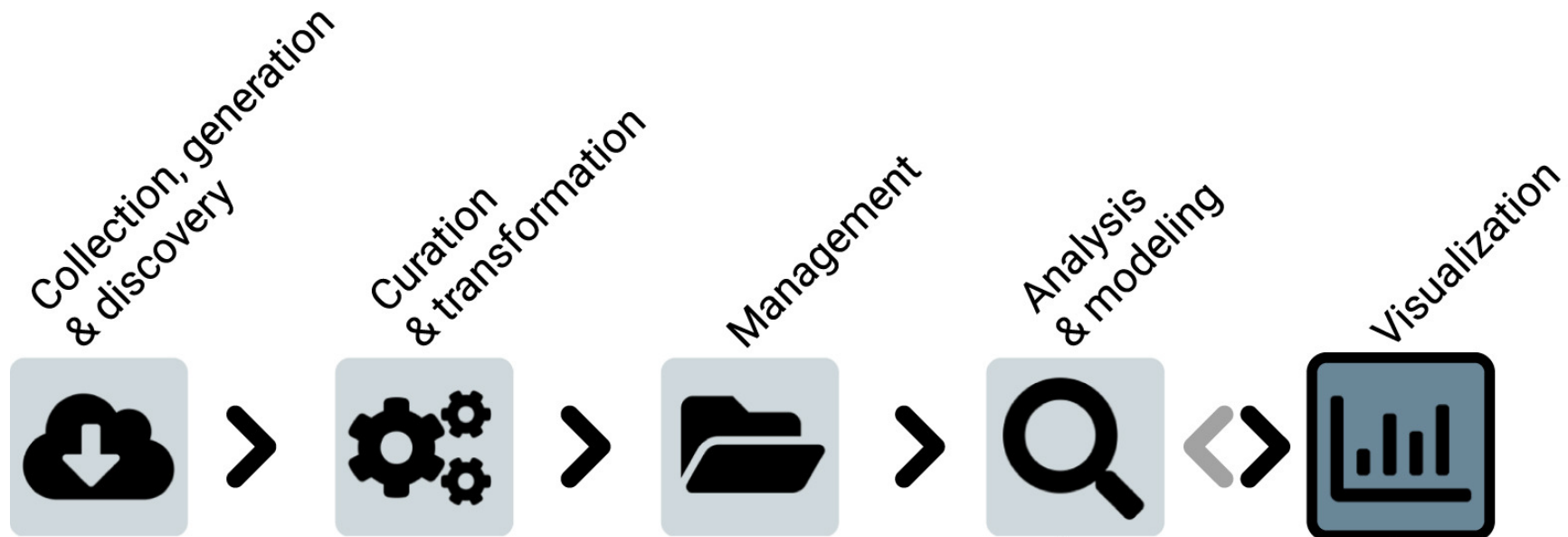
Visual analytics systems



Two circular sunburst charts showing hierarchical data. The charts are divided into segments representing different categories. The charts are titled 't: 54h (9h)'.

What is a data pipeline?

The directed flow of data, e.g.:



What is a data pipeline?

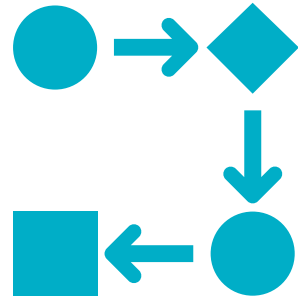


A dataflow graph: nodes (ingest, store, transform, serve) with edges and dependencies you can draw and reason about (what runs first? what's incremental vs full refresh?).



A socio-technical process: the *artifacts and practices* that keep it reliable: **code** (e.g., SQL, Python, transforms, tests), **configuration** (e.g., schemas, data contracts, schedules, access rules), and **management** (e.g., object store, databases/warehouses).

What is a data pipeline?

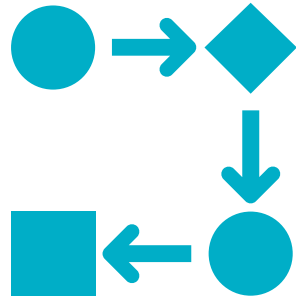


Dataflow



Data management

What is a data pipeline?



Dataflow

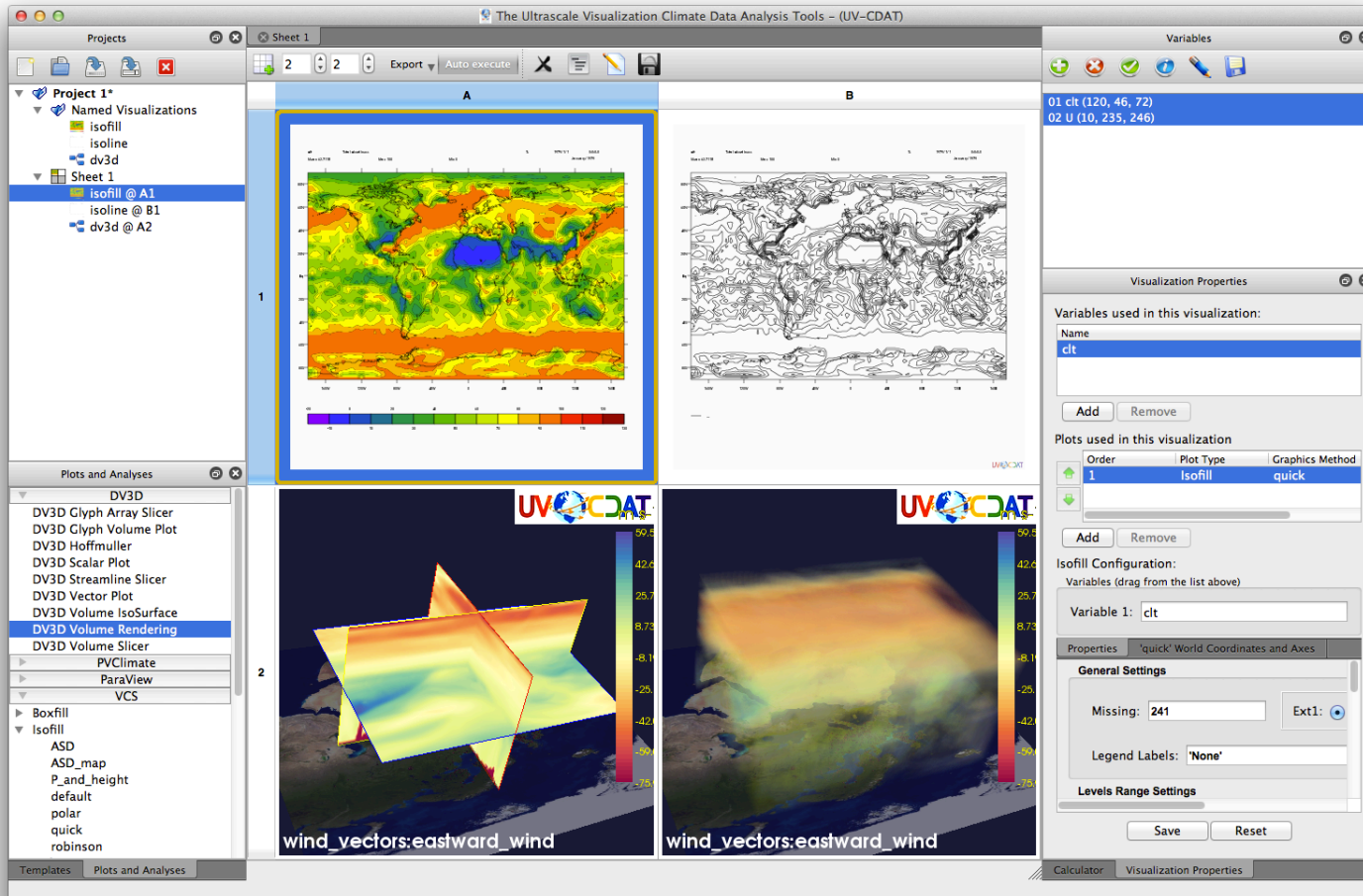


Data management

Dataflows in visualization

Systems and frameworks that support the creation of complex visualizations

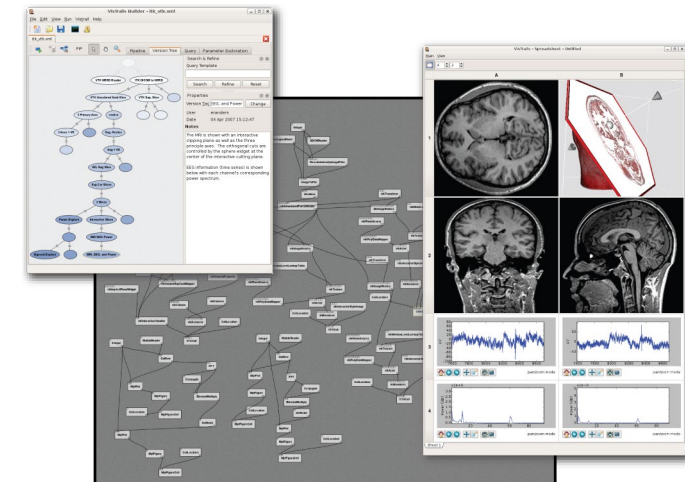
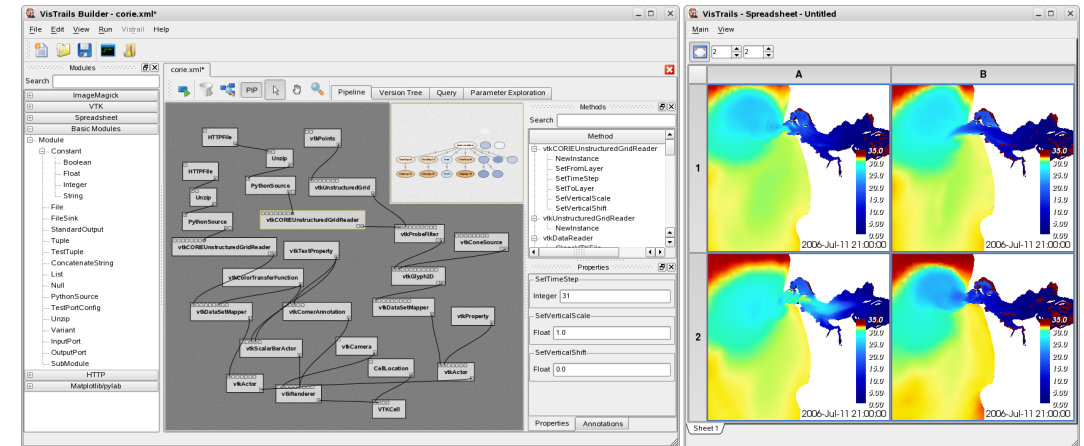
- Ability to create complex pipelines
- Ability to track changes to visualizations



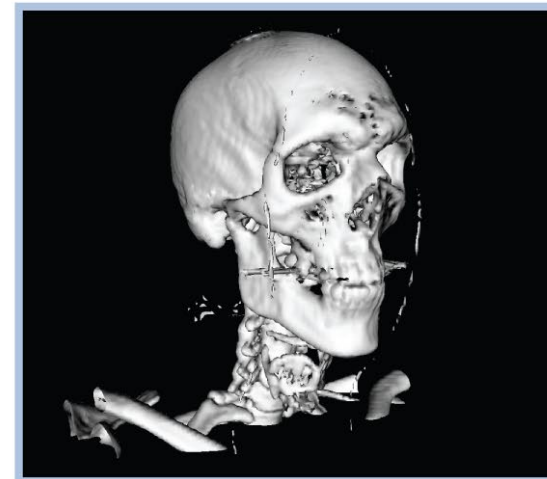
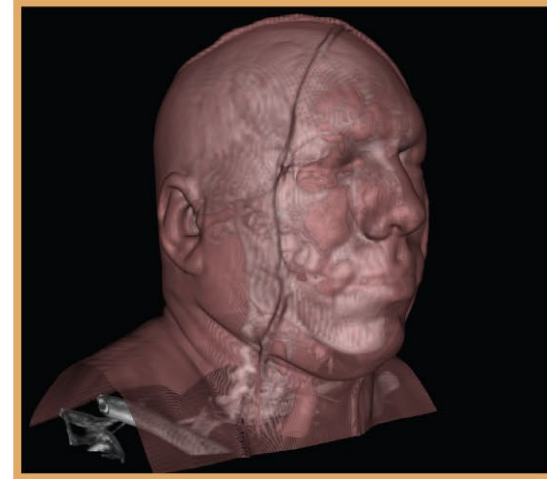
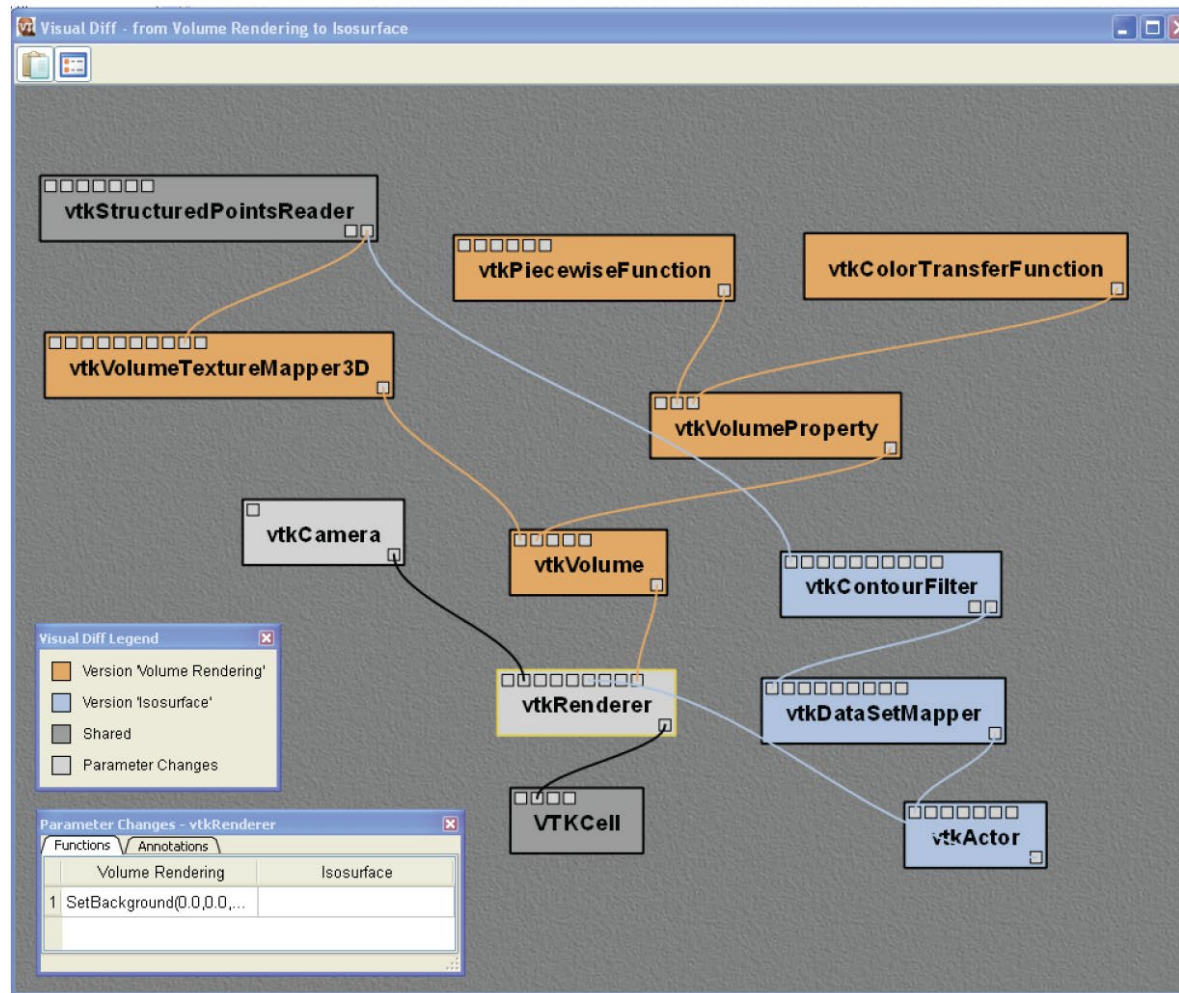
Dataflows in visualization

VisTrails (Callahan et al., 2006)

- System that enables interactive, multiple-view visualizations
- Simplifies the creation and maintenance of a large number of visualizations
- Detailed provenance of visualization results
- Separation between pipeline specification and execution



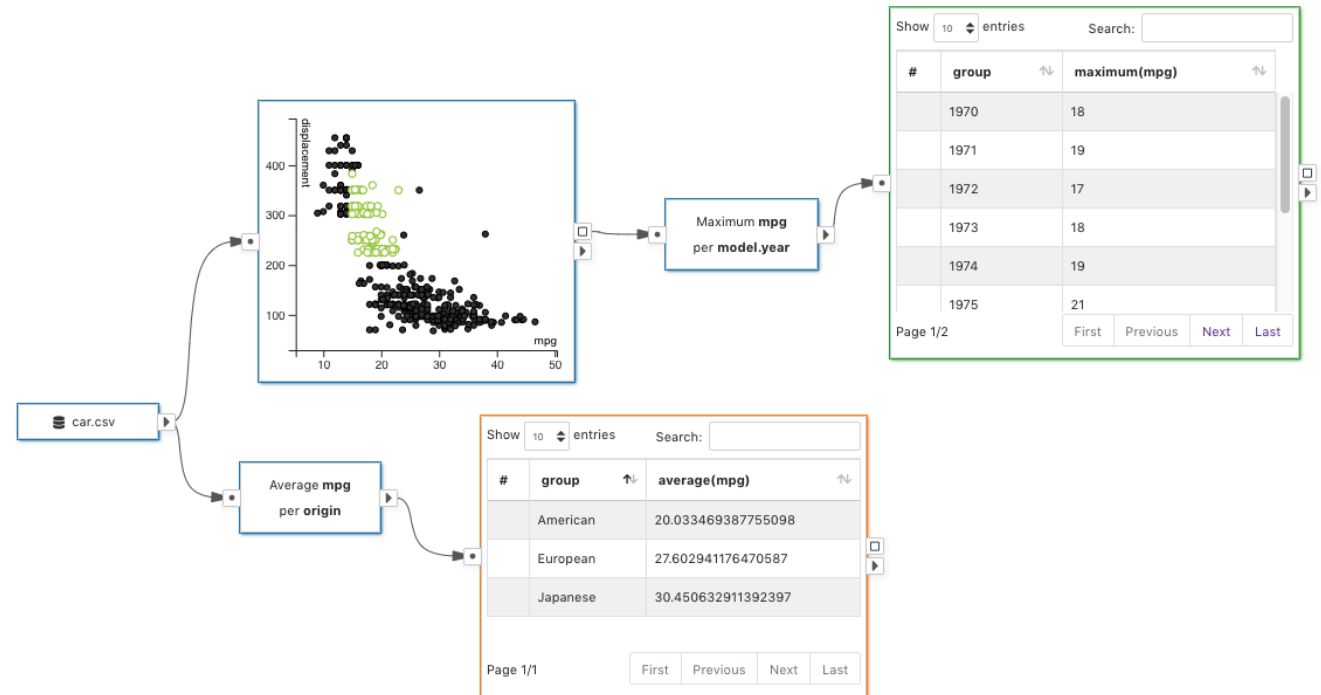
Dataflows in visualization



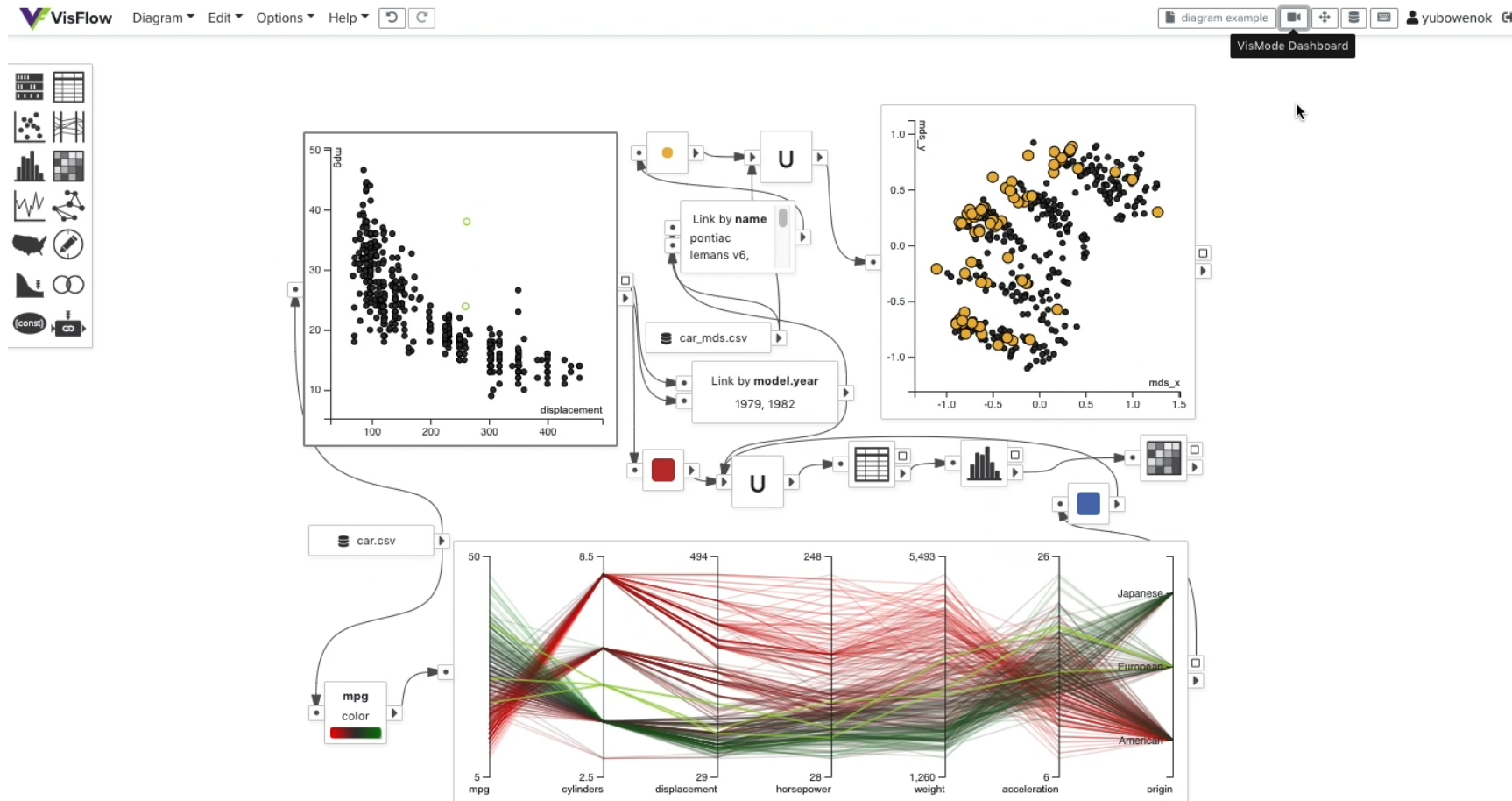
Dataflows in visualization

VisFlow (Yu and Silva, 2016)

- Web-based dataflow framework for visual data exploration
- Uses a subset dataflow so that users can select, manipulate, brush and link data subsets across multiple visualizations.



Dataflows in visualization



that allows the user to interactively visualize and explore their data



Diversity of workflows

Assessing the landscape of toolkits, frameworks, and authoring tools for urban visual analytics systems



Leonardo Ferreira^a, Gustavo Moreira^a, Maryam Hosseini^b, Marcos Lage^c, Nivan Ferreira^d, Fabio Miranda^{a,*}

^a University of Illinois Chicago, USA

^b Massachusetts Institute of Technology, USA

^c Universidade Federal Fluminense, Brazil

^d Universidade Federal de Pernambuco, Brazil

ARTICLE INFO

Keywords:

Visual analytics
Visualization toolkits
Visualization grammars
Visualization authoring
Urban visual analytics

ABSTRACT

Over the past decade, there has been a significant increase in the development of visual analytics systems dedicated to addressing urban issues. These systems distill intricate urban analysis workflows into intuitive, interactive visual representations and interfaces, enabling users to explore, understand, and derive insights from large and complex data, including street-level imagery, street networks, and building geometries. Developing urban visual analytics systems, however, is a challenging endeavor that requires considerable programming expertise and interaction between various multidisciplinary stakeholders. This situation often leads to monolithic and isolated prototypes that are hard to reproduce, combine, or extend. Concurrently, there has been an increase in the availability of general and urban-specific toolkits, frameworks, and authoring tools that are open source and abstract away the need to implement low-level visual analytics functionalities. This paper provides a hierarchical taxonomy of urban visual analytics systems to contextualize how they are usually designed, implemented, and evaluated. We develop this taxonomy across three distinct levels (i.e., dimensions, categories, and tags), juxtaposing visualization with analytics, data, and system dimensions. We then assess the extent to which current open-source toolkits, frameworks, and authoring tools can effectively support the development of components tailored to urban visual analytics, identifying their strengths and limitations in addressing the unique challenges posed by urban data. In doing so, we offer a roadmap that can guide the effective employment of existing resources and chart a pathway for developing and refining future systems.

> 130 papers (Ferreira et al., 2024)

The State of the Art in Visual Analytics for 3D Urban Data

Fabio Miranda¹, Thomas Ortner², Gustavo Moreira¹, Maryam Hosseini², Milena Vuckovic³, Filip Biljecki⁴, Claudio T. Silva⁵, Marcos Lage⁶, Nivan Ferreira⁷

¹Department of Computer Science, University of Illinois Chicago, USA

²City Form Lab, Massachusetts Institute of Technology, USA

³VRVis Zentrum für Virtual Reality und Visualisierung Forschungs-GmbH, Austria

⁴Department of Architecture and Department of Real Estate, National University of Singapore, Singapore

⁵Tandon School of Engineering and Center for Data Science, New York University, USA

⁶Instituto de Computação, Universidade Federal Fluminense, Brazil

⁷Centro de Informática, Universidade Federal de Pernambuco, Brazil

Abstract

Urbanization has amplified the importance of three-dimensional structures in urban environments for a wide range of phenomena that are of significant interest to diverse stakeholders. With the growing availability of 3D urban data, numerous studies have focused on developing visual analysis techniques tailored to the unique characteristics of urban environments. However, incorporating the third dimension into visual analytics introduces additional challenges in designing effective visual tools to tackle urban data's diverse complexities. In this paper, we present a survey on visual analytics of 3D urban data. Our work characterizes published works along three main dimensions (why, what, and how), considering use cases, analysis tasks, data, visualizations, and interactions. We provide a fine-grained categorization of published works from visualization journals and conferences, as well as from a myriad of urban domains, including urban planning, architecture, and engineering. By incorporating perspectives from both urban and visualization experts, we identify literature gaps, motivate visualization researchers to understand challenges and opportunities, and indicate future research directions.

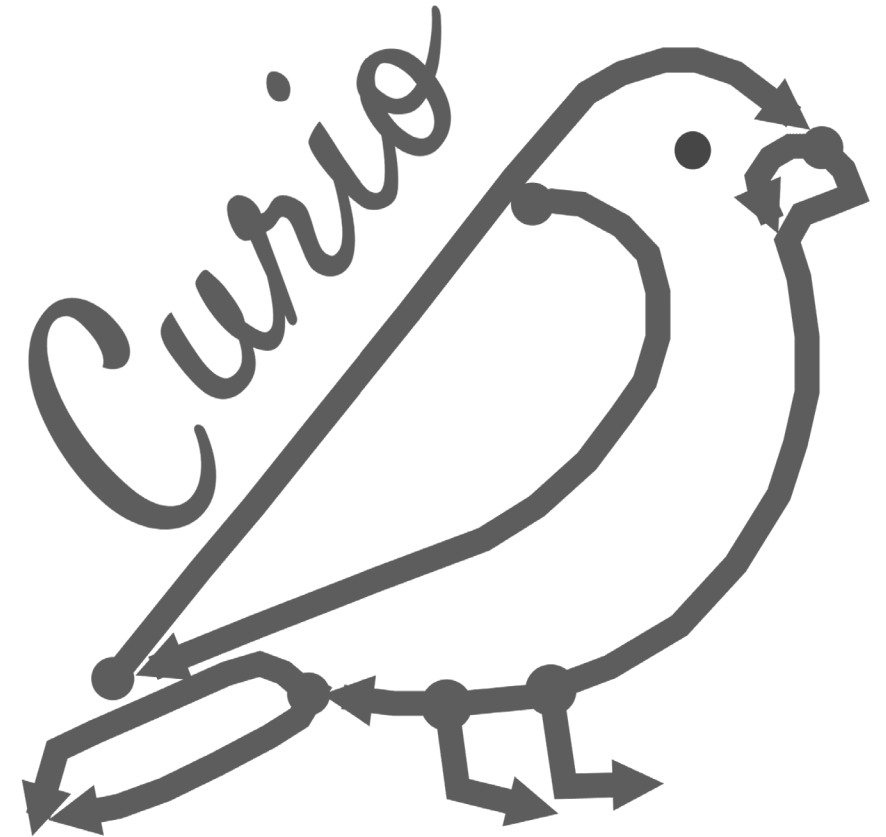
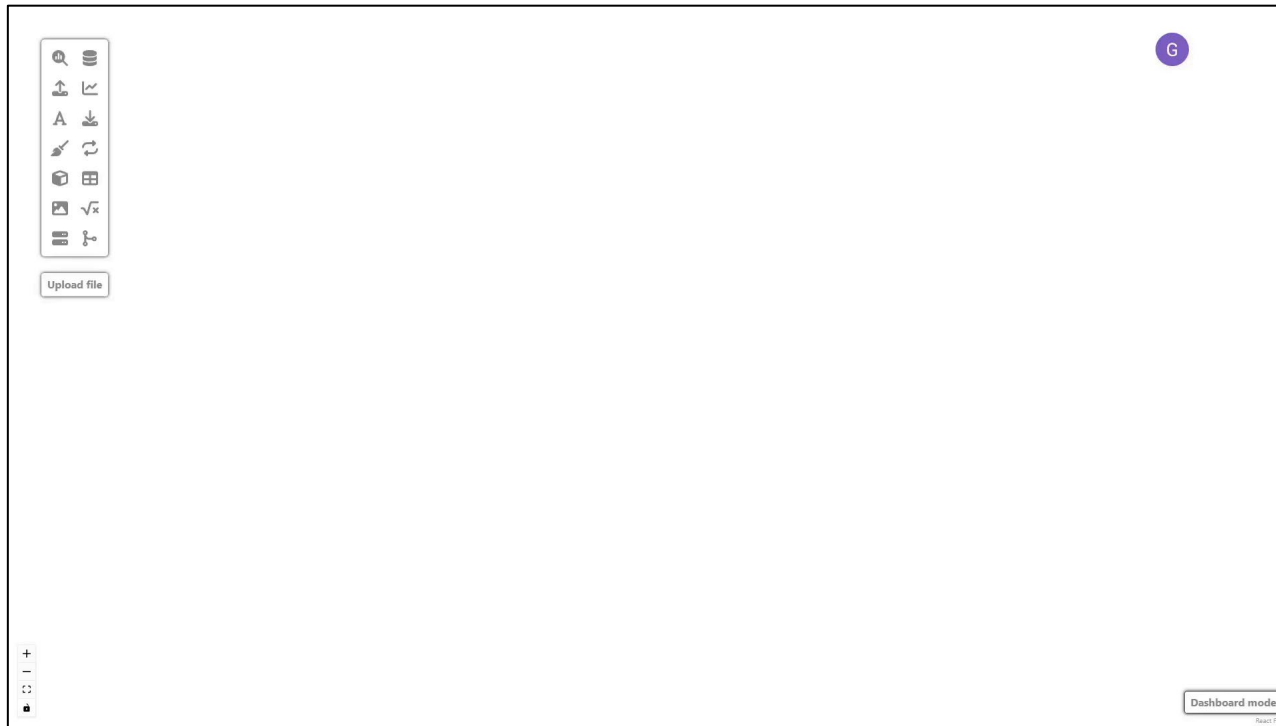
CCS Concepts

• **General and reference** → Surveys and overviews; • **Human-centered computing** → Visualization; • **Applied computing** → Earth and atmospheric sciences; Environmental sciences; Cartography; Architecture (buildings);

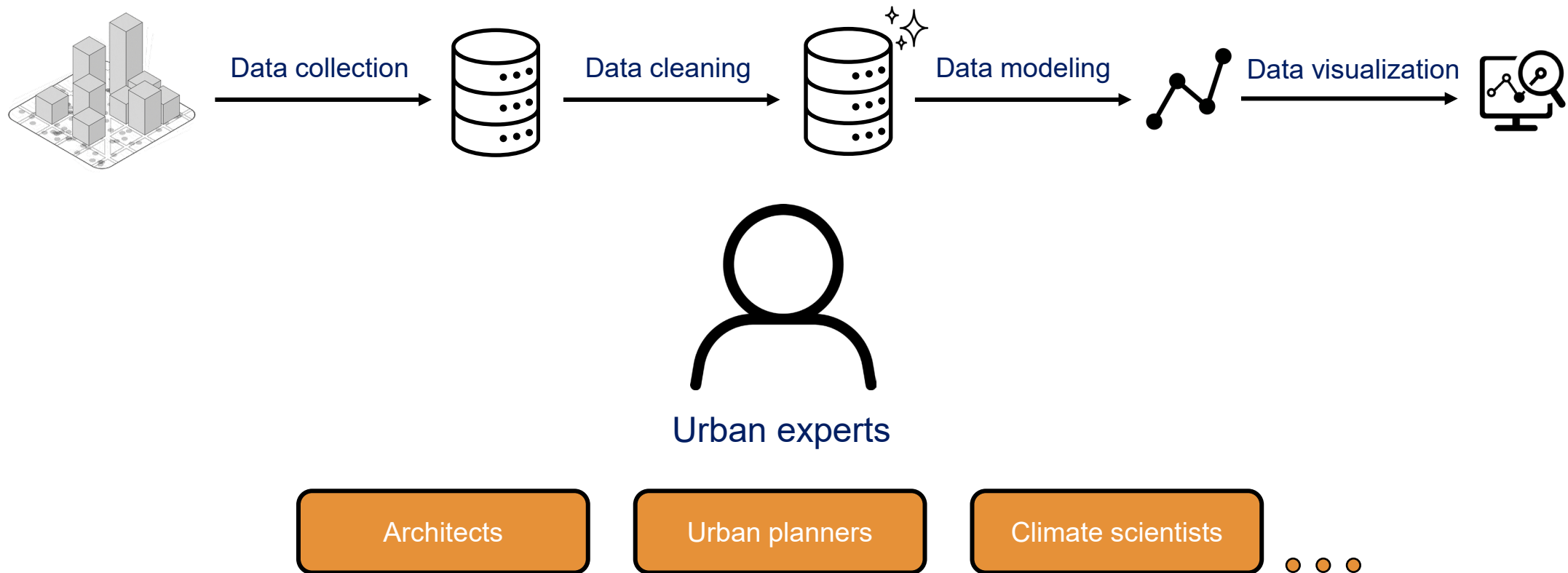
> 170 papers (Miranda et al., 2024)

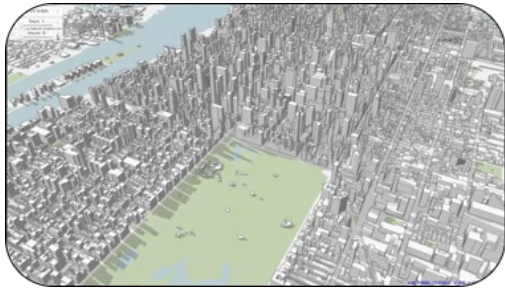


Curio (Moreira et al., 2025)



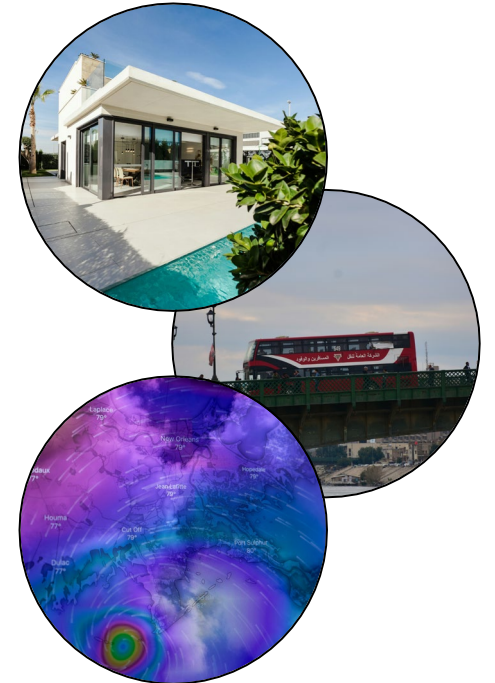
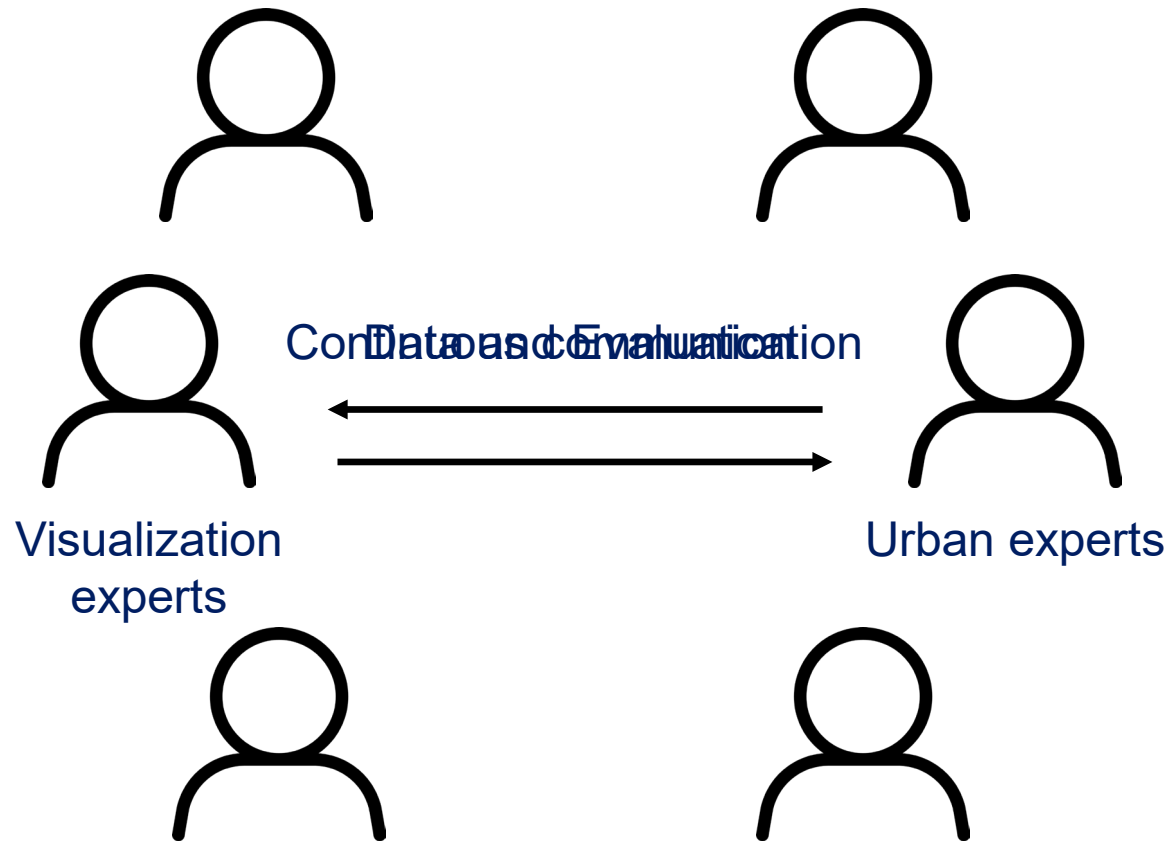
Typical workflow





Develop interoperability
collaboration across
design and
implementation

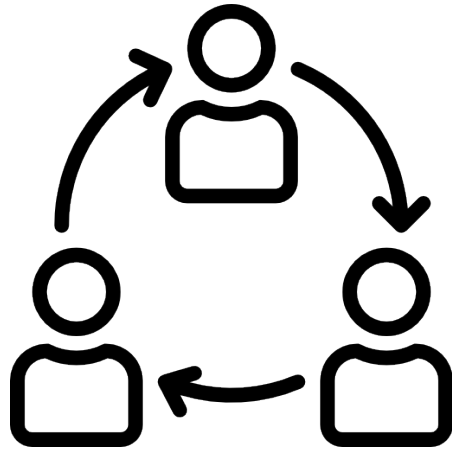
Visual analytics
system



Domain specific
problems

Curio's design goals

(1) Collaborative visual analytics



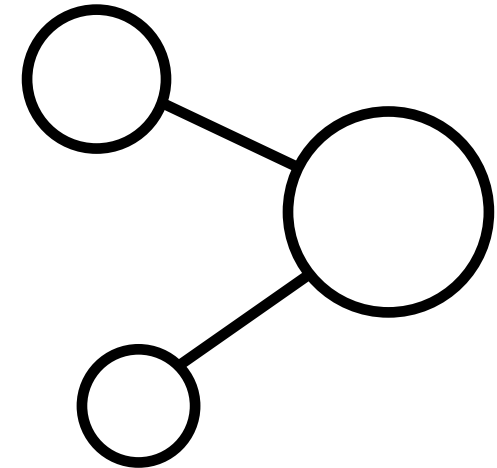
Shared dataflows

(2) Flexibility and modular design



Reusable templates

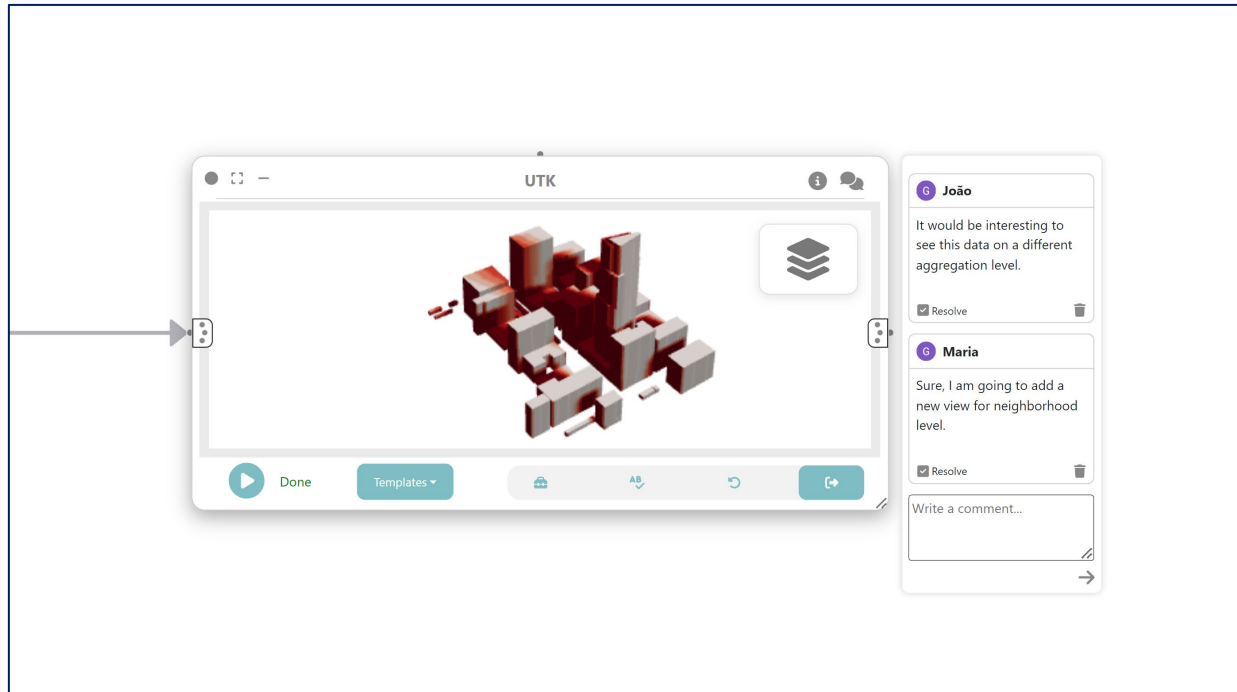
(3) Reproducibility and provenance of modules



Dataflow provenance

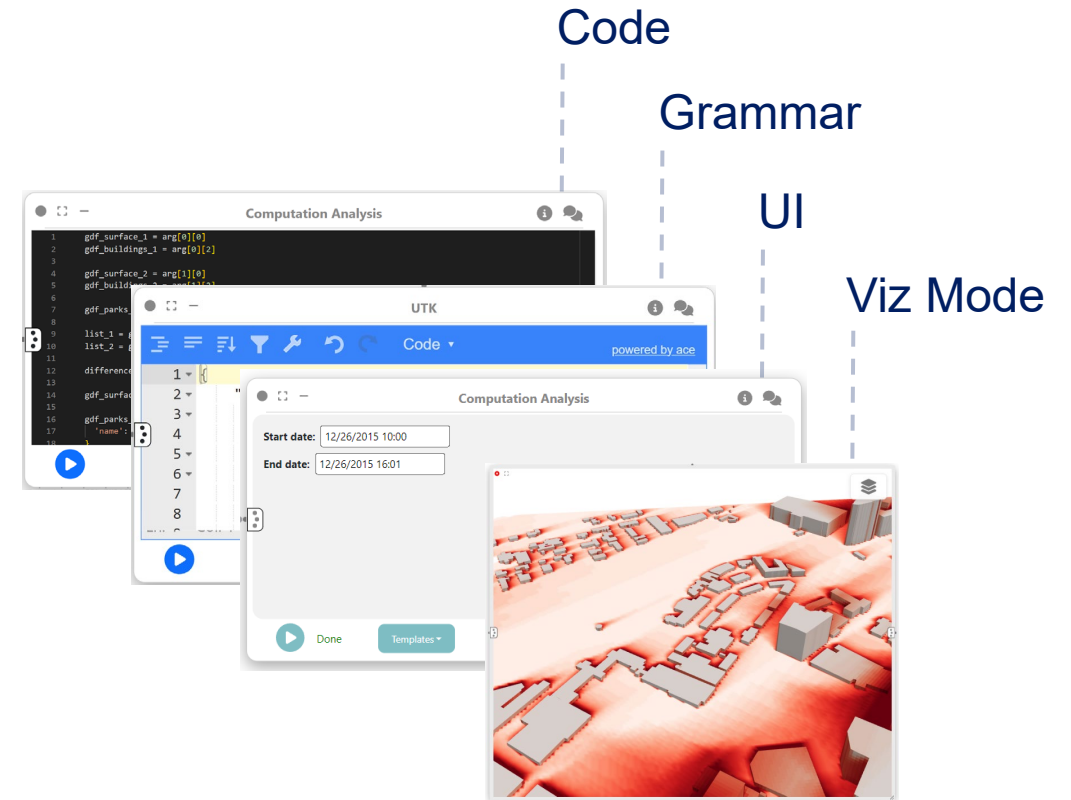


Collaboration



(1) Collaborative visual analytics

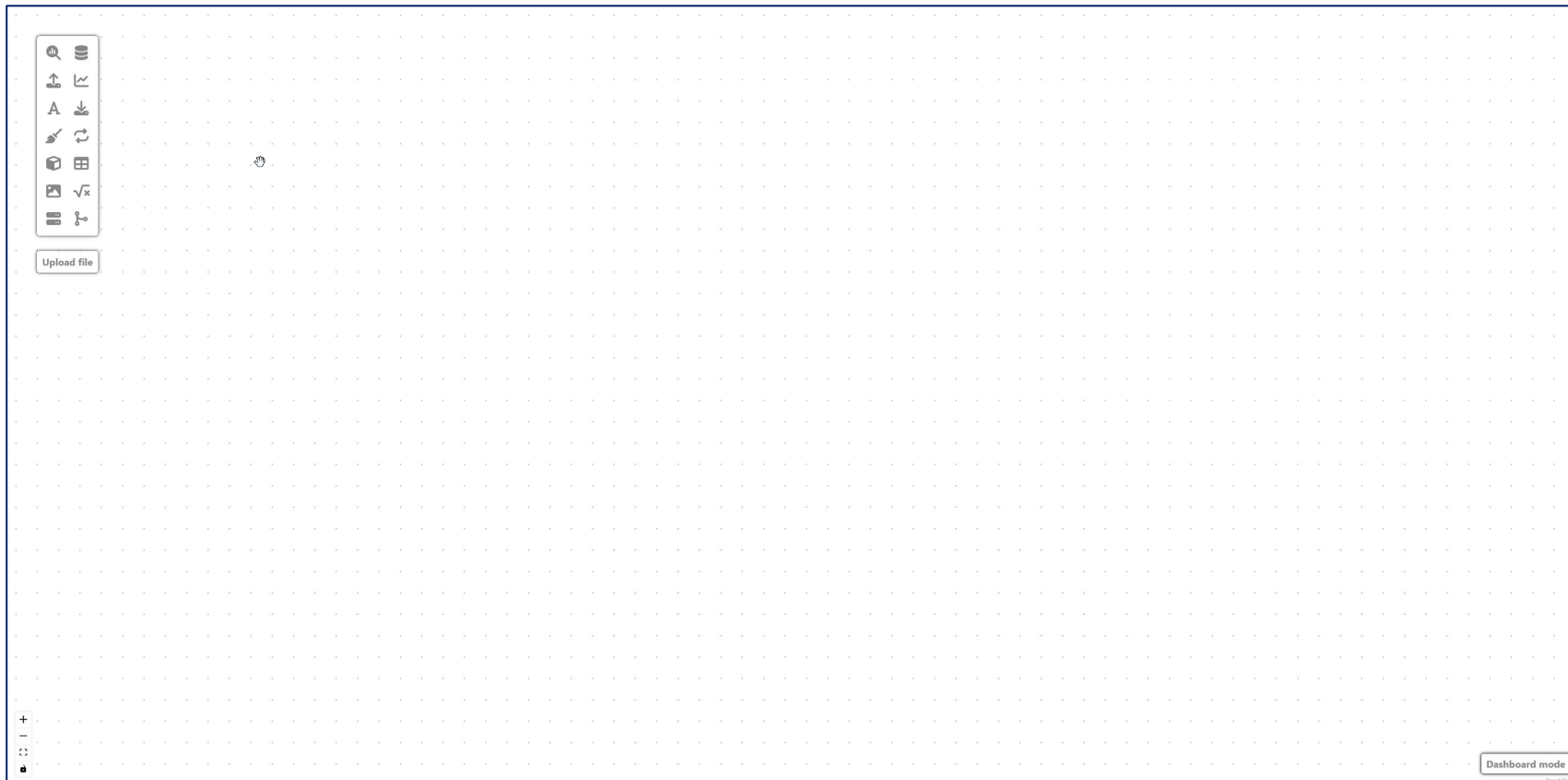
Multiple users



Multiple levels of abstraction



Templates



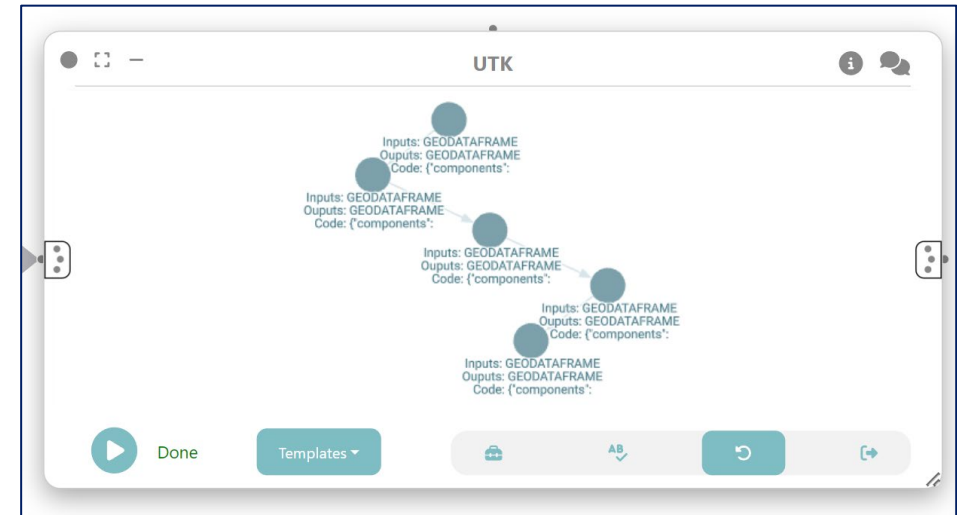
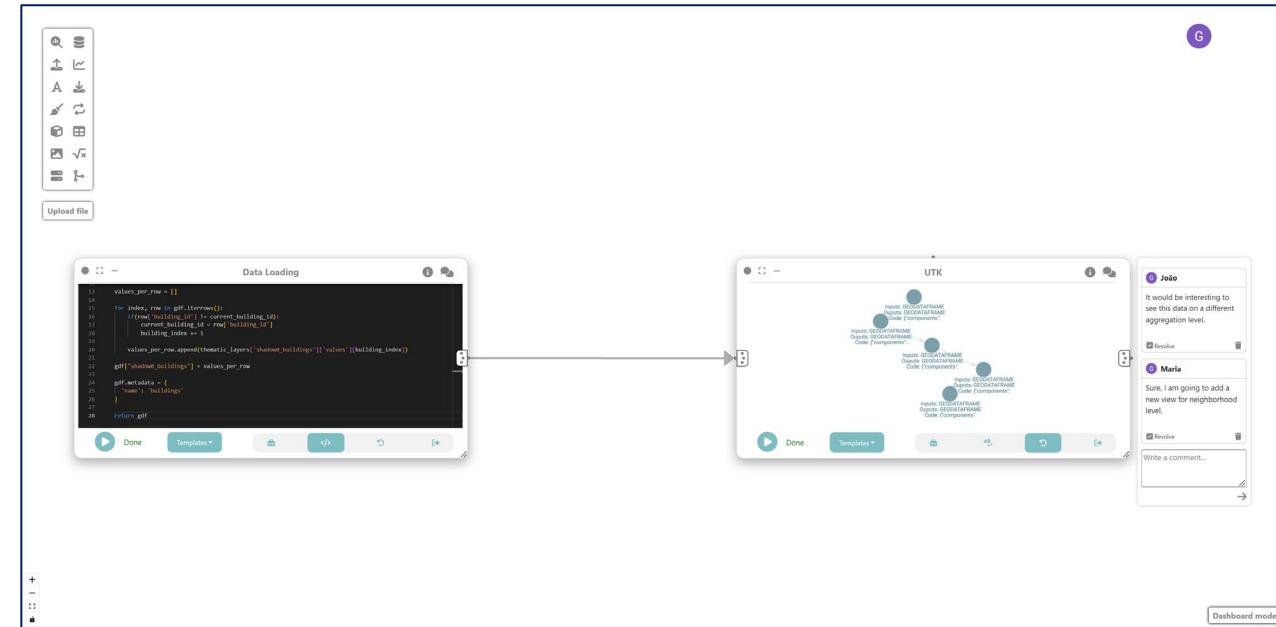
(2) Flexibility and modular design



Provenance

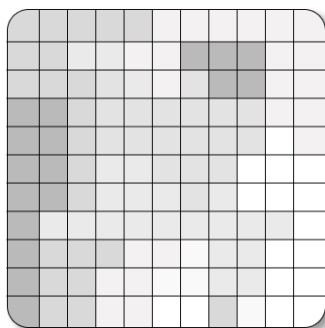
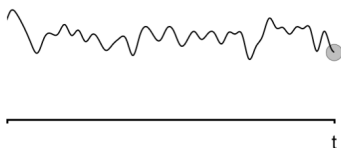
- Dataflow level and Node level
- Navigating between versions
- Follows W3C PROV standard

(3) Reproducibility and provenance of modules



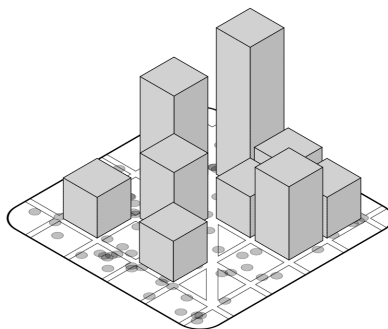
Thematic layers

Point Layer

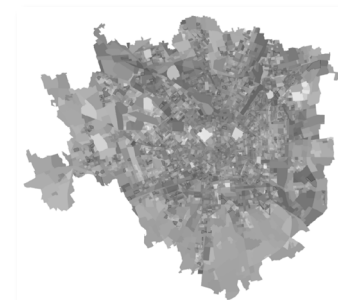
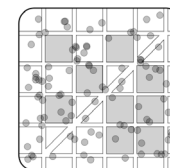


Grid

Image layers



3D



2D



Network

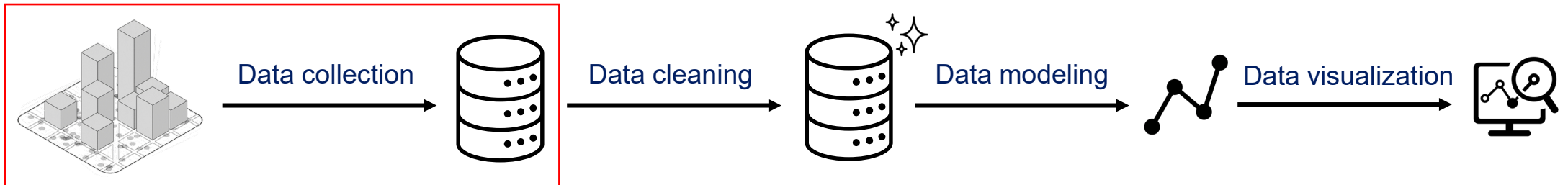
Physical layers

What-if scenario planning



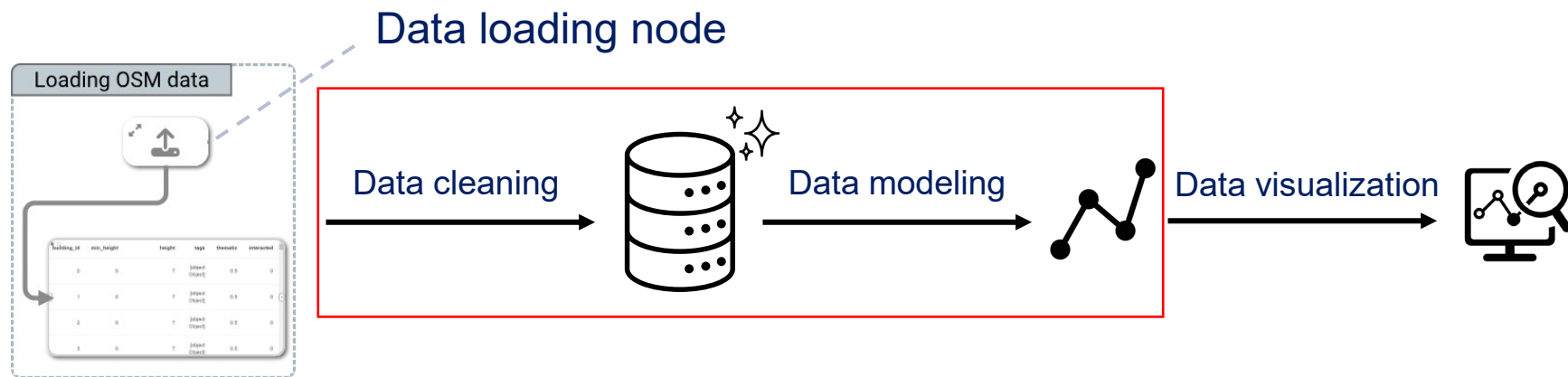


Curio's dataflow





Curio's dataflow



Node content

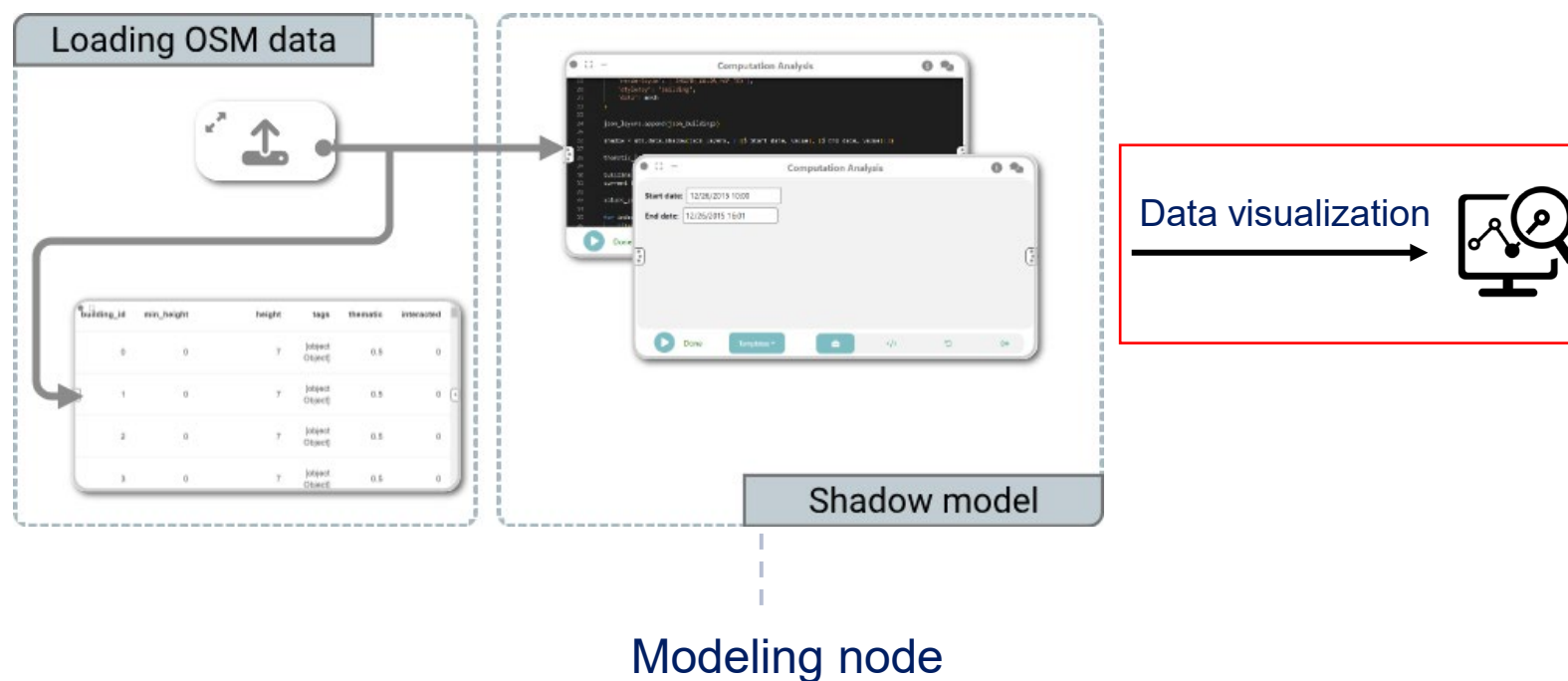
Data Loading

```
1 import utk
2
3 uc = utk.OSM.load([42.336844, -71.113459, 42.345559, -71.099216], layers=[{'name': 'buildings', 'args': {'
4
5 # buildings
6 gdf_buildings = uc.layers['gdf']['sections'][0]
7 gdf_buildings['thematic'] = 0.5
8 gdf_buildings['metadata'] = {
9   'name': 'buildings'
10 }
11
12 # surface
13 json_surface = uc.layers['json'][1]
14 gdf_surface = uc.layers['gdf']['objects'][1]
15 gdf_surface['metadata'] = {
16   'name': 'surface'
17 }
```

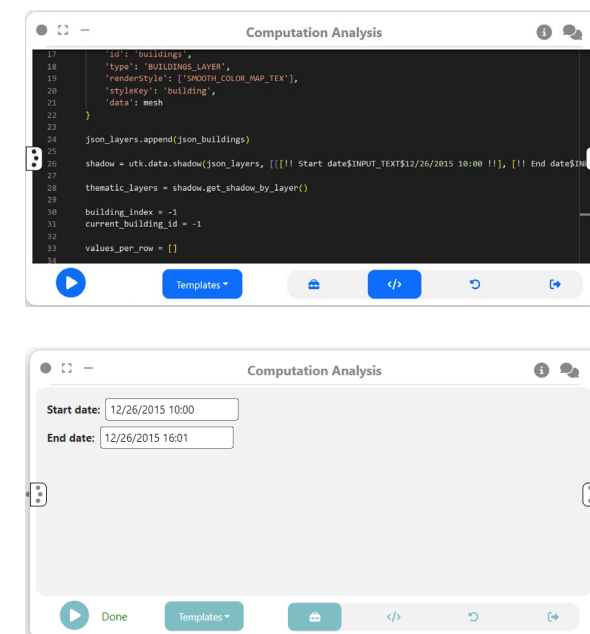
building_id	min_height	height	tags	thematic	interacted
0	0	7	[object Object]	0.5	0
1	0	7	[object Object]	0.5	0
2	0	7	[object Object]	0.5	0
3	0	7	[object Object]	0.5	0



Curio's dataflow

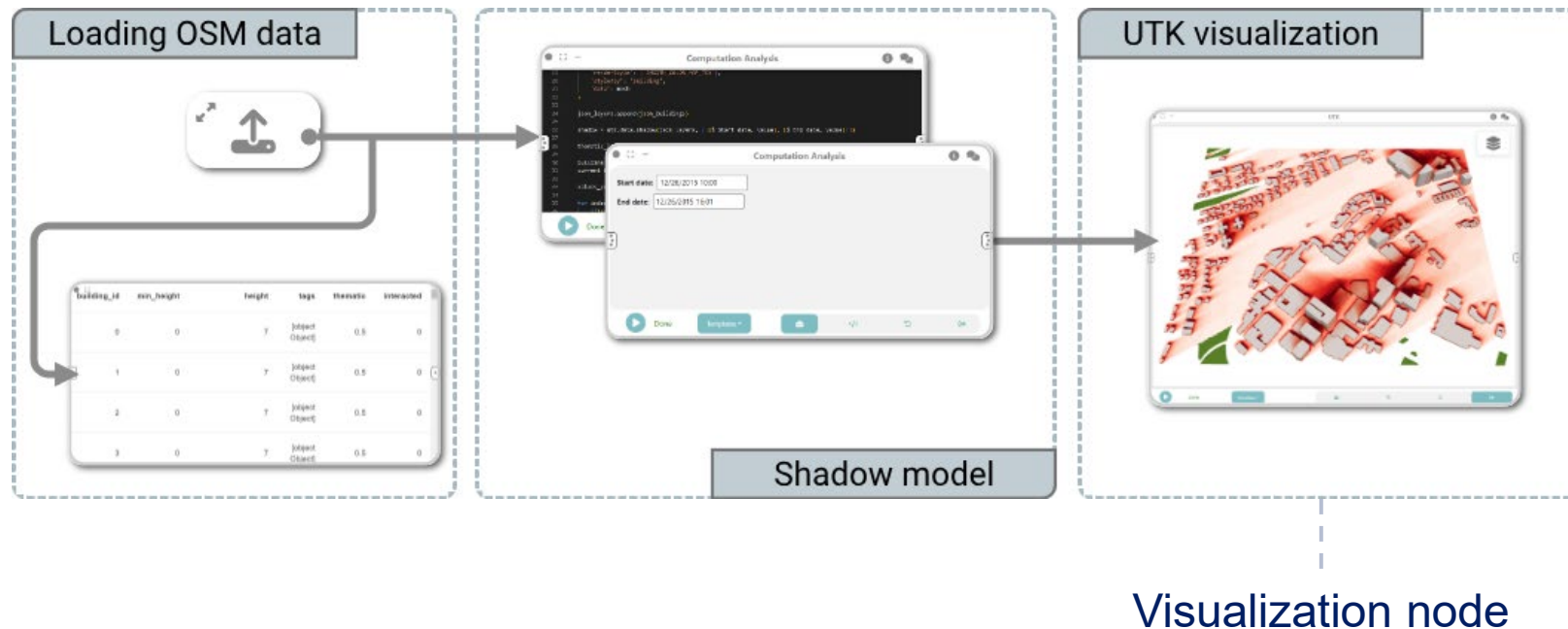


Node content



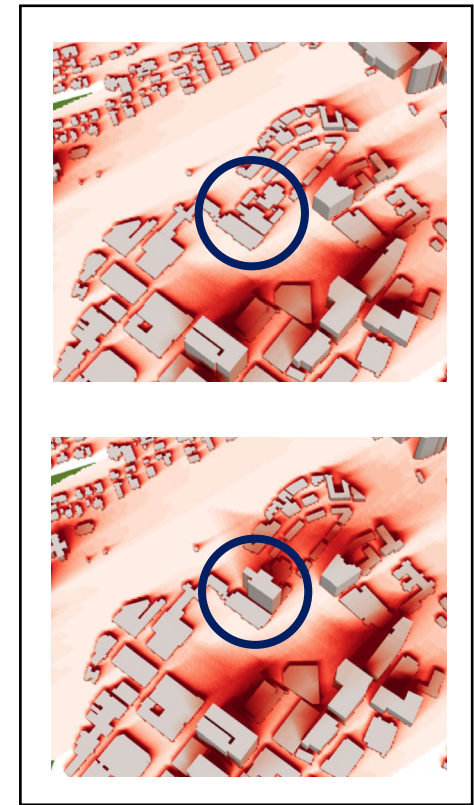


Curio's dataflow



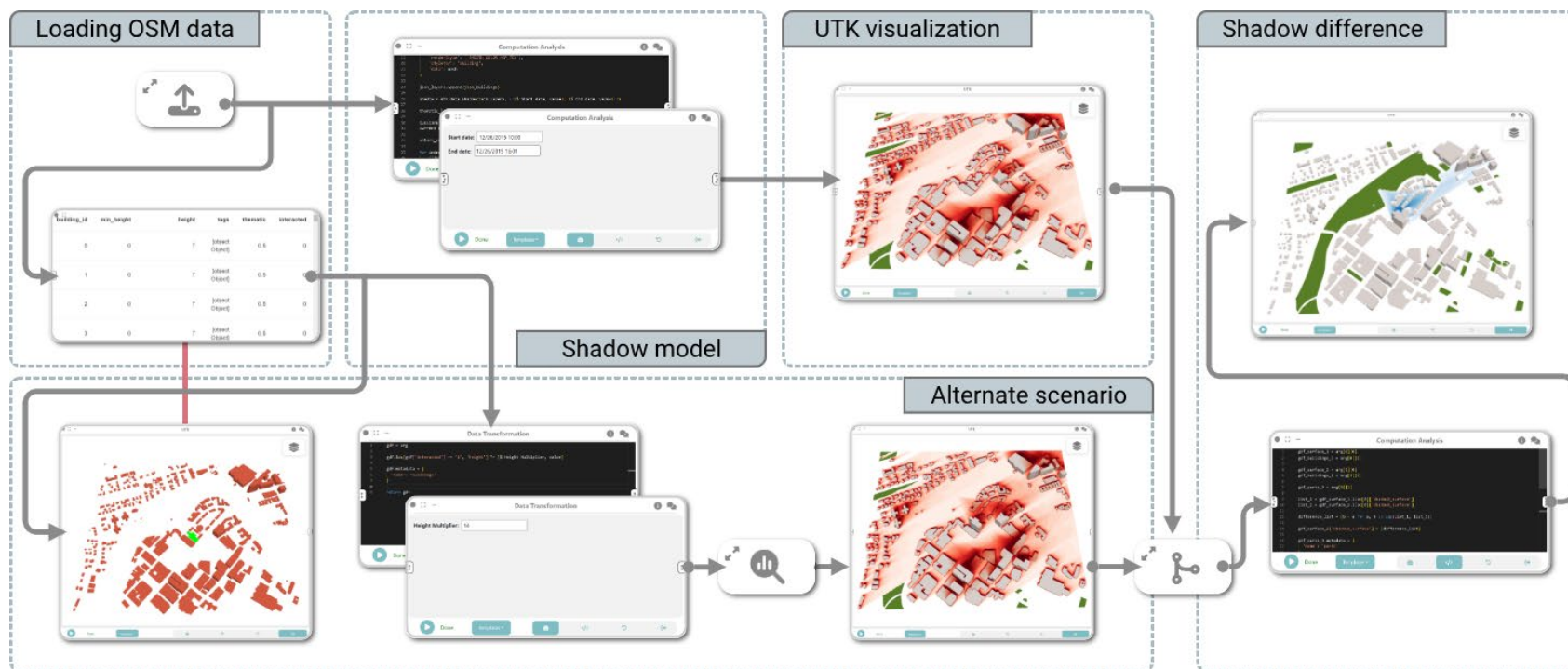
Node content







Curio's dataflow



Node content

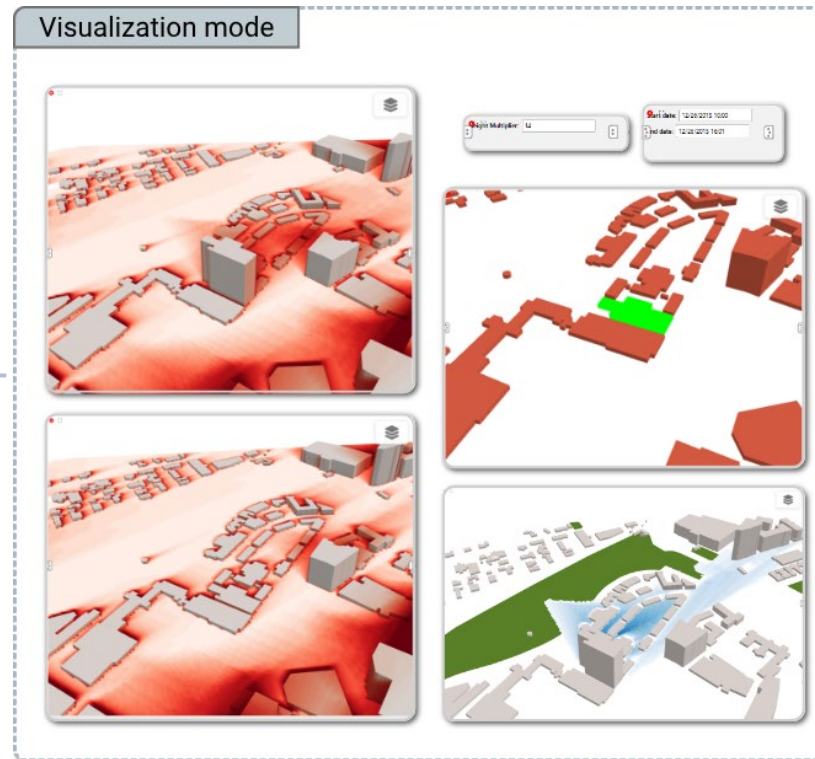
```
Computation Analysis
1 gdf_surface_1 = arg[0][0]
2 gdf_buildings_1 = arg[0][2]
3
4 gdf_surface_2 = arg[1][0]
5 gdf_buildings_2 = arg[1][2]
6
7 gdf_parks_3 = arg[0][1]
8
9 list_1 = gdf_surface_1.iloc[['shadow_surface']]
10 list_2 = gdf_surface_2.iloc[['shadow_surface']]
11
12 difference_list = [b - a for a, b in zip(list_1, list_2)]
13
14 gdf_surface_2[['shadow_surface']] = difference_list
15
16 gdf_parks_3.metadata = {
17     'name': 'parks'
18 }
```

Operations
between layers



Curio's dataflow

Highest level of abstraction

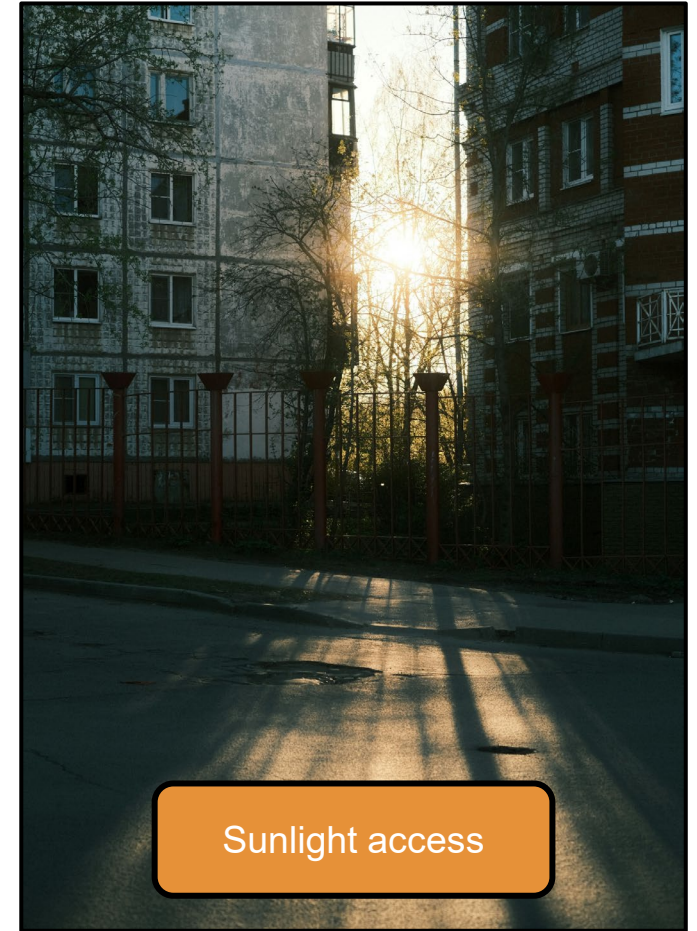
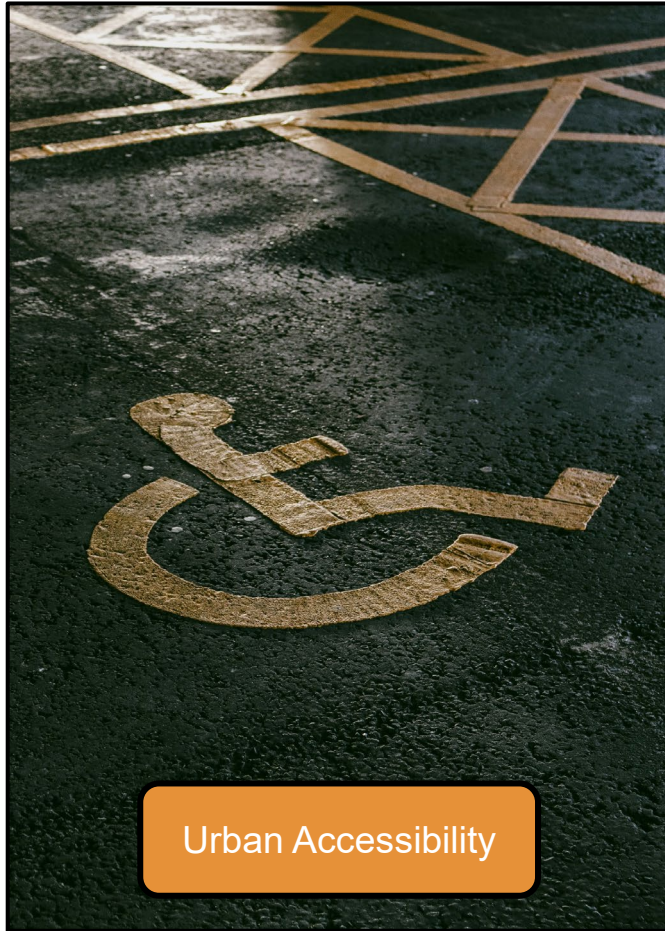


Node content

Height Multiplier: 14

Start date: 12/26/2015 10:00

End date: 12/26/2015 16:01



Usage Scenarios



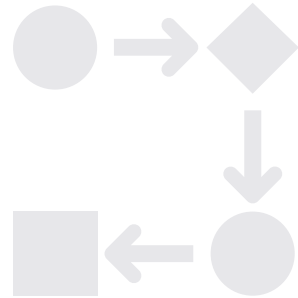
Upload file



Dashboard mode



What is a data pipeline?



Dataflow

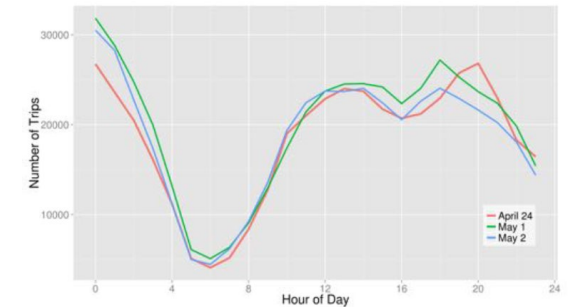
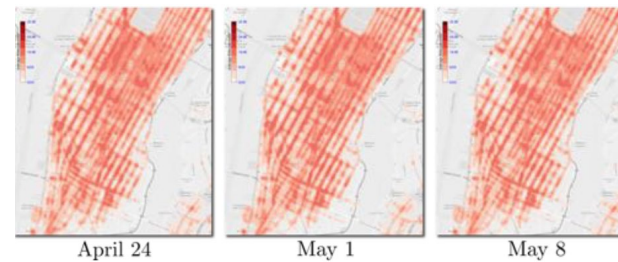


Data management

Data challenges

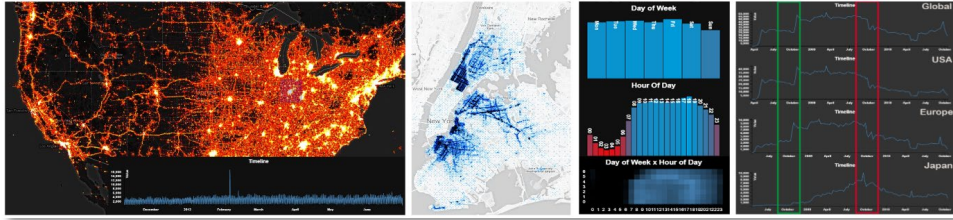


- 365×24 1-hour slices in one year.
- Which slides are interesting?

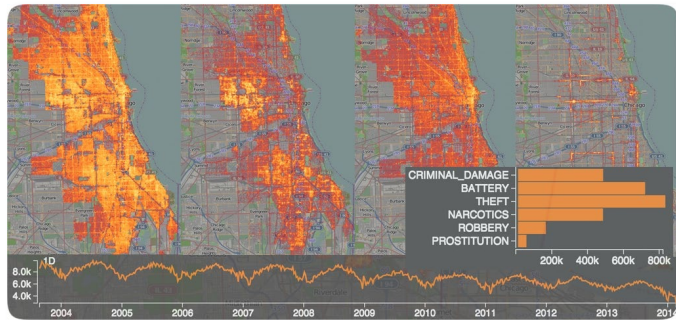


Accelerating data interaction methods

OLAP queries



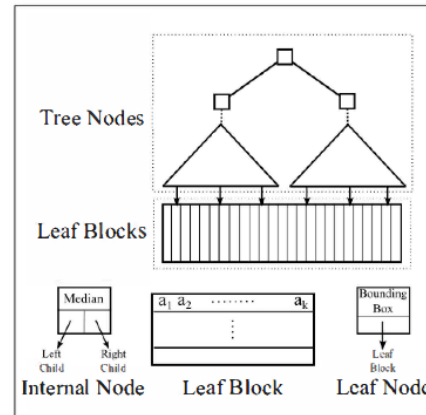
Hashedcubes [Pahins et al., 2017]



Nanocube [Lins et al., 2013]

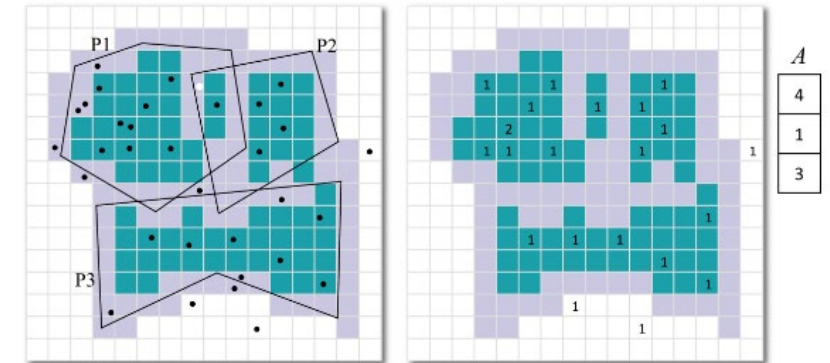
TopKube [Miranda et al., 2018]

Selection



STIG [Doraiswamy et al., 2015]

Spatiotemporal joins



Raster join [Tzirita Zacharitou, Doraiswamy et al., 2018]

Data structures for visualization

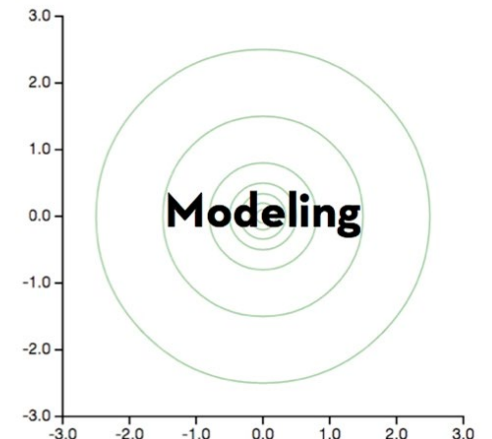
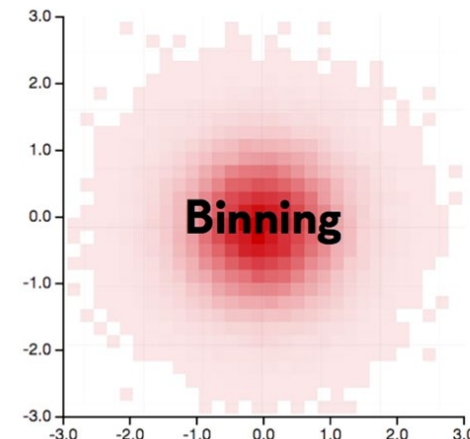
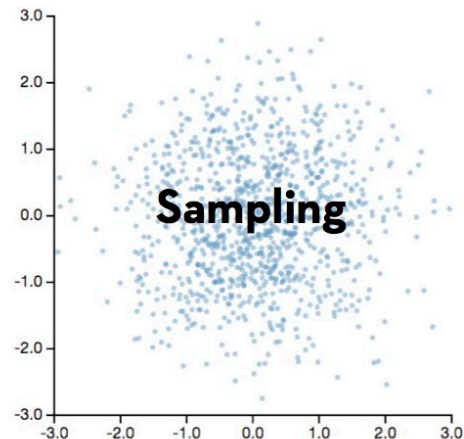
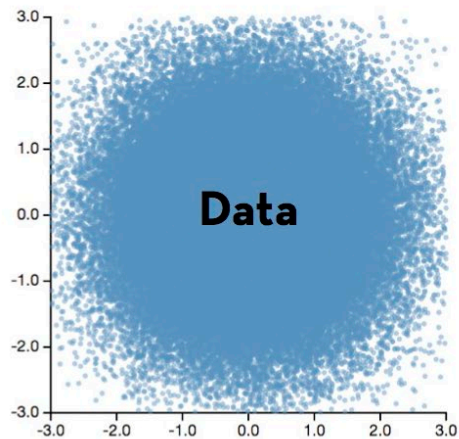
- immense [Liu et al., 2013]
- Nanocube [Lins et al., 2013]
- TopKube [Miranda et al., 2018]
- Time Lattice [Miranda et al., 2019]
- Learned cubes
- ...

Visualization requirements

- Visualizations have a specific set of “interaction patterns” and resolutions.
- Data schemes need to be aware of these limitations.

Perceptual and interactive scalability should be limited by the chosen resolution of the visualized data, not number of records.

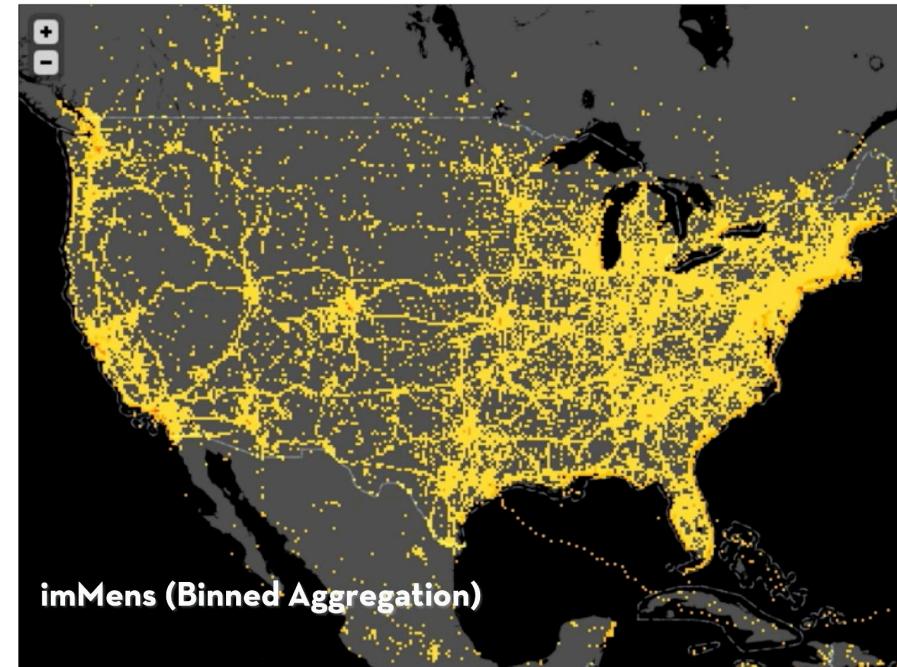
Data reduction techniques



Different reduction techniques applied on the same dataset

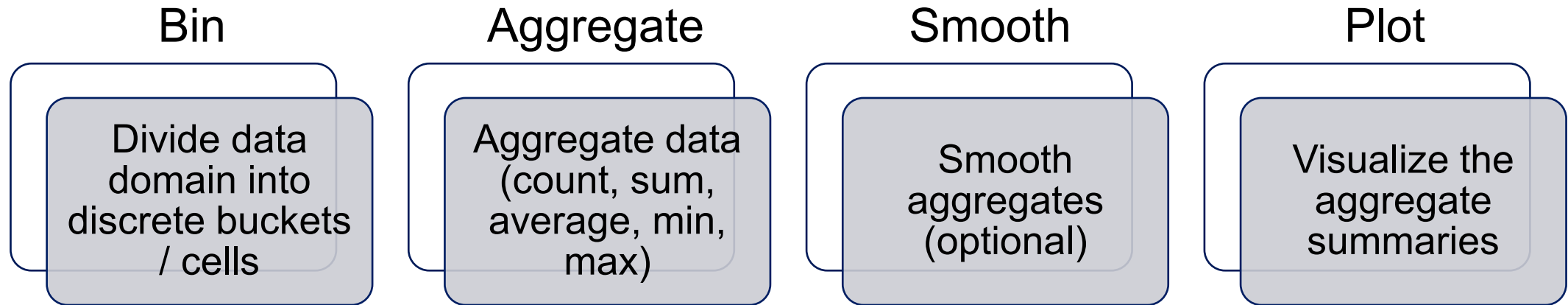
[Liu et al., 2013]

Data reduction techniques



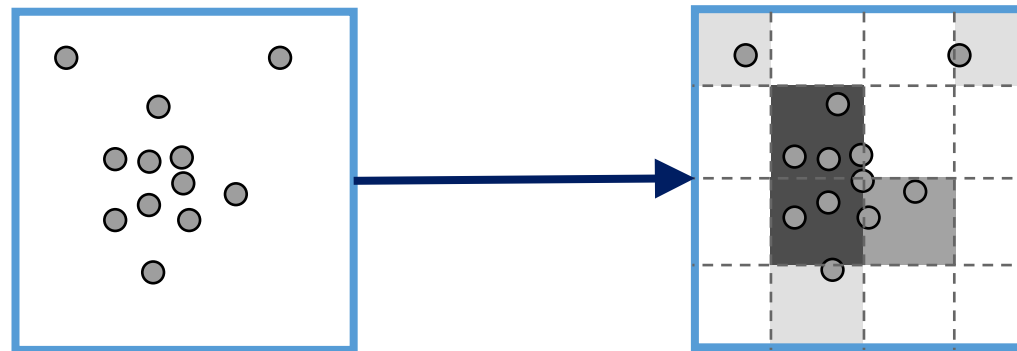
[Liu et al., 2013]

Data binning



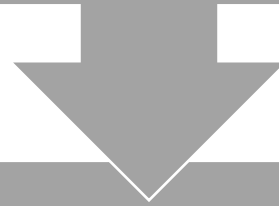
Binning

- Divide data domain into discrete buckets / cells
 - Categorical data: already discrete
 - Numbers: choose bin intervals
 - Temporal: choose time unit (hour, day, month, ...)
 - Spatial: bin x, y coordinates after cartographic projection



Aggregation

Aggregate the data to create a summary, grouped by chosen bins.



Different classes of aggregation functions:

Distributive: functions defined by structural recursion.

Algebraic: functions expressed by finite algebraic expressions defined over distributive functions (decomposable aggregate functions).

Holistic: all other functions

Aggregation

- Distributive:

$$\begin{aligned} \text{COUNT}(x) &= 1 \\ \text{COUNT}(X \uplus Y) &= \text{COUNT}(X) + \text{COUNT}(Y) \end{aligned}$$

$$\begin{aligned} \text{SUM}(x) &= x \\ \text{SUM}(X \uplus Y) &= \text{SUM}(X) + \text{SUM}(Y) \end{aligned}$$

$$\begin{aligned} \text{MAX}(x) &= x \\ \text{MAX}(X \uplus Y) &= \max(\text{MAX}(X), \text{MAX}(Y)) \end{aligned}$$

Aggregation

- Algebraic:

$$AVG(X \uplus Y) = \frac{SUM(X) + SUM(Y)}{COUNT(X) + COUNT(Y)}$$

$$SUM(X^2 \uplus Y^2) = SUM(X^2) + SUM(Y^2)$$

$$STDDEV(x) = \sqrt{\frac{SUM(x^2)}{COUNT(x)} - AVG(x)^2}$$

$$STDDEV(X \uplus Y) = \sqrt{\frac{SUM(X \uplus Y)}{COUNT(X \uplus Y)} - AVG(X \uplus Y)^2}$$

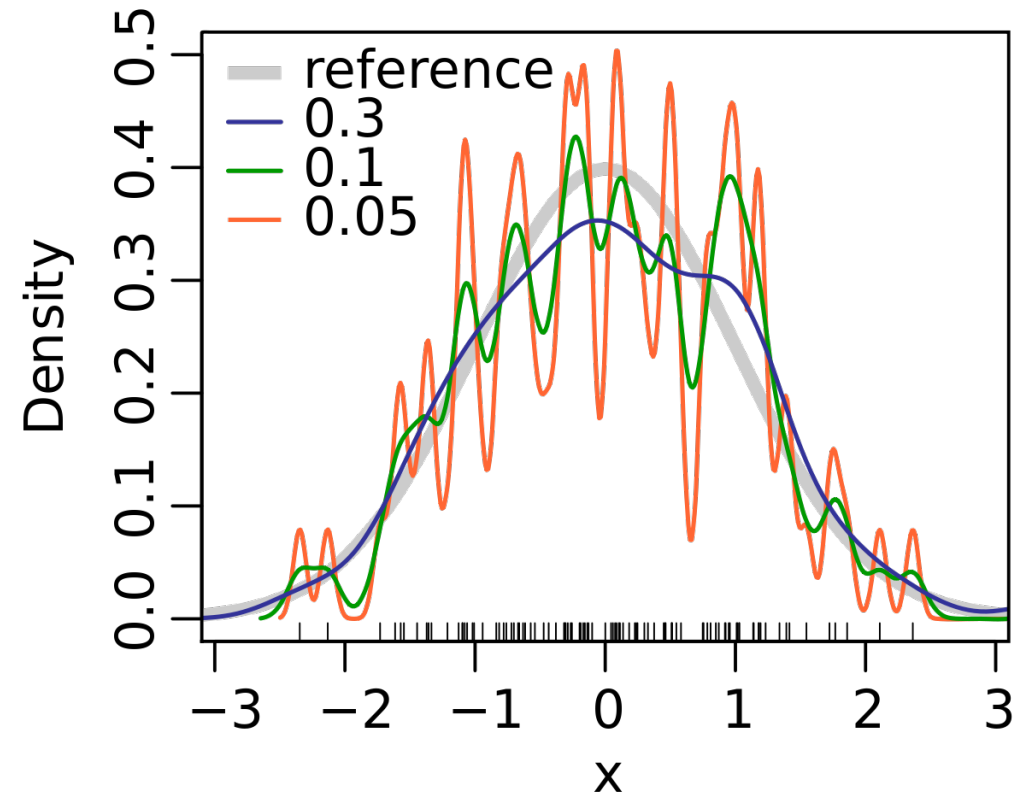
$$\begin{aligned}\sigma &= \sqrt{E[(X - \mu)^2]} \\ &= \sqrt{E[X^2] - (E[X])^2}\end{aligned}$$

Aggregation

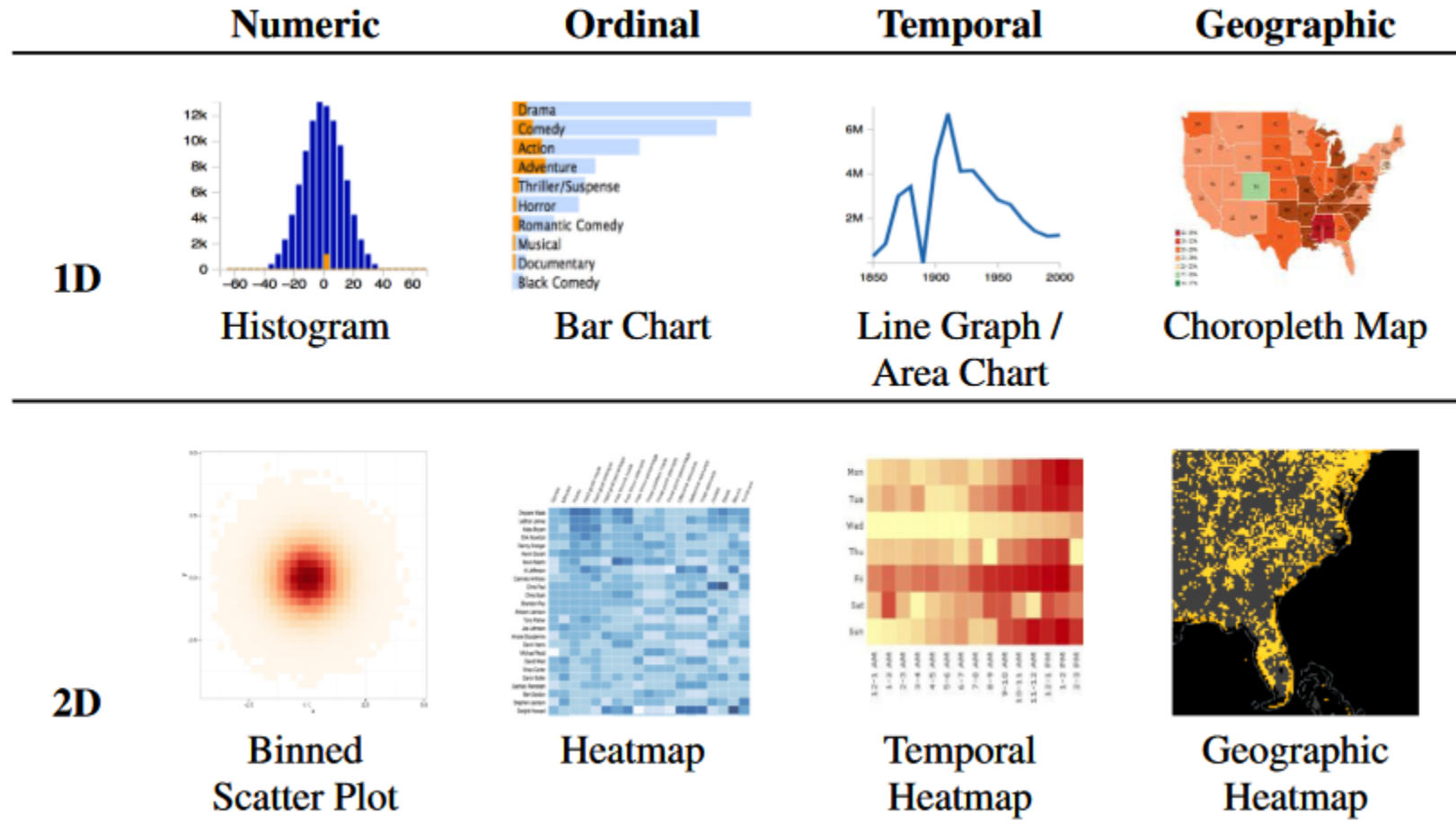
- Holistic: all other functions, no bound on the size of the storage needed to describe sub-aggregate.
 - Most frequent
 - Median
 - ...

Smoothing

- Perform smoothing (e.g., KDE) on aggregated data to better approximate underlying continuous density:
 - Large bandwidth: coarse density with little details.
 - Small bandwidth: too much detail and not general enough to cover new or unseen examples.

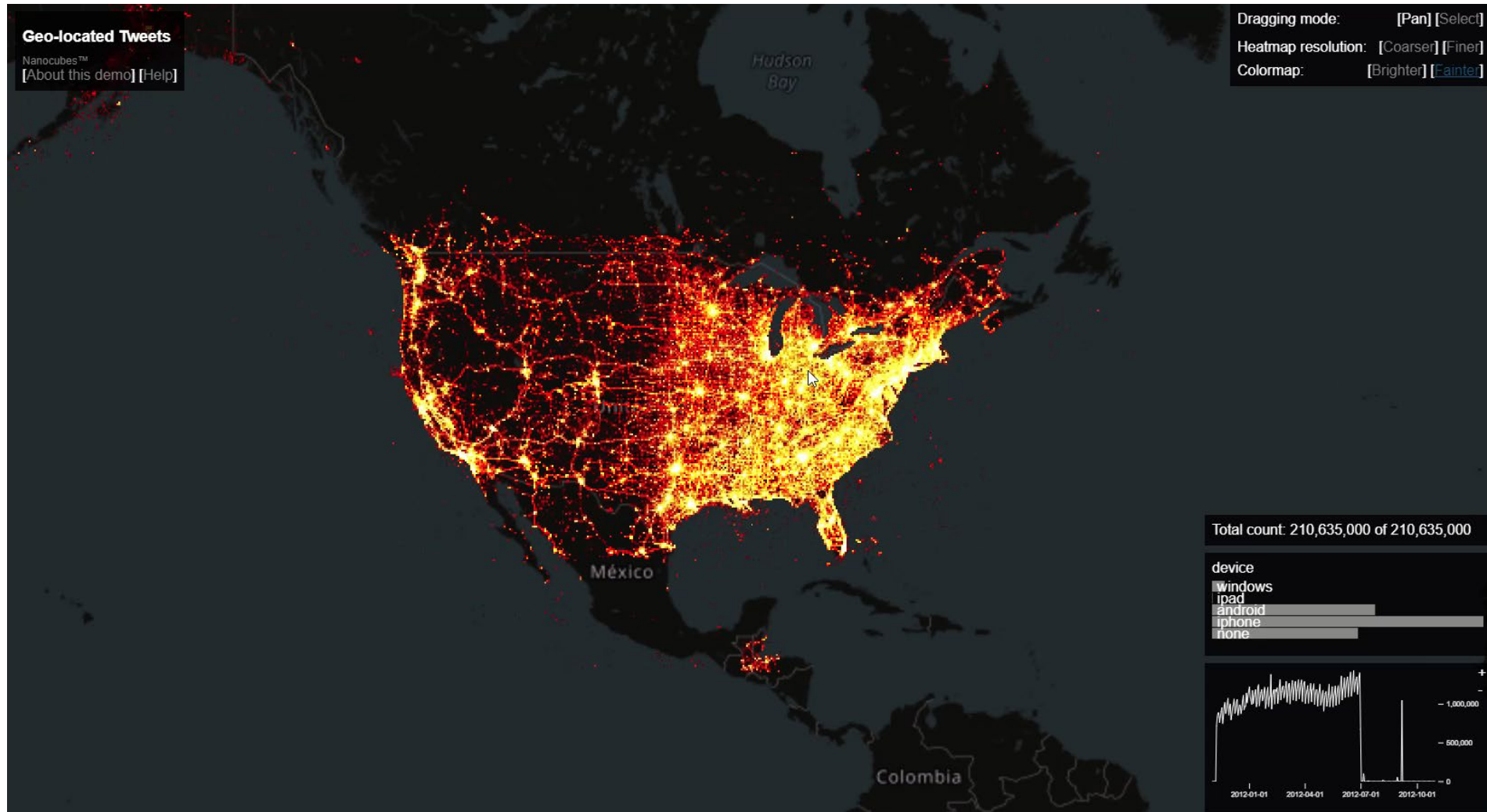


Visualization designs for binned plots

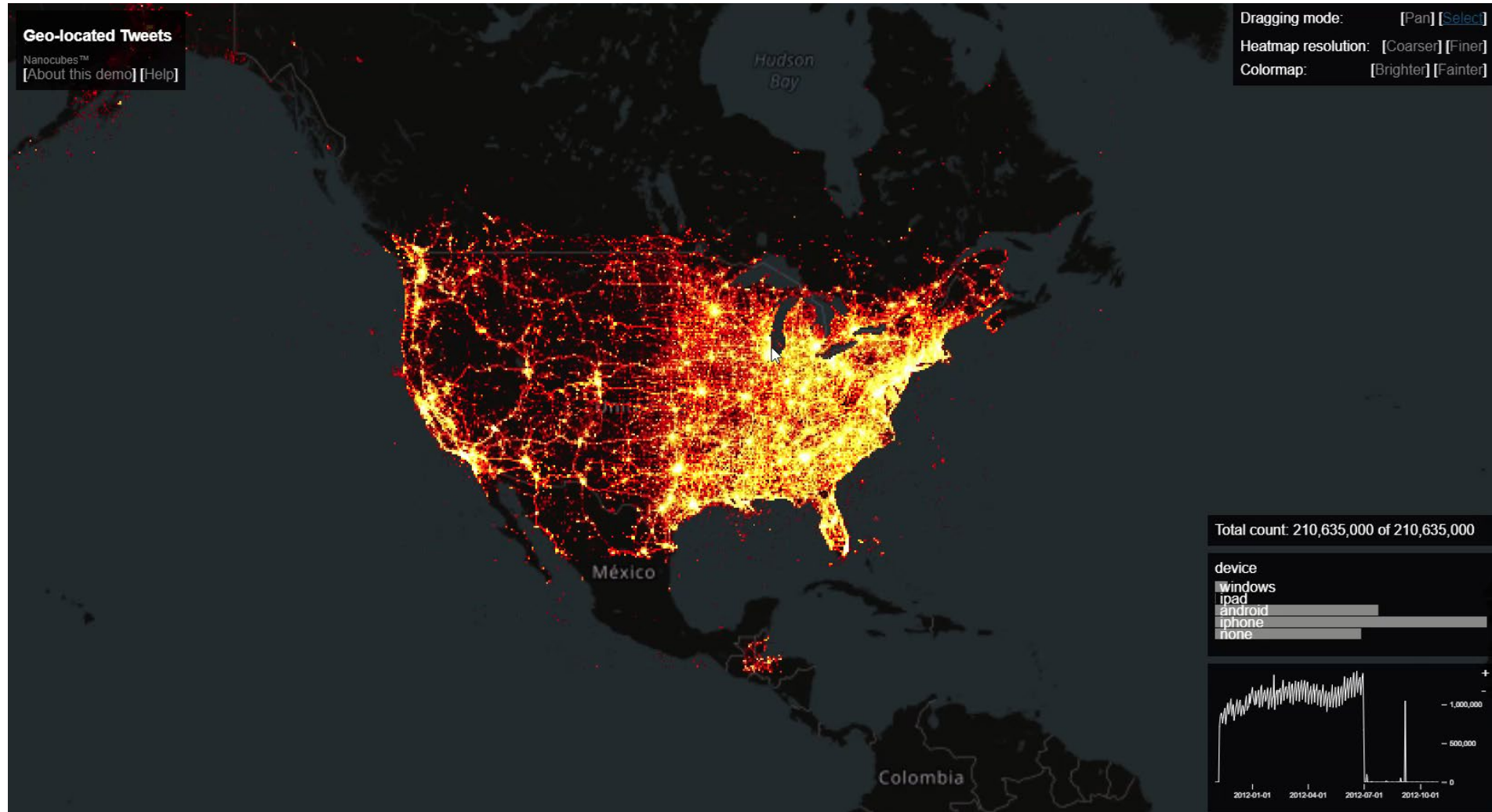


[Liu et al., 2013]

Visualization requirements: brushing & linking

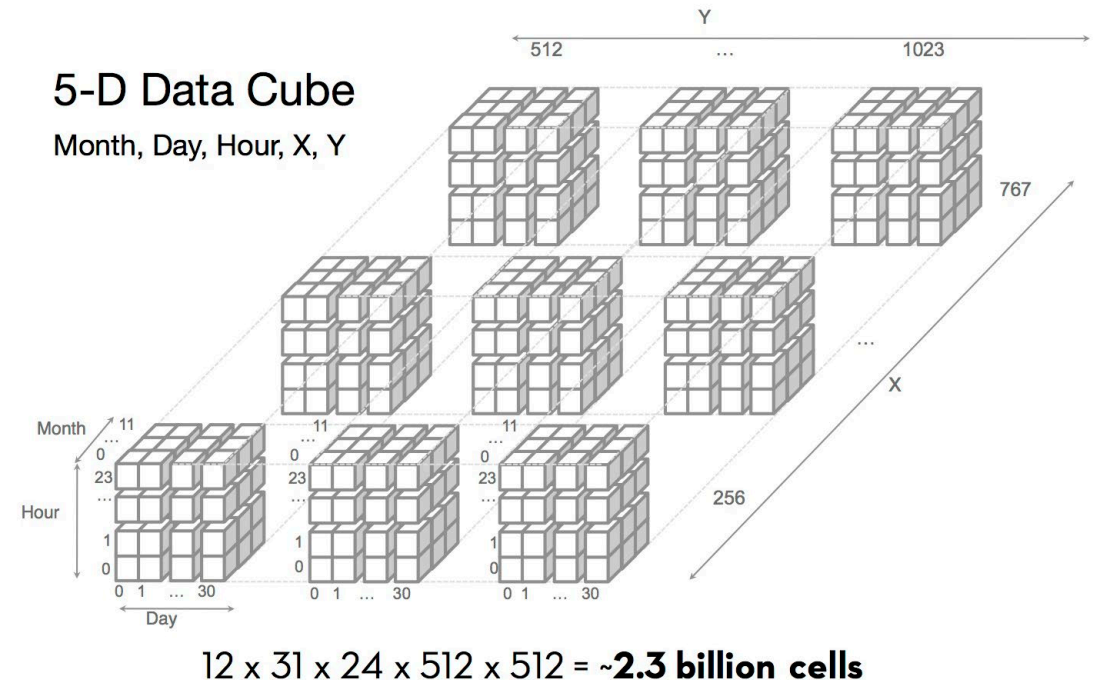


Visualization requirements: spatial brushing & linking



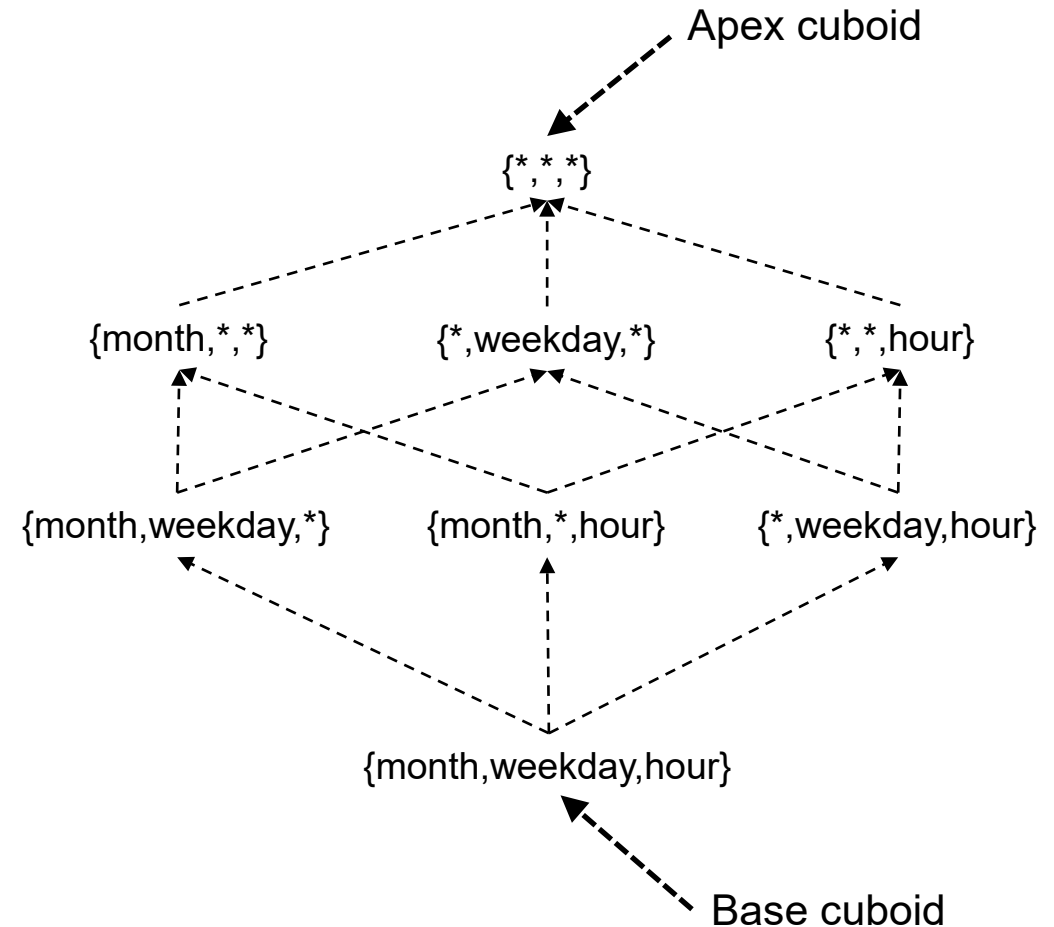
Datacube

- A datacube stores *all* combinations of bins across dimensions.
- Considering the following binning schema:
 - Temporal: 12 (months), 31 (days), 24 (hours)
 - Spatial: 512 x 512
 - Total: ~2.3 billion cells (!!!)



Datacube

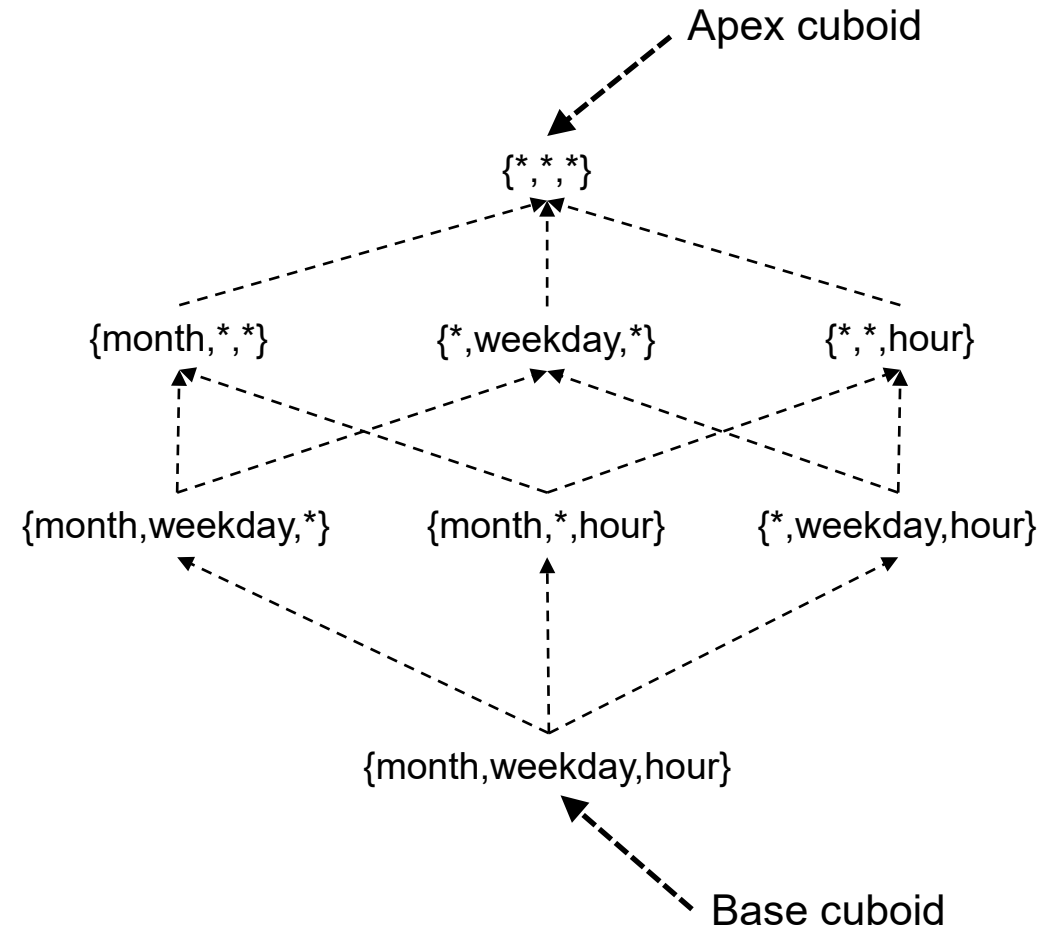
- Datacube can be viewed as a lattice of cuboids:
 - Top cuboid contains only one cell.
 - Base cuboid contains all cells.
- Materialization:
 - Full: materialize every cuboid.
 - Partial: trade-off between storage space and response time.
 - None: on-the-fly aggregation.



Total number of cuboids: 2^n , with n dimensions

Datacube

- Elements of a dimension can be stored as a hierarchy:
 - Set of parent-child relationships where the parent summarizes its children.
 - Block $\rightarrow$ City $\rightarrow$ State $\rightarrow$ Country



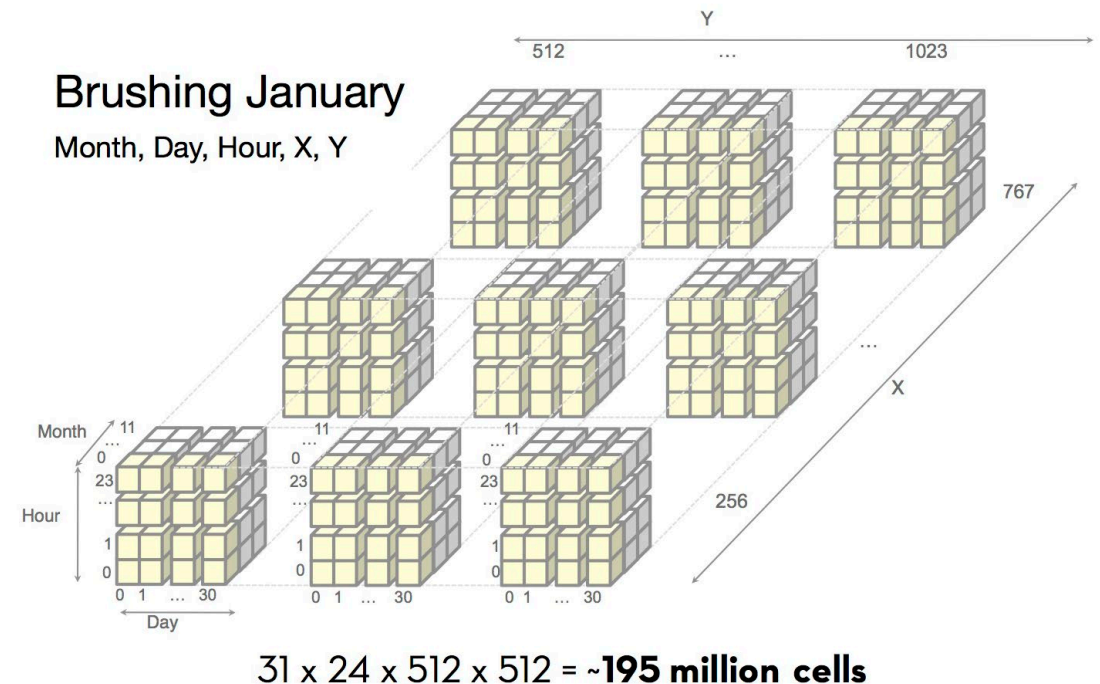
Total number of cuboids: 2^n , with n dimensions

Slice, dice, drill down, roll up

- Slice: selects one particular dimension from a cube, providing a new sub-cube.
- Dice: selects two or more dimensions from a cube, providing a new sub-cube.
- Drill down: descending the hierarchy.
- Roll up: data is aggregated by ascending the hierarchy (e.g., level of a city to the level of a country).

imMens

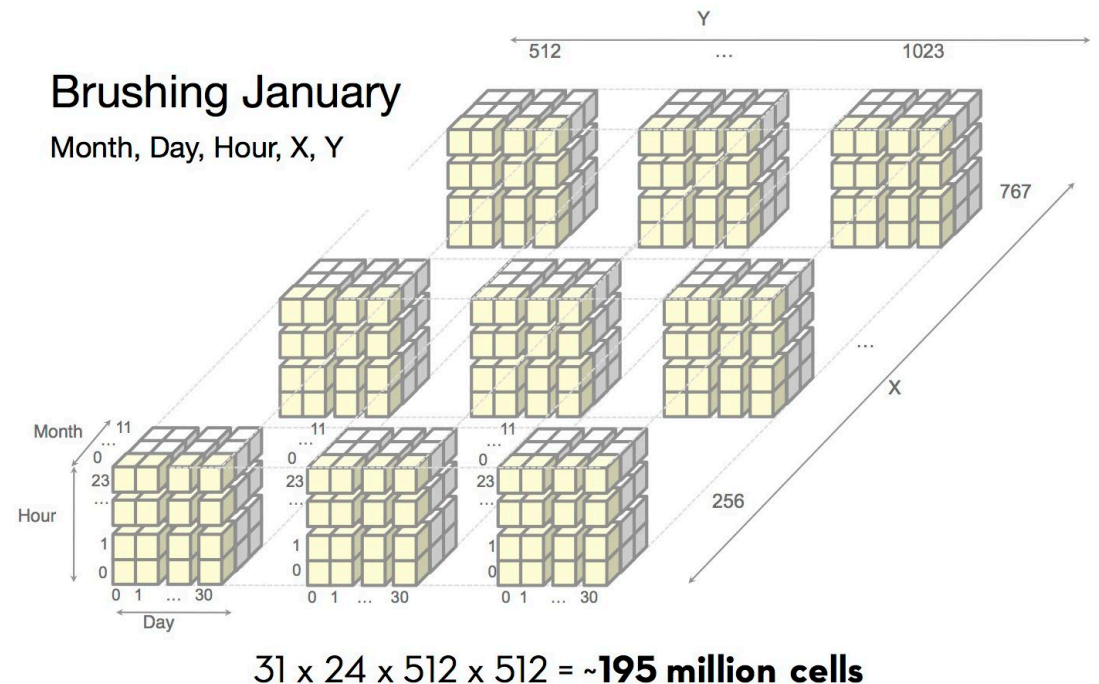
- Performing operations over cuboids can be prohibitively expensive.
 - E.g., Aggregating over the month of January will result in performing computation over 1/12th of the cube.



[Liu et al., 2013]

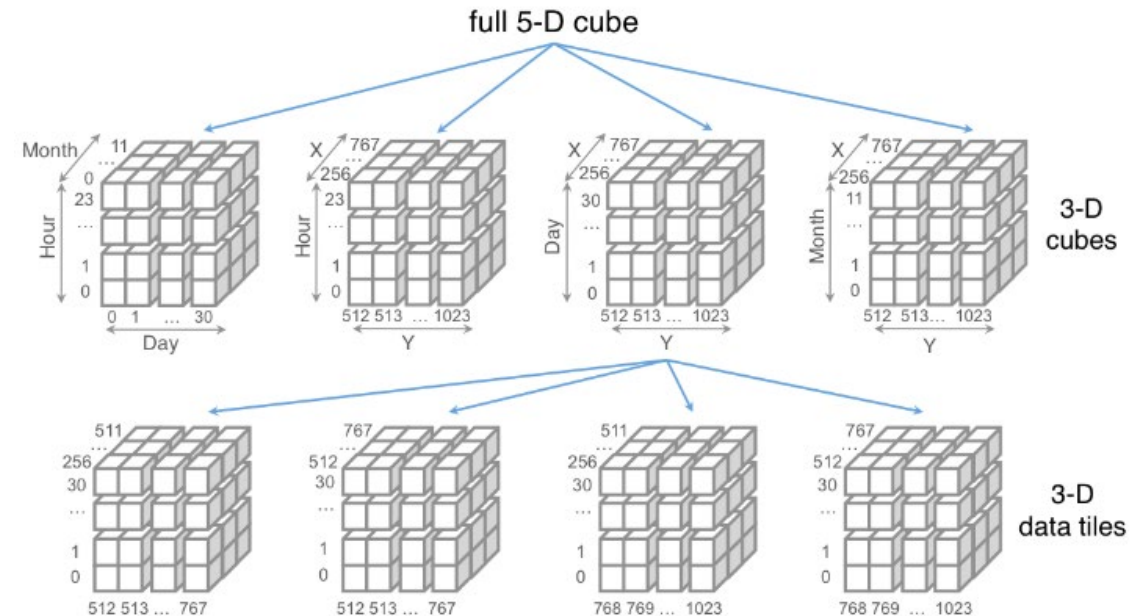
imMens

- imMens addresses this issue by precomputing image tiles with aggregations.
- Like OSM tiles, but now storing data: multivariate data tiles.
- How to solve the combinatorial explosion of multiple dimensions?



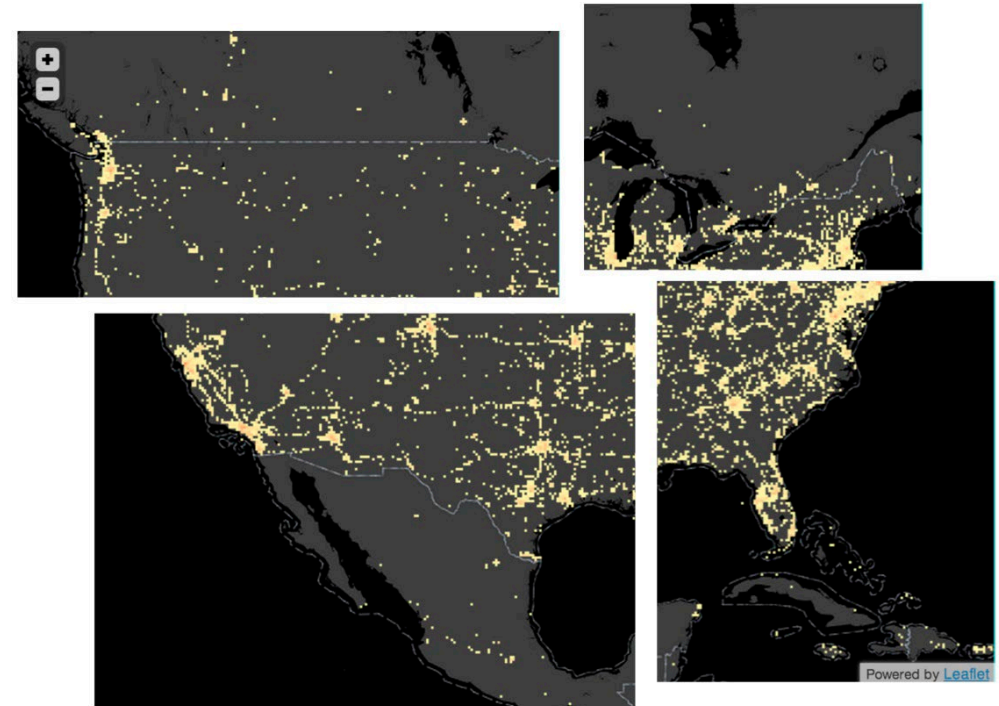
imMens

- For a pair of binned plots, maximum number of dimensions needed to support brushing & linking is four.
 - Between two binned scatterplots that do not share a dimensions.
- Idea: decompose the full cube into a collection of smaller 3- or 4-dimensional projections.



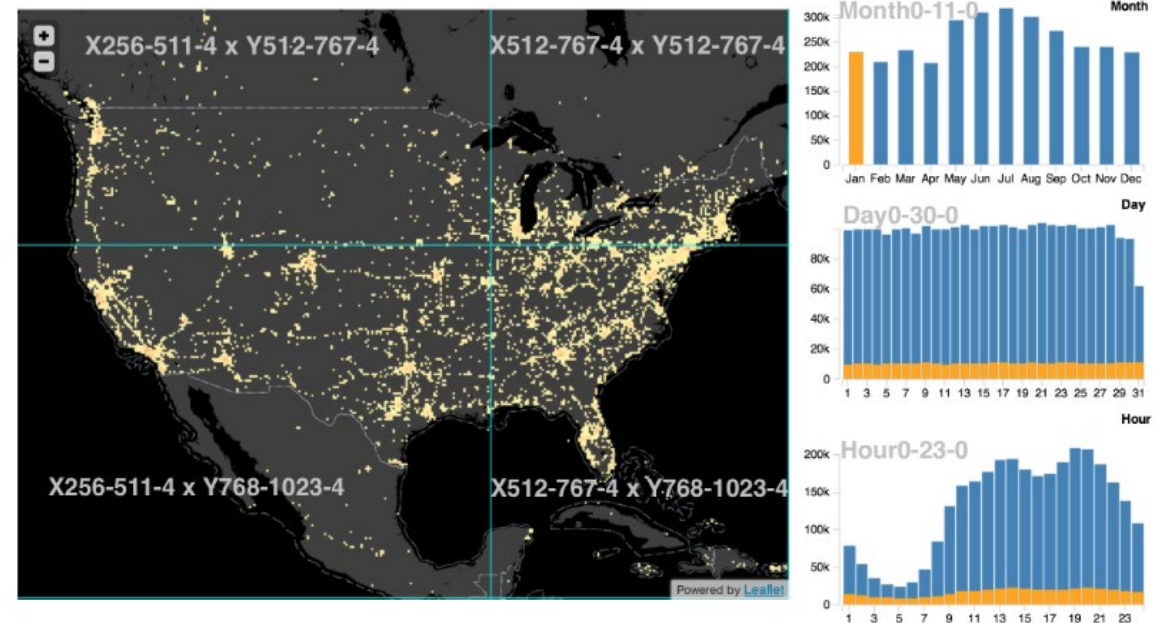
imMens

- Certain dimensions may still require a large number of bins.
 - Spatial dimension: many bins to represent the entire globe.
 - Handle this by breaking up these projections by index ranges (similar to OSM tiles).



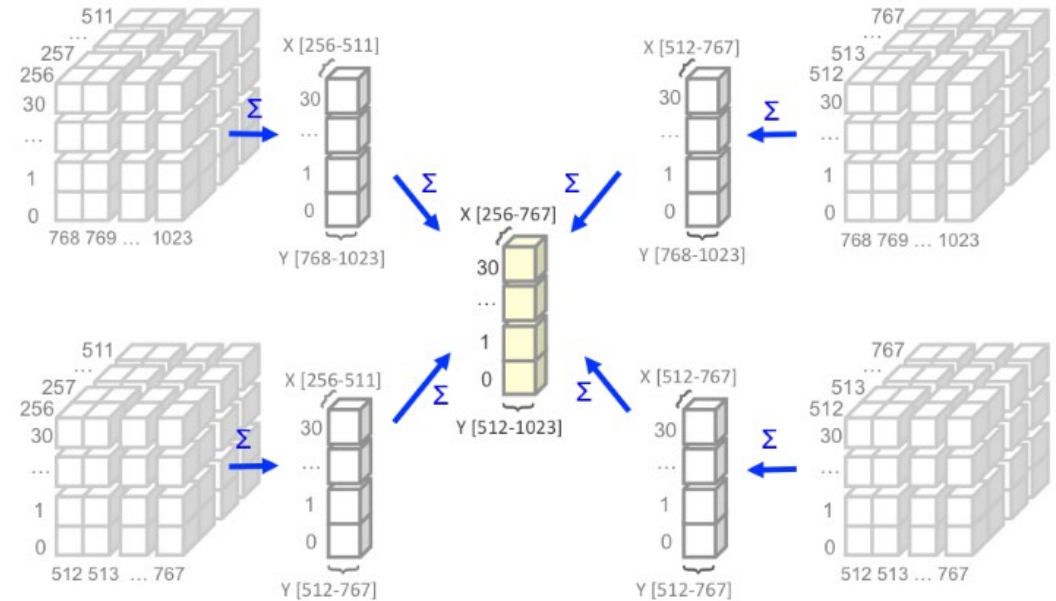
imMens

- Brightkite example:
 - Four dimensions – space, month, day, hour.
 - Space: 4 512x512 tiles.
 - Time: 1 tile each.
 - Supporting brushing & linking:
 - Space – month: 4 x 1 tiles
 - Space – day: 4 x 1 tiles
 - Space – hour: 4 x 1 tiles
 - Month – day – hour: 1 tile
 - Total: 13 tiles

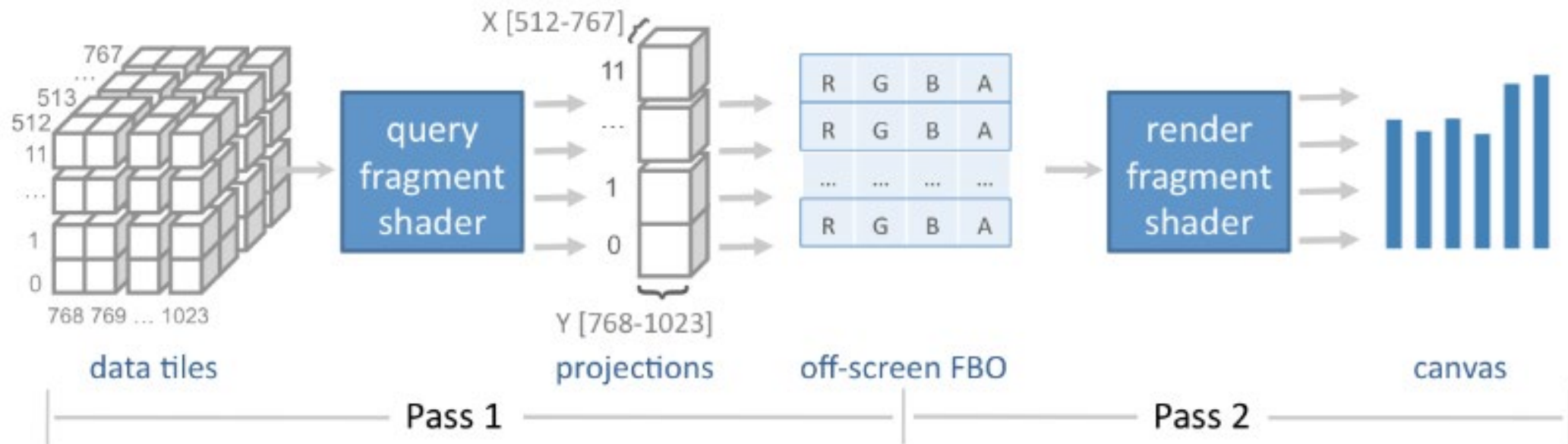


imMens

- Brushing & linking involves aggregating data tiles.
 - User selects a region → compute aggregation → highlight corresponding histograms.



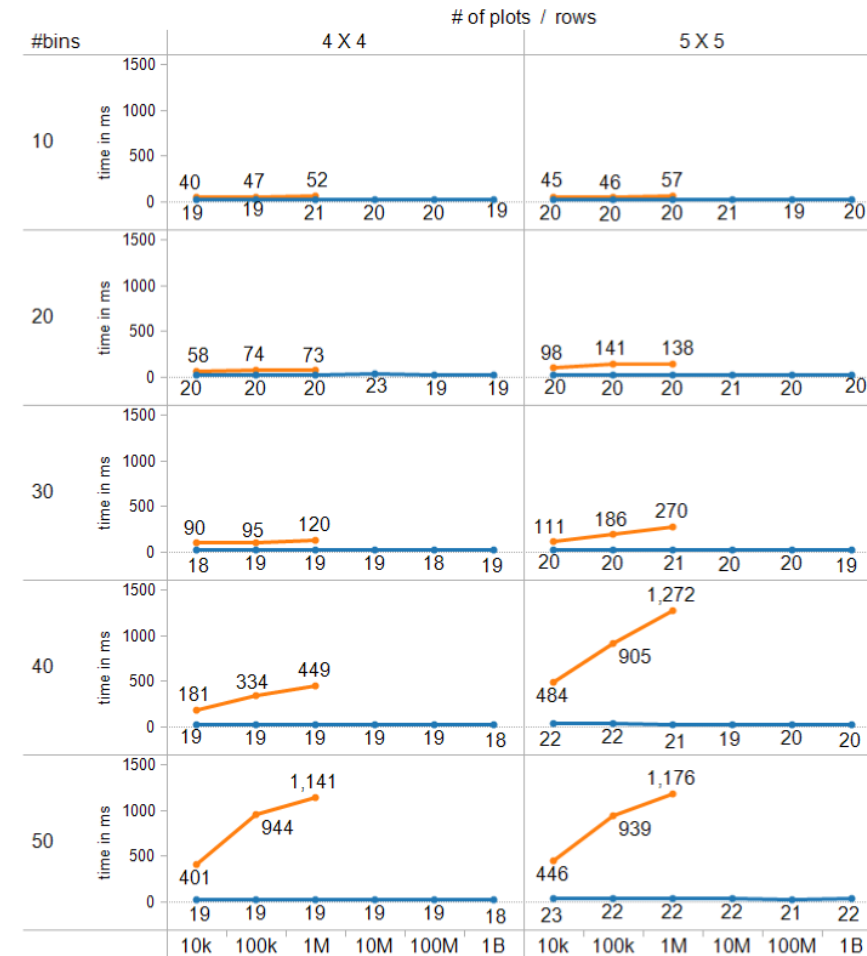
imMens



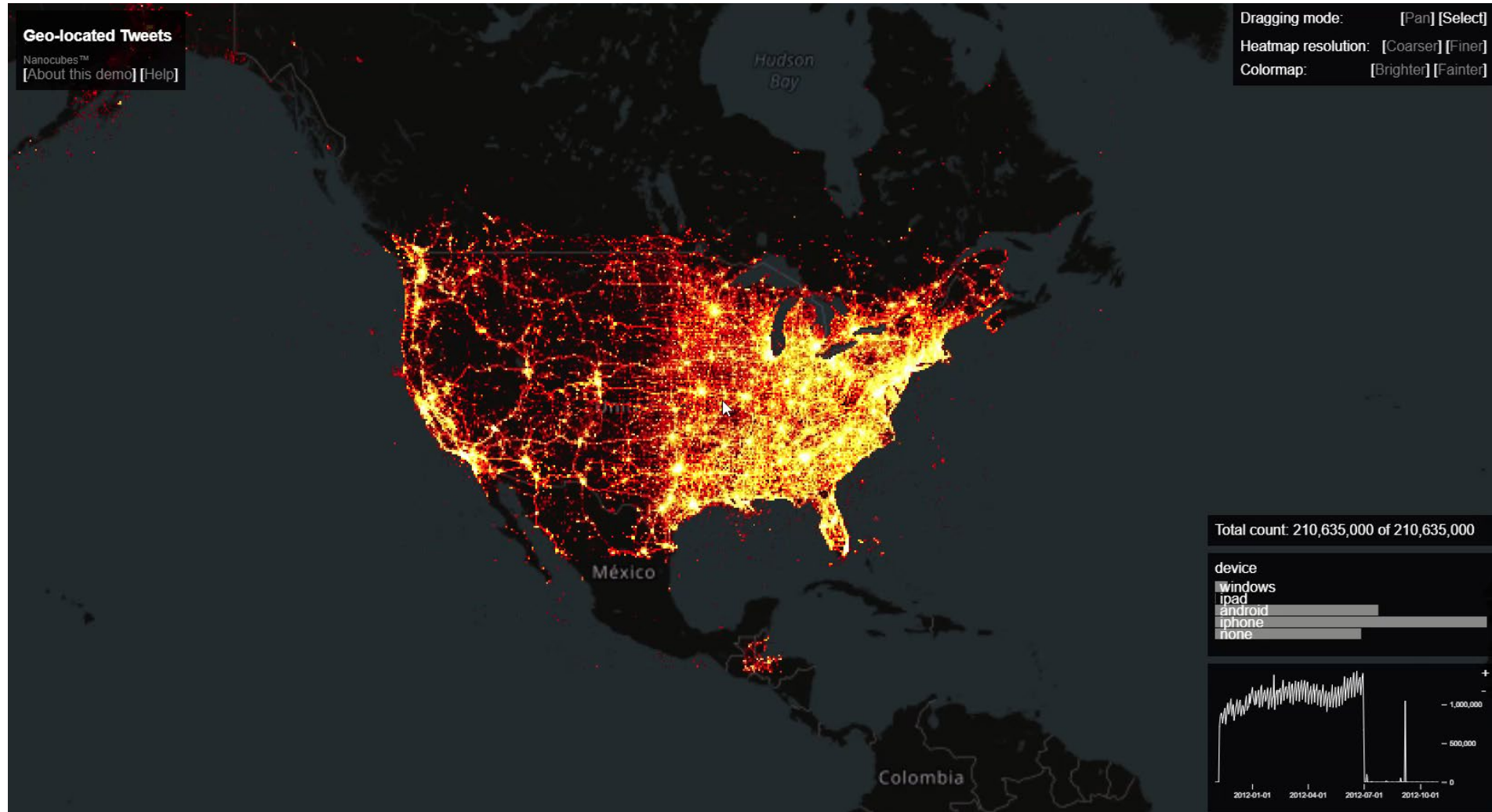
Two-pass approach for parallel data querying and rendering using WebGL fragment shaders.

imMens

Constant query time, even for datasets with 1B data points.

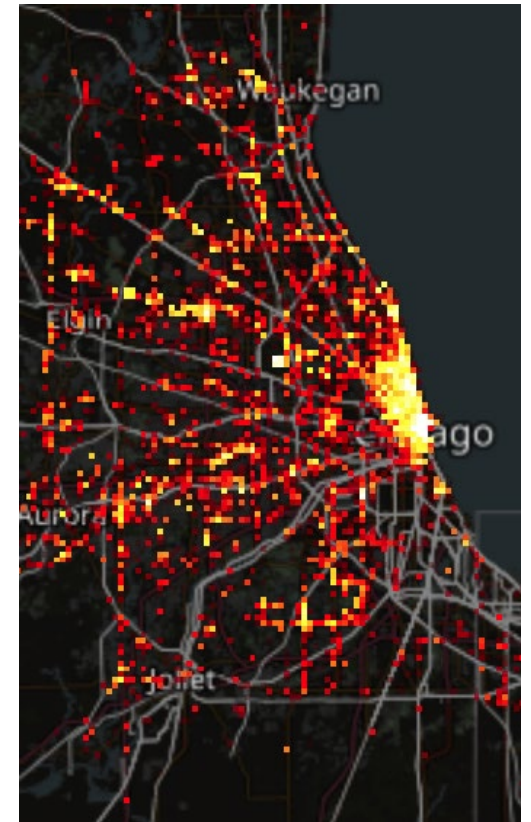
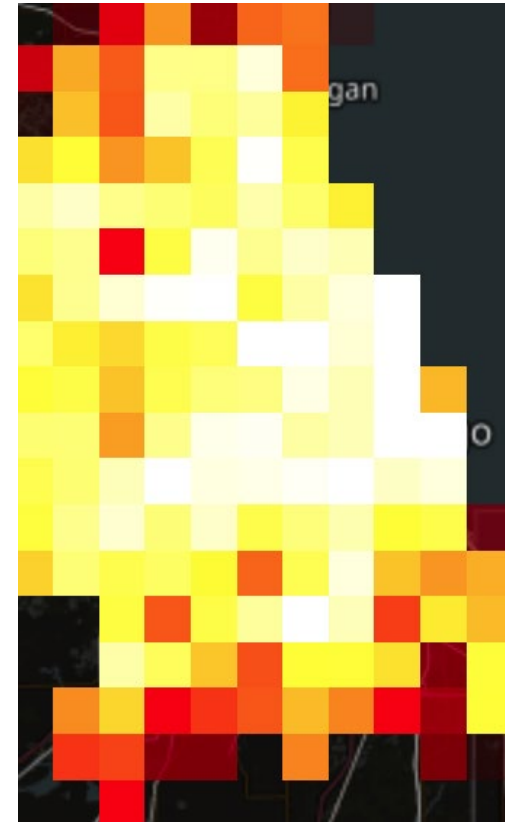


Visualization requirements: panning and zooming



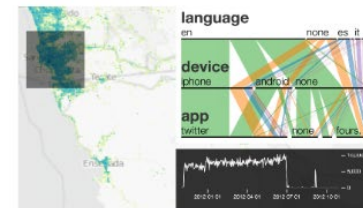
Nanocubes

- How to handle different spatial and temporal resolutions?
 - imMens supports “overview first, zoom and filter, details-on-demand”, but not inside spatiotemporal dimensions.
- Trade-off:
 - Fine resolution (small bins / cells): high memory consumption, but highly detailed.
 - Coarser resolution (large bins / cells): low memory consumption, but no details.

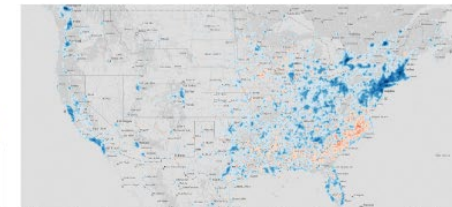


Nanocubes

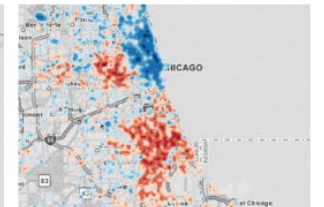
- Data structure that supports drilling down both spatial and temporal dimensions, while maintaining manageable memory usage.
- Sparse data structure that bins and counts data points.
- Important trick: allow for shared links across dimensions, and in the same dimension.



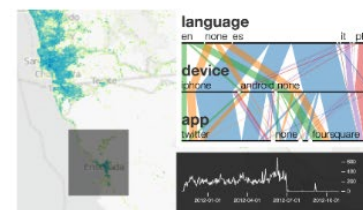
Linked view of tweets in San Diego, US



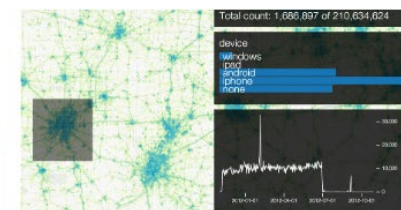
US-wide choropleth map of relative device popularity



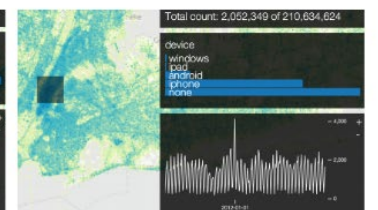
Close-up view of Chicago



Linked view of tweets in Ensenada, Mexico



Superbowl, Indianapolis

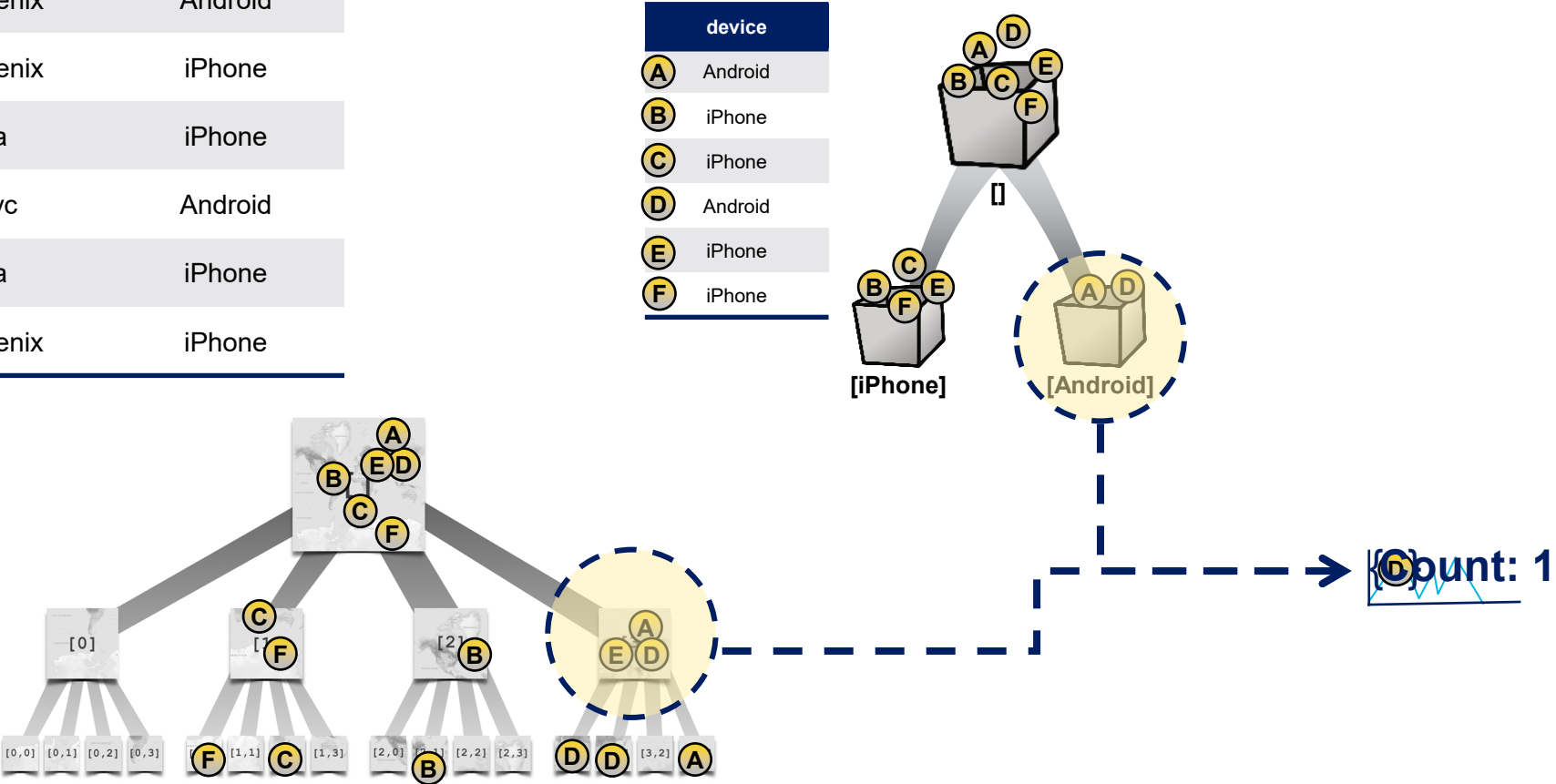


New Year's Eve, Midtown Manhattan

Nanocubes hierarchy

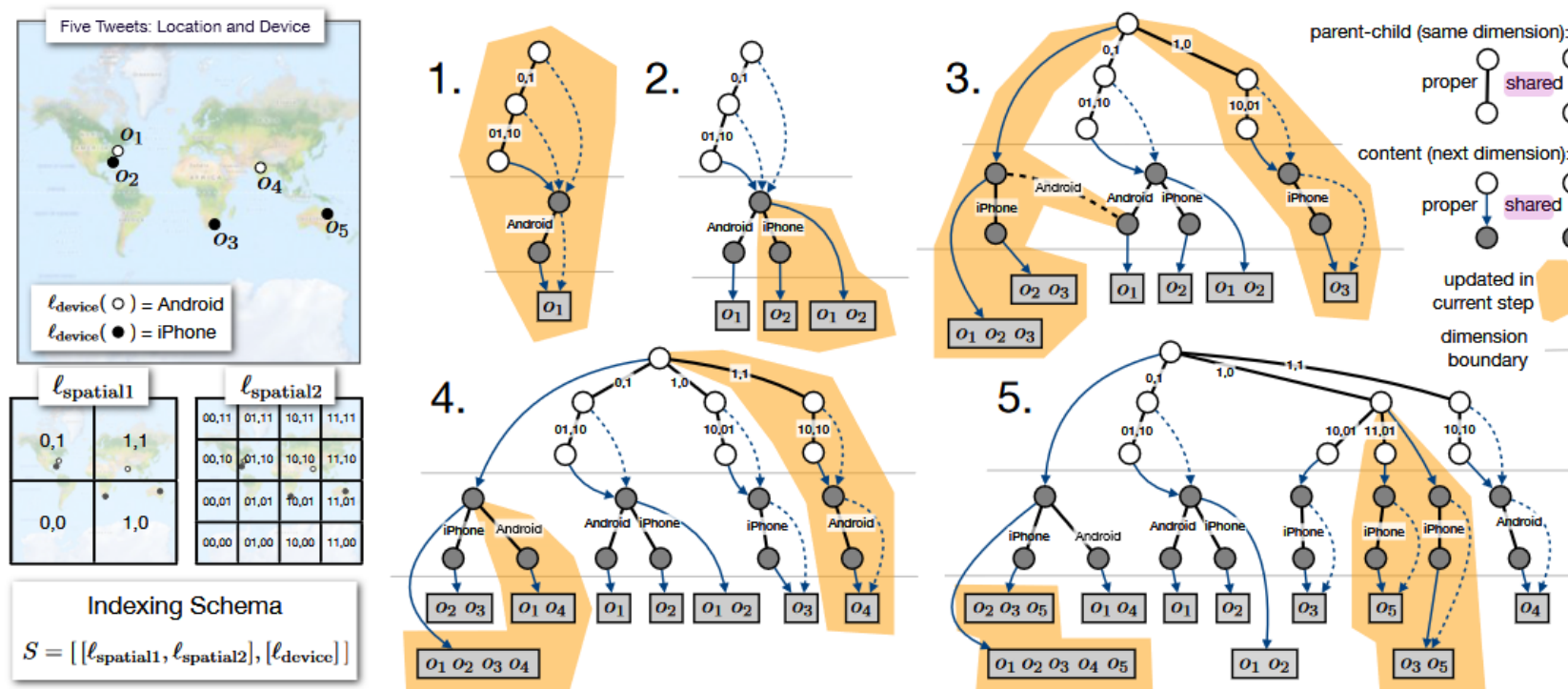
latitude	longitude	keyword	device
42.102908	-73.242852	#phoenix	Android
29.617161	-81.636398	#phoenix	iPhone
23.014051	75.120052	#la	iPhone
26.014051	75.120052	#nyc	Android
28.014051	74.120052	#la	iPhone
23.014051	75.120052	#phoenix	iPhone

	latitude	longitude
A	42.102908	-73.242852
B	29.617161	-81.636398
C	23.014051	75.120052
D	26.014051	75.120052
E	28.014051	74.120052
F	29.61161	-81.63638



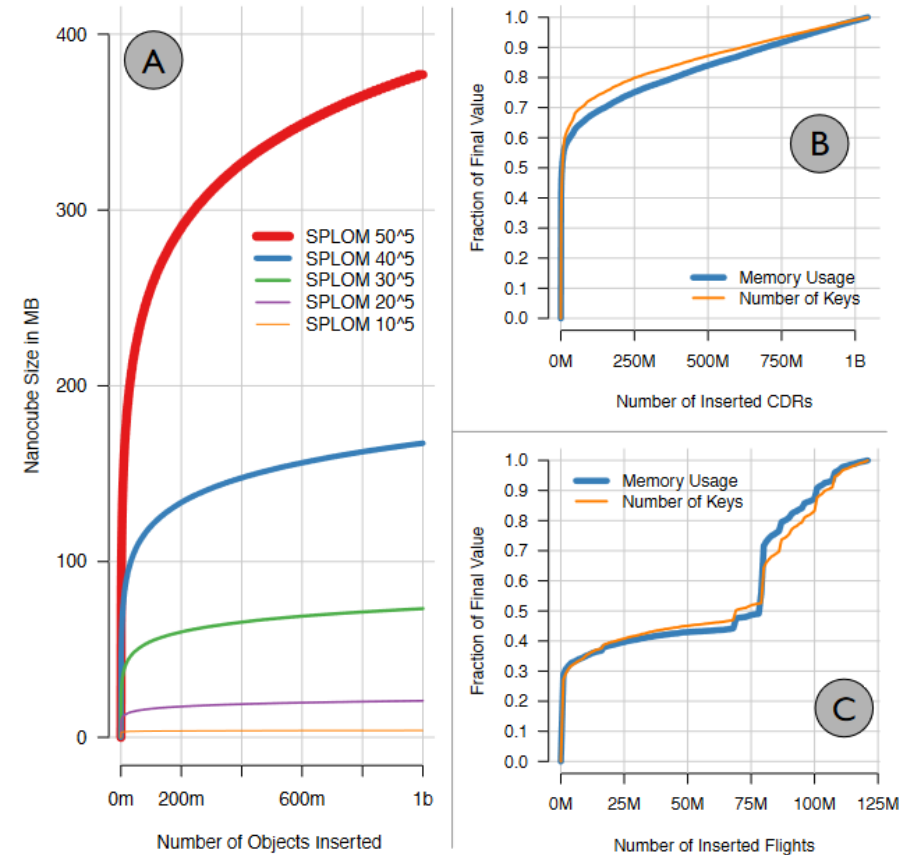
Nanocubes: shared links

- Important trick: allow for shared links across dimensions, and in the same dimension.
- Sharing is responsible for significant memory savings.



Experiments

- Mean query time was $800\mu s$ (less than 1 millisecond!), with a maximum of 12 milliseconds.
- Memory requirements for Twitter dataset (210 million tweets):
 - Without sharing: 37 GB
 - With sharing: 10 GB



Spatiotemporal + keywords data



Spatiotemporal + keywords data

flickr



WIKIPEDIA
The Free Encyclopedia

twitter



Opportunity to not only explore **where** and **when** things happen, but also **what** is happening

Density map Timeseries

Top Keywords

Top-k queries

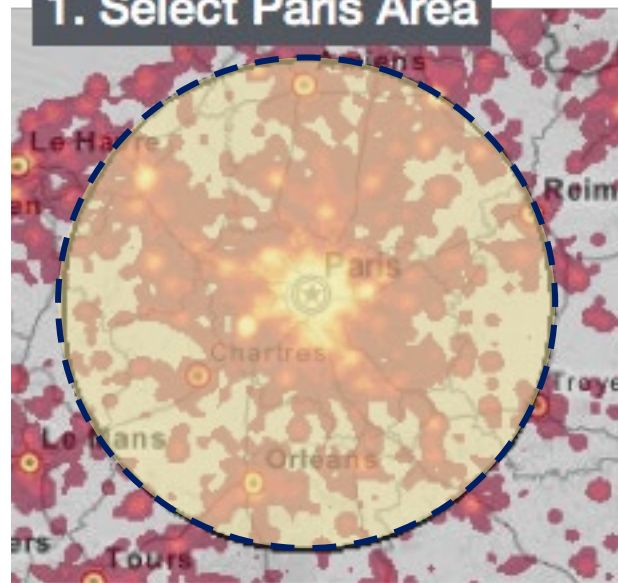
- Top-k: **k** most popular keywords.
 - Compute list of most popular hashtags in a given space and time from a **large** set of data points.
- How a set of keywords changes over space and time?
 - Where and when certain hashtags are popular.
- How the volume of keywords compare with each other?
 - #patriots vs #giants.

Top-k: what is trending?

latitude	longitude	time	keyword
42.102908	-73.242852	Sept 29 2016 00:00	#patriots
29.617161	-81.636398	Sept 29 2016 00:05	#giants
-21.527420	32.493101	Sept 29 2016 00:10	#jets
26.014051	75.120052	Sept 29 2016 00:22	#patriots
-22.698453	145.080990	Sept 29 2016 00:31	#giants

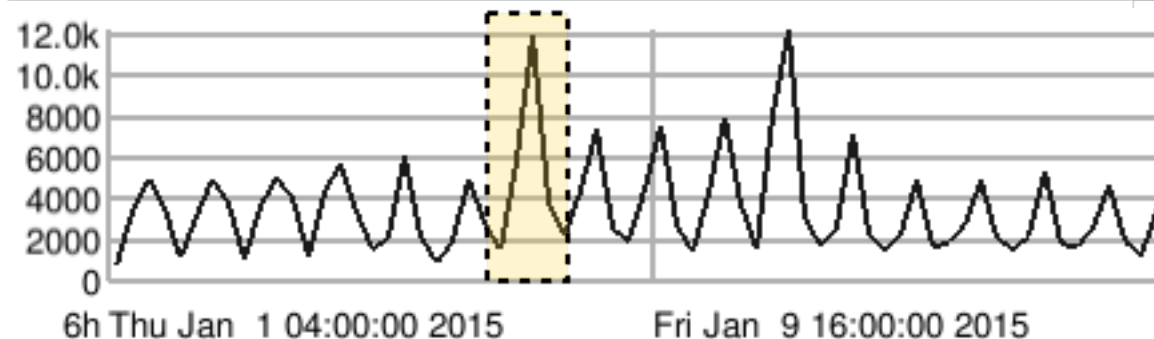
Top-k: what is trending?

1. Select Paris Area



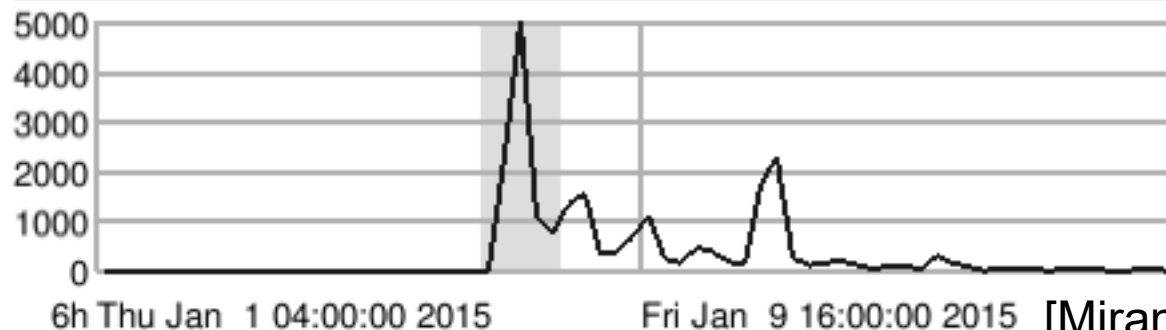
1. #jesuischarlie	4,456
2. #charliehebdo	4,190
6. #charliehebdo	418
9. #noussoumescharlie	197

2. Observe Uncommon Spike on Wed. Jan 7, 2015



3. Select this Spike and Observe Top-10 Hashtags

4. Select Charlie Hebdo's Top Hashtags and Observe its Temporal Volume Pattern

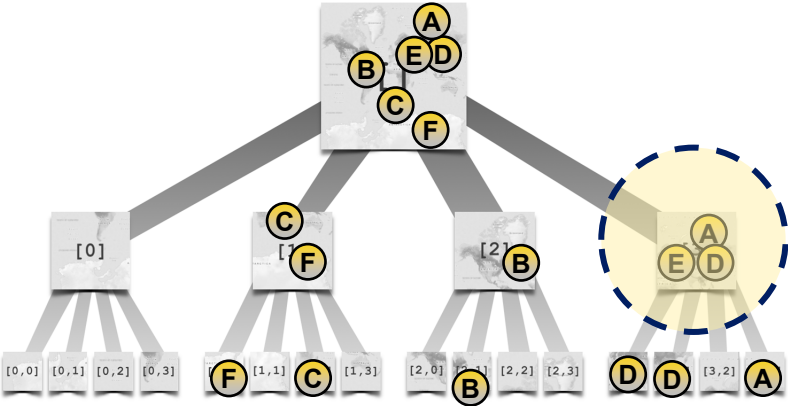


[Miranda et al., 2018]

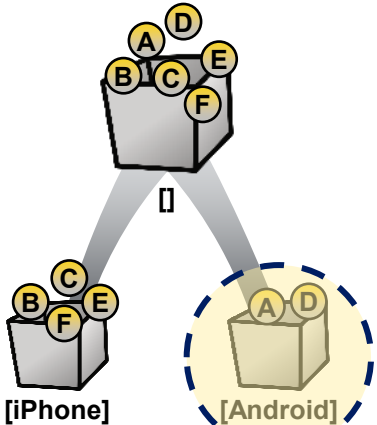
TopKube: A rank-aware datacube

latitude	longitude	keyword	device
42.102908	-73.242852	#phoenix	Android
29.617161	-81.636398	#phoenix	iPhone
23.014051	75.120052	#la	iPhone
26.014051	75.120052	#nyc	Android
28.014051	74.120052	#la	iPhone
23.014051	75.120052	#phoenix	iPhone

	latitude	longitude
A	42.102908	-73.242852
B	29.617161	-81.636398
C	23.014051	75.120052
D	26.014051	75.120052
E	28.014051	74.120052
F	29.61161	-81.63638



device
A Android
B iPhone
C iPhone
D Android
E iPhone
F iPhone

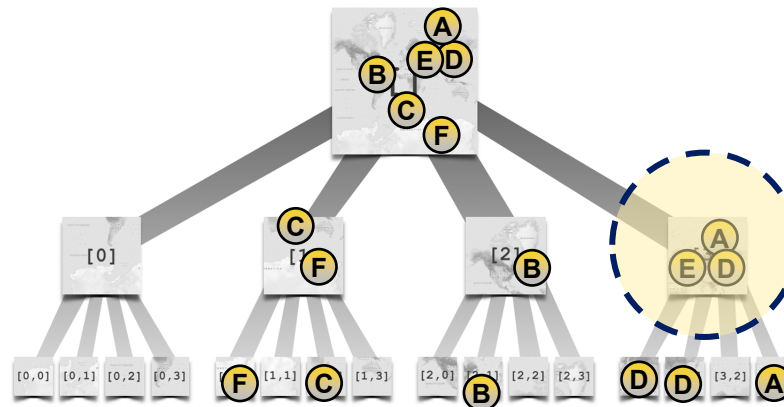


K	c	p
10	1	1
22	2	2
15	0	0

TopKube: A rank-aware datacube

- Following data cube model, aggregate every record along a hierarchy of bins.
- The data structure is a mapping of bins to a pre-computed summary (e.g. count, timeseries).
- In this case, use a ranking summary.

	latitude	longitude
A	42.102908	-73.242852
B	29.617161	-81.636398
C	23.014051	75.120052
D	26.014051	75.120052
E	28.014051	74.120052
F	29.61161	-81.63638



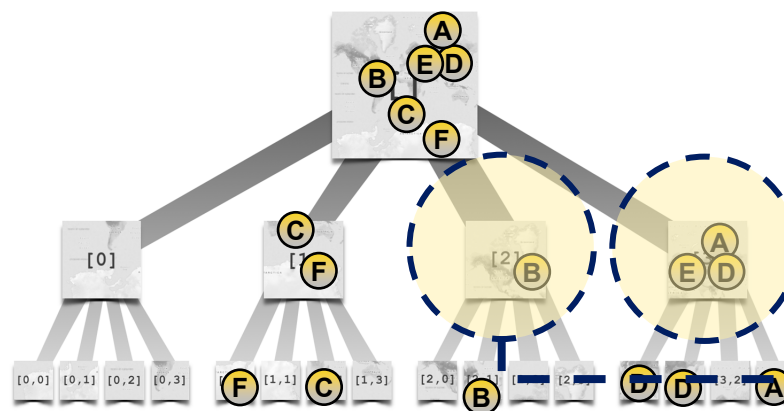
latitude	longitude	keyword
42.102908	-73.242852	#phoenix
29.617161	-81.636398	#phoenix
23.014051	75.120052	#la
26.014051	75.120052	#nyc
28.014051	74.120052	#la
23.014051	75.120052	#phoenix

K	c	p
0	10	1
1	22	2
2	15	0

TopKube: A rank-aware datacube

- Following data cube model, aggregate every record along a hierarchy of bins.
- The data structure is a mapping of bins to a pre-computed summary (e.g. count, timeseries).
- In this case, use a ranking summary.

	latitude	longitude
A	42.102908	-73.242852
B	29.617161	-81.636398
C	23.014051	75.120052
D	26.014051	75.120052
E	28.014051	74.120052
F	29.61161	-81.63638

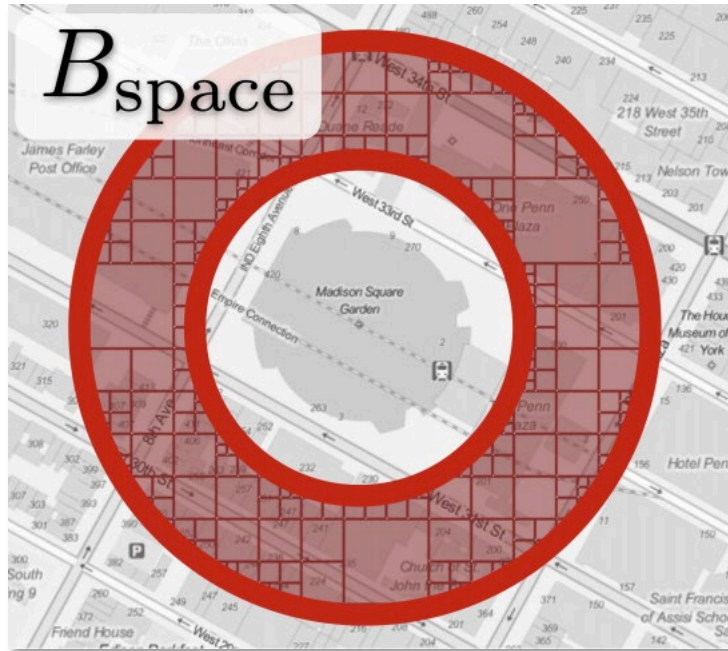


latitude	longitude	keyword
42.102908	-73.242852	#phoenix
29.617161	-81.636398	#phoenix
23.014051	75.120052	#la
26.014051	75.120052	#nyc
28.014051	74.120052	#la
23.014051	75.120052	#phoenix

K	c	p
0	10	1
1	22	2
2	15	0

K	c	p
0	10	1
1	22	2
2	15	0

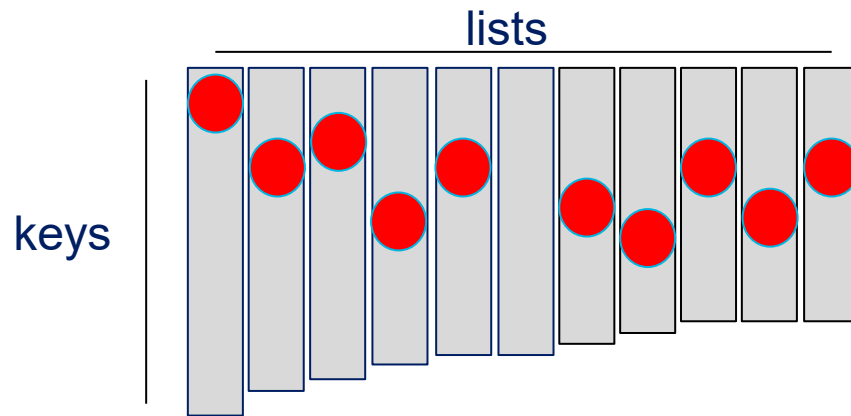
TopKube: A rank-aware datacube



- Spatial and temporal selections usually result in several (thousands) ranking summaries.
- How to efficiently merge them?

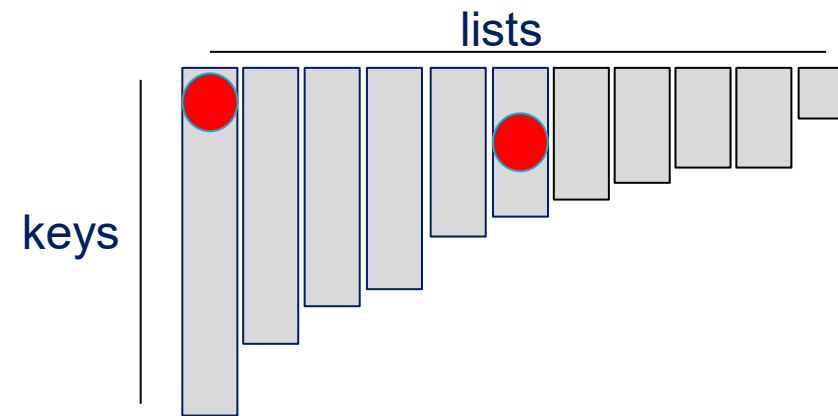
Top-k from ranked lists

Threshold Algorithm [Fagin et al., 2003]



Ideal scenario:

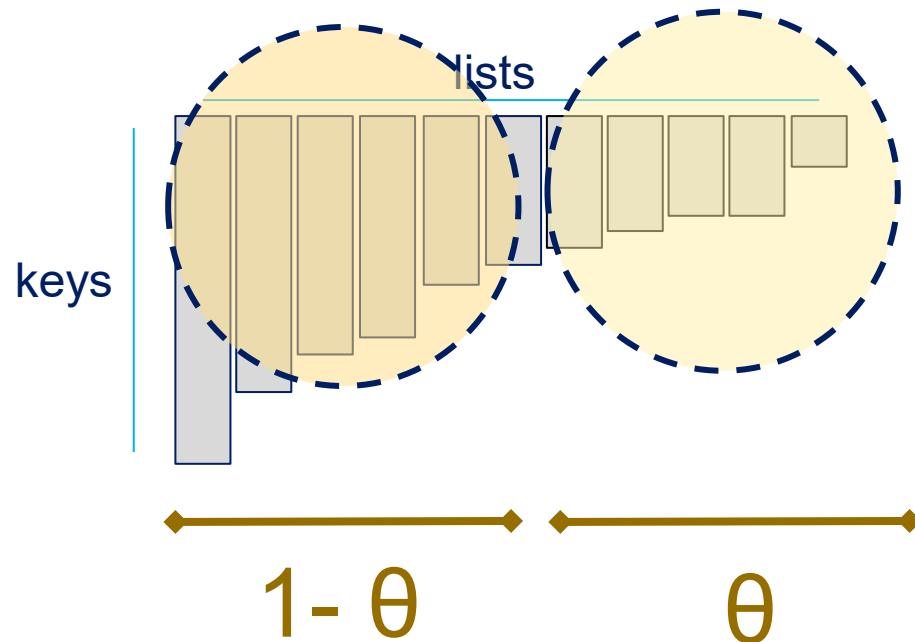
- A key is in almost all lists
- Low number of misses



Most common scenario:

- A key is in very few lists
- Large number of misses

Top-k from ranked lists



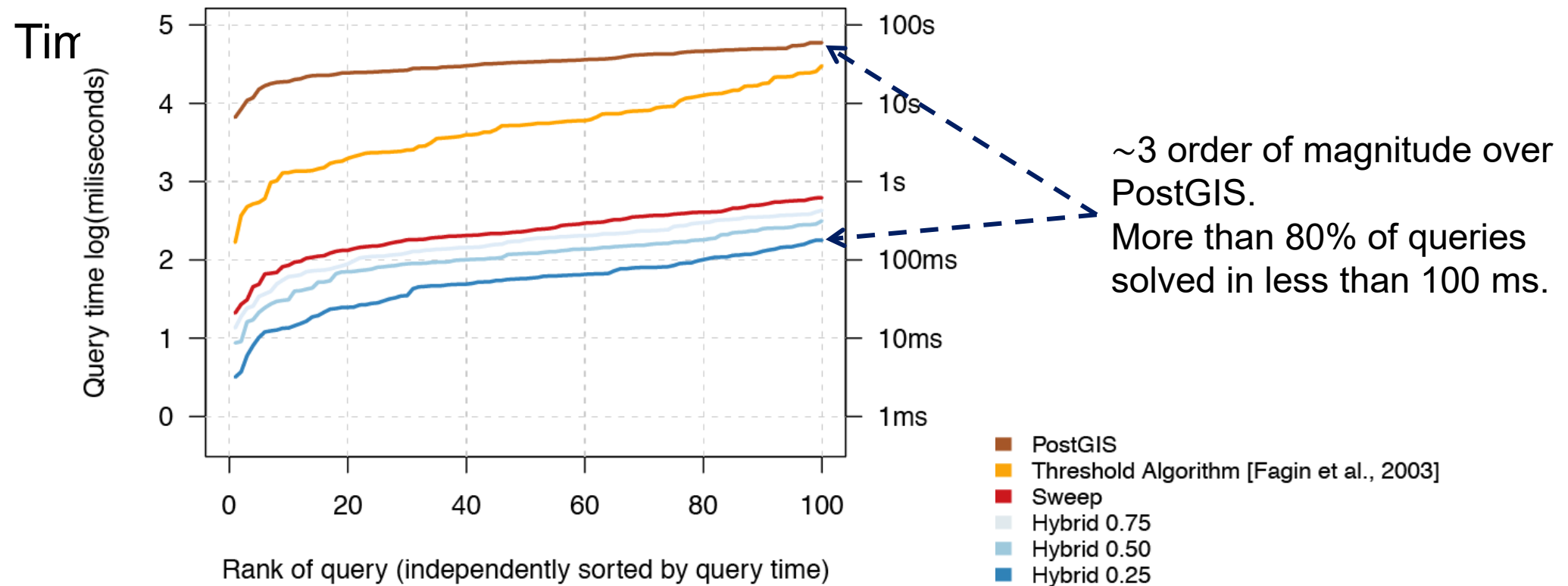
TopKube:

- Run Sweep on α smallest (easiest) problems.
- Run Fagin's Threshold Algorithm on the denser (harder) problems.

Goal:

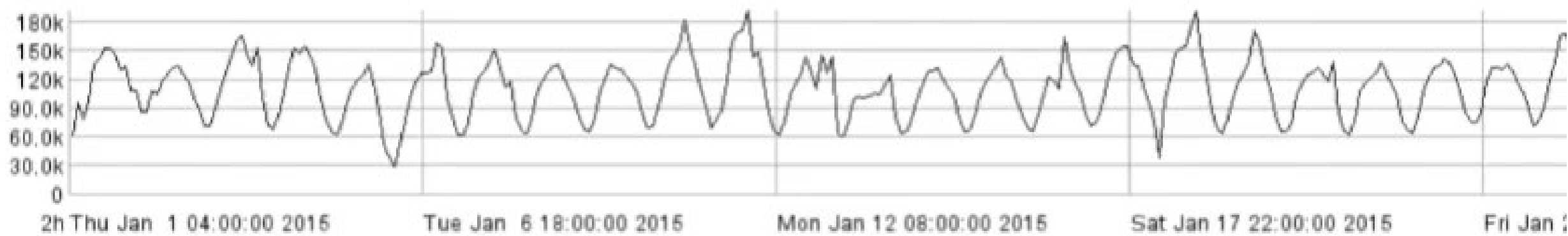
- Increase key density.
- Decrease the number of wasted searches.

TopKube





#np	101,702
#love	96,223
#healthcare	93,449
#nowplaying	85,525
#vscocam	84,024
#lrt	74,398
#jesuischarlie	70,950
#askbelieber	69,622
#trndnl	68,686
#selfie	62,228
#melopidoenfnac	61,067
#photo	59,745
#endomondo	59,528
#camfollowme	59,514
#followmecaniff	58,705
#tbt	57,907
#askdirectioner	56,377
#askdirectioner	56,377



Sensor data



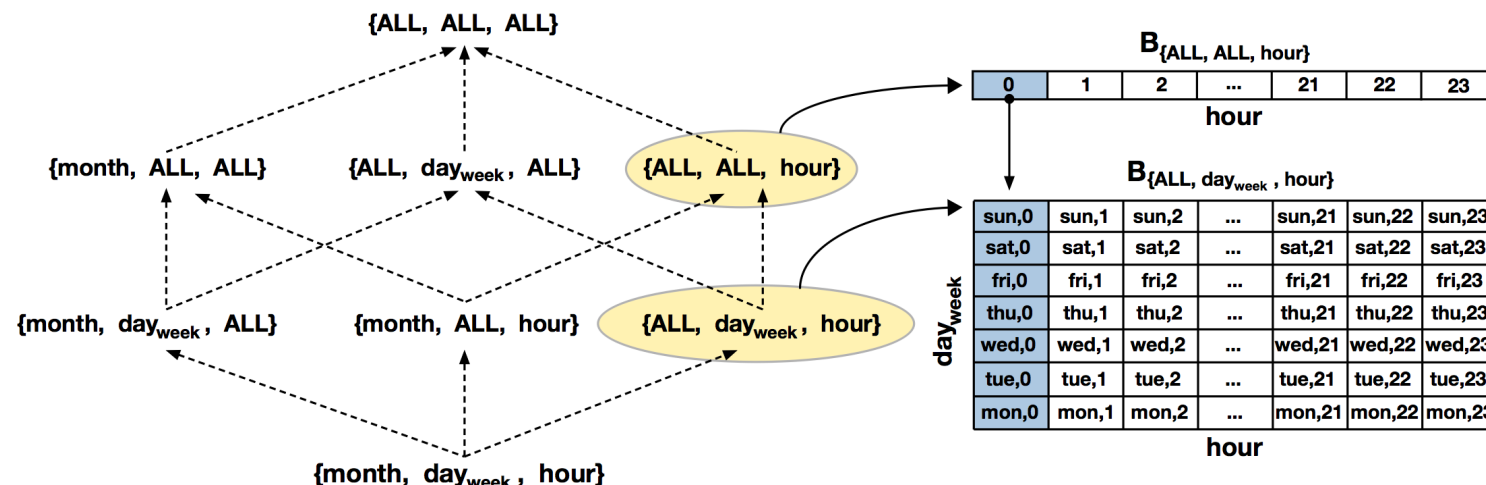
45 sensors
3 boroughs
34 years of high-resolution decibel
data (if combined)

Time Lattice

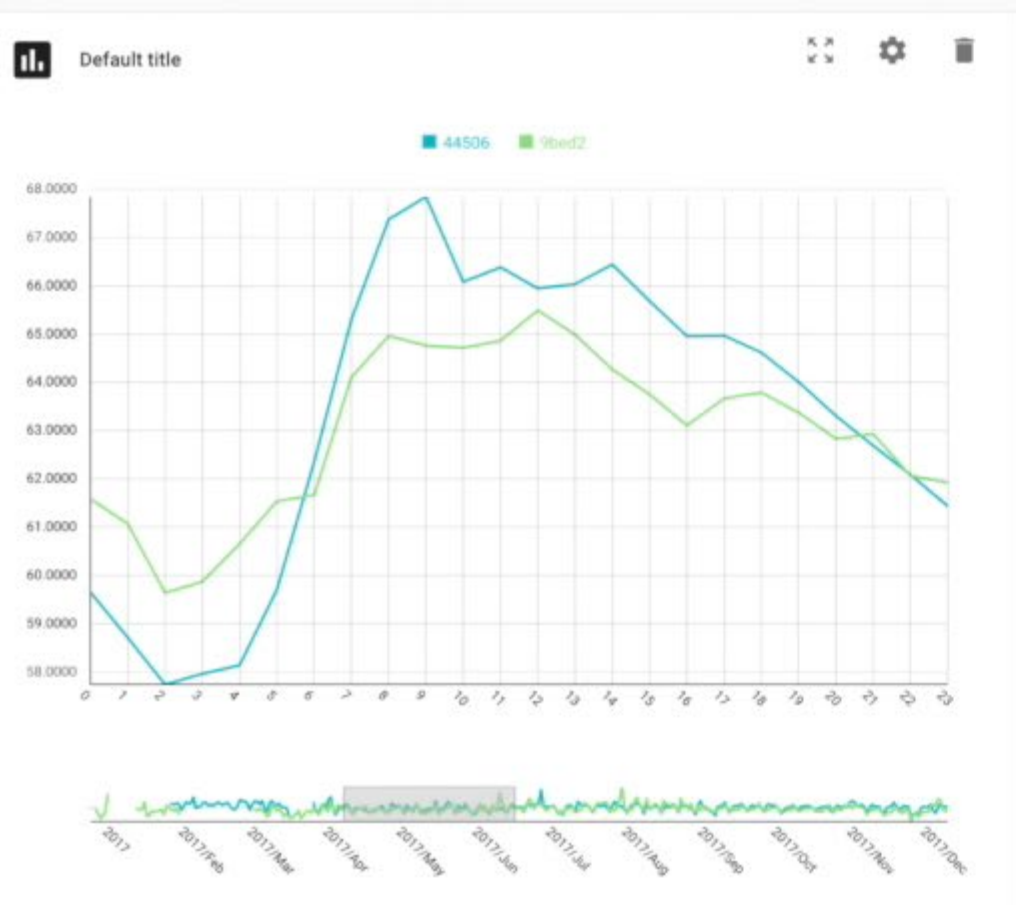
- Support queries having constraints at multiple time resolutions:
 - Average decibel at each hour of the day.
 - Average decibel at day of the week.
 - Average decibel at each day of the week, between 8am – 6pm.
- Support range queries at multiple resolutions:
 - Average decibel between March 1st and March 15th, at hour resolution.
- Support updates from new data
- **Small memory** overhead.
- Allow low latency queries over large time series (**< 500 ms**).

Time Lattice

- Data structure that supports multiple resolution queries at interactive rates.
- Makes use of the implicit hierarchy present in temporal resolutions to materialize a sub-lattice of a data cube.



[Miranda et al., 2019]



Card title	Card width	Card height
Default title	750	650



Start date/time
2016-10-01 00:00

End date/time
2017-12-31 23:59

Constraints (Months)

JAN	FEB	MAR	APR	MAY	JUN
JUL	AUG	SEP	OCT	NOV	DEC

Constraints (Week Days)

MON	TUE	WED	THU	FRI	SAT	SUN
-----	-----	-----	-----	-----	-----	-----

Constraints (Hours)

0	1	2	3	4	5	6	7	8	9	10	11
12	13	14	15	16	17	18	19	20	21	22	23

Group by
Hour

Resolution
Default

Aggregation
Average

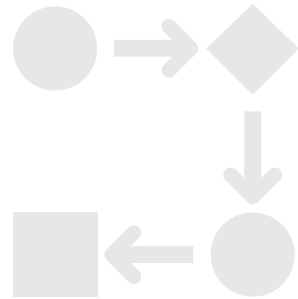
Deviation scale
0

Percentile
0.9

Min value
0

Max Value
10

What is a data pipeline?

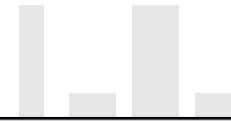


Dataflow



Data management

What is a data pipeline?

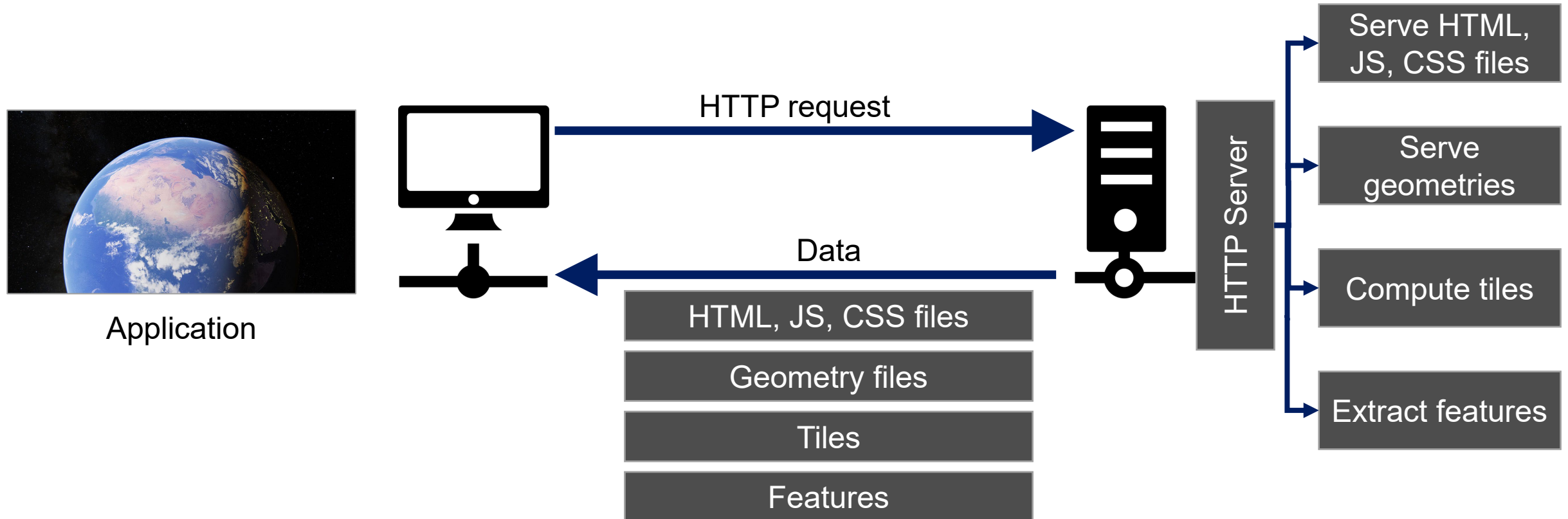


How to operationalize these?

Dataflow

Data management

Client and server



Flask



- Python framework for developing web applications.
- Lightweight applications (when compared to Django).
- Easy integration between front-end and back-end components.

Flask: minimal application

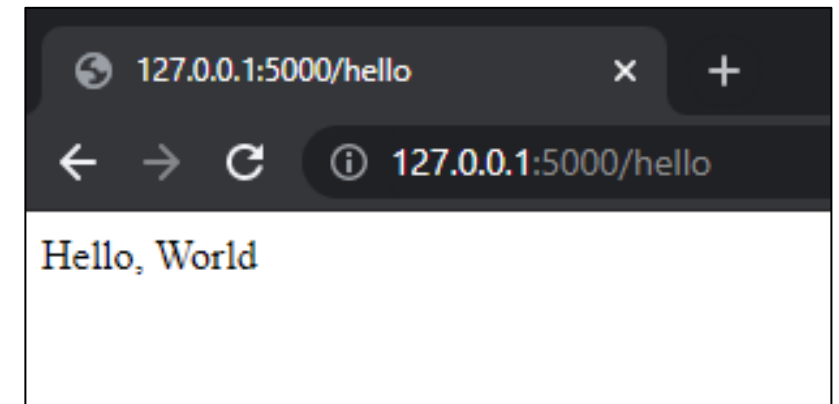
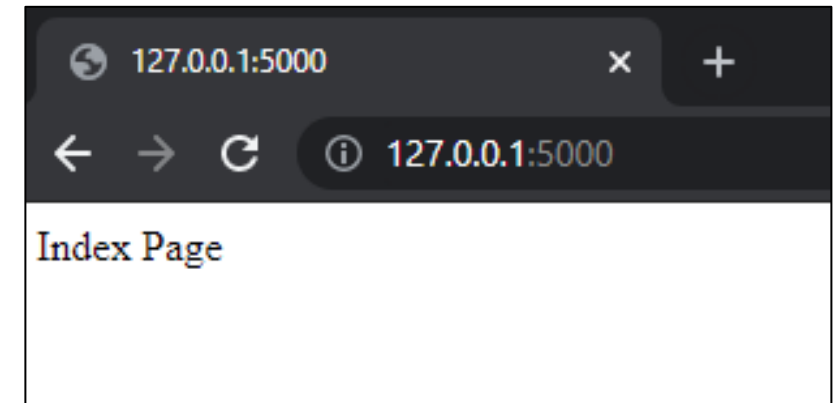
```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def index():
    return 'Index Page'

@app.route('/hello')
def hello():
    return 'Hello, World'
```

```
user@DESKTOP MINGW64 ~/
$ export FLASK_APP=example FLASK_ENV=development
$ flask run
```



Flask: minimal application

- Web applications use different HTTP methods when accessing URLs.
- You can use the methods argument to handle different HTTP methods.

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()
```


HTTP request methods

- HTTP is designed to enable communication between clients and servers.
- HTTP works as a request-response protocol between a client and a server.
- HTTP methods:
 - **GET**
 - **POST**
 - **PUT**
 - **HEAD**
 - **DELETE**
 - **PATCH**
 - **OPTIONS**

HTTP request methods

- GET:
 - Used to request data from a specified resource.
 - One of the most common HTTP methods.

```
/test?name1=value1&name2=value2
```

- POST:
 - Used to send data to a server.
 - Data sent to the server with POST is stored in the **request body** of the HTTP request.

```
POST /test HTTP/1.1
Host: w3schools.com
name1=value1&name2=value2
```

HTTP request methods

	GET	POST
Back button / Reload	Harmless	Data will be re-submitted
Bookmarked	Can be bookmarked	Cannot be bookmarked
Cached	Can be cached	Not cached
History	Parameters remain in browser history	Parameters are not saved in browser history
Restrictions on data length	Length of a URL is limited: 2048 characters	No restrictions
Restrictions on data type	Only ASCII characters	No restrictions. Binary data is also allowed
Security	Less secure, data sent is part of the URL	Safer, parameters are not stored in browser history
Visibility	Data is visible to everyone in the URL	Data is not displayed in the URL

From: https://www.w3schools.com/tags/ref_httpmethods.asp

Flask and HTTP methods

```
from flask import Flask
from flask import request

@app.route('/example/name1=<value1>&name2=<value2>', methods = ['GET', 'POST'])
def example(value1, value2):
    if request.method == 'GET':
        # ...
        pass
    if request.method == 'POST':
        data = request.form # a multidict containing POST data
        # ...
        pass
    else:
        # POST Error 405 Method Not Allowed
        pass
```